Modelly: Fast and Reproducible Agent-Based Modelling in Python

with Calibration and Sensitivity Analysis Built In

Rémi Pelloux
OpenPro
rp@openpro.ai
https://github.com/RemiPelloux/Modelly

Abstract

Agent-based simulations are often too slow for the sweeps a study needs, hard to reproduce, and hard for a reviewer to check. Modelly is a Python framework, written from scratch, whose programming model is compatible with Mesa's. Agents remain objects while their state can live in model-owned arrays (columns) or counted populations (cohorts), so that a model's population-wide phases can be rewritten as array operations one at a time, and the steps of a study (sweeps, calibration, sensitivity analysis, emulators, data assimilation, provenance) share one seeding discipline. On eight workloads written idiomatically for each framework, the column form runs its steps 2.9 to 33 times faster than per-agent Mesa 3.5 code (12.5 times on flocking against Mesa 3.5 given the same KD-tree search) and within 1.8 to 4.7 times of hand-written numpy without agent objects, and an ablation attributes the gain to the array form of the phases. Mesa 3.5 models migrated with a codemod run 1.04 to 2.2 times faster (1.04 to 1.7 on eight of Mesa's own ten examples). Against the author's Mesa 2.3 reference implementations, over 100 paired seeds, 29 of 67 statistic–variant comparisons are shown equivalent within half a standard deviation after Holm correction (63 at nominal significance), and no difference survives Holm correction, and a trajectory-level study finds the faster forms as close to Mesa 2.3 as Mesa 3.5 is on the dynamics tested, with one timing difference shown. On one Linux x86-64 machine, the determinism suite's seventeen examples give bit-identical digests under CPython 3.12, 3.14 and 3.15, compiled or pure; other platforms were not tested. Modelly also splits a run across processes with identical results, runs columns on a GPU, differentiates models, solves equations inside each agent, compares policies on common random numbers, assimilates live data and exports a model to a web page.

Keywords: agent-based modelling; Python; reproducibility; approximate Bayesian computation; global sensitivity analysis; distributed simulation; GPU; data assimilation; benchmarking; scientific software

1 Introduction

Agent-based models (ABMs) represent a system as a population of autonomous agents that follow local rules and interact through a shared environment [69]. Their appeal is that aggregate patterns, such as an epidemic curve, a segregated city [105] or an unequal wealth distribution, emerge from rules a domain expert can state and criticise. Their weakness is the cost of answering questions with them. A single run tells little: a credible study runs many replicates per parameter combination, explores the parameter space, fits the parameters to data, asks which of them matter, and reports all of it in a form others can rerun. Each of these steps multiplies the number of runs, and each is usually assembled by hand around the simulation library.

Python has become a common language for this work, and Mesa [58, 117] its best-known ABM framework. Python makes models easy to write and to connect to the scientific stack [41, 123], but per-agent Python code is slow, and the surrounding tasks (sweeps, calibration, sensitivity analysis, provenance) are left to the modeller or to separate tools [45, 62, 64]. Faster platforms exist in Java and Julia [20, 68, 83] and on GPUs [95]; in Python, mesa-frames trades the per-agent programming model for vectorised speed [3], and Repast4Py distributes a run over MPI processes [19].

Modelly is a new framework, written from scratch, that reduces this cost without asking the modeller to leave Python or to give up agents. Its programming model is compatible with Mesa's, so a Mesa user is at home from the first line and a codemod migrates an existing Mesa model's framework calls. A researcher writes the model as plain Python with agents as objects, rewrites the phases that act on the whole population as array operations where speed matters, one phase at a time, and gets:

- **Speed without leaving Python.** Agent attributes can live in model-owned `numpy` arrays (*columns*), identical agents can be counted rather than instantiated (*cohorts*), and the framework's inner loops run in compiled C, each with a pure-Python twin that gives the same results (Section 3). On eight workloads, with those phases written as arrays, the column form runs 2.9 to 33 times faster than Mesa 3.5 (12.5 times on flocking against Mesa 3.5 given the same KD-tree search) and 4.6 to 42 times faster than Mesa 2.3, and changing the insect model's representation to counted cohorts makes it 131 times faster than individuals on Mesa 3.5, with the lead growing with the population on the grid epidemic and Schelling's model; a Mesa 3.5 model migrated by the codemod, with no phase rewritten, runs 1.04 to 2.2 times faster. An ablation that changes one thing per rung (framework, compiled runtime, data structure, search algorithm, array representation) attributes the large gains to the array representation of the phases (Section 5.3). Whether the faster forms compute the same model is tested: against the author's Mesa 2.3 reference implementations, over 100 seeds per variant, 29 of 67 statistic–variant comparisons of the faster forms are shown equivalent within a prespecified margin of half a standard deviation after Holm correction (63 at nominal significance), and 48 are shown equivalent within ±5% of Mesa's mean; a trajectory-level study of three workloads (whole curves, spatial clusters, time to converge, extinction probability, cycle period) finds the faster forms as close to Mesa 2.3 as Mesa 3.5 is (Section 5).
- **The whole study in one package.** Parameter sweeps, approximate Bayesian calibration, Morris, Sobol and PAWN sensitivity analysis, emulators and history matching, the comparison of interventions on common random numbers, with the variance reduction the pairing brings measured rather than assumed, causal attribution to agents, provenance manifests, verifiable archive bundles, model cards, a model hub that checks hashes and signatures, and ODD descriptions [36, 39] are part of the framework, each a few lines of code, callable from R and Julia as well as Python, and share one seeding discipline, so a study's results do not depend on the executor, the number of workers or the order in which runs finish (Section 4). Each method is checked against problems whose answers are known (Section 6).
- **Results others can rerun.** Seeded generators reproduce the standard library's random stream; a run's manifest records everything needed to repeat it, and `modelly verify` repeats it and names the first output that differs; a determinism suite checks that the bundled examples give the same digests compiled or in pure Python and straight or resumed from a checkpoint, and, on one Linux x86-64 machine under CPython 3.12, 3.14 and 3.15, compiled and pure, that all seventeen examples give bit-identical digests; macOS, Windows, ARM and the published wheels were not tested for this version (Section 6). The paper states each guarantee's scope: determinism on one build, numerical agreement across platforms, and bitwise agreement where it holds.
- **Towards large and data-driven studies.** The same model can be split across processes

with results byte-identical to one process (a check of correctness; its parallel speed-up beyond one laptop is not yet shown), run its array phases on a GPU (steps 28 times faster than one CPU core at ten million agents), be differentiated for gradient calibration, assimilate live observations, couple agents to diffusing fields, carry an ordinary, delay or stochastic differential equation in every agent (each agent taking the steps of its own `solve_ivp` call), be emulated when runs are expensive, start from a synthetic population, and be shared as a web page (tested in Chromium) (Section 7).

What is new. The methods Modelly integrates are established: approximate Bayesian computation, Sobol and Morris sensitivity analysis, emulation, data assimilation and Mesa's programming model are all prior work (Section 2). The contribution of this paper is (i) the objects/columns/cohorts architecture, in which the three levels of storage share one programming model and a model moves between them one phase at a time, with a stated proposition, and counterexamples, for when a counted population keeps the individual model's law (Section 3); (ii) the integration of model execution, deterministic experiment scheduling and verifiable artefacts under one seeding discipline, so that a calibration or a sensitivity analysis is reproducible, resumable and executor-independent without extra code (Section 4); (iii) a Mesa-compatible interface with a measured migration path, on Mesa's own examples (Section 5.1); and (iv) tested execution contracts (compiled and pure cores, split runs, GPU backend) whose guarantees are stated by scope and checked on one machine under three interpreters (Sections 6 and 7). The evidence for (i) is the main empirical work of the paper: the speed-ups, the ablation that attributes them, and the equivalence and trajectory studies that test whether the faster forms are the same model.

The rest of the paper situates Modelly among existing tools (Section 2), describes its design (Section 3) and research workflow (Section 4), evaluates its speed and its agreement with Mesa (Section 5) and the correctness of its methods (Section 6), measures how it scales out and connects to data (Section 7), sets out the studies it serves (Section 8), and ends with the threats to the validity of these measurements and the software's limitations (Section 9).

2 Related work

General-purpose ABM platforms. NetLogo [127] is widely used in teaching, with its own language and an integrated interface; it excels at exploratory models, and a Python analysis reaches it through a bridge such as PyNetLogo [54]; GAMA [114] specialises in spatial models. Repast Simphony [83] and MASON [68] are mature Java platforms built for performance and large simulations, and Repast4Py [19] distributes a single run over MPI processes from Python, with scaling reported on a cluster; Modelly also splits a run, under a contract that makes the split run reproduce the single-process one byte for byte (Section 7.1), but its parallel speed-up has been measured only on one laptop, and runs column phases on a GPU through CuPy or JAX rather than through a dedicated language (Section 7.2). Agents.jl [20] brings a performant, concise framework to Julia, and FLAME GPU [95] runs agent models on GPUs. Reviews of the field [1, 92] have long noted the trade-off between ease of use and speed that these tools resolve in different ways.

Python. Mesa [58, 73, 117] provides the agent, model, space and data-collection abstractions that most Python ABMs use, and a browser visualisation. AgentPy [33] combines a similar model with experiment and Sobol tooling built on SALib [45], though its last release (0.1.5) dates from December 2021; Melodie [130] adds calibration and scenario management with Cython acceleration; and mesa-frames [3] represents agent sets as data frames to vectorise Mesa models. Modelly shares no code with Mesa, but its programming model is compatible with Mesa's: its codemod (`python -m modelly.migrate`) performs the mechanical rewrites of a Mesa model and

reports what needs a decision (schedulers, legacy grids, batch runs); the two Mesa 2.3 models migrated this way, changing only the framework calls, give bit-identical results on every seed tried, and the migrated Mesa 3.5 models equivalent distributions (Section 5.1). Unlike a dataframe representation, Modelly's columns keep the per-agent object: each agent reads and writes its row as an attribute, so a model can be vectorised one phase at a time (Section 3.2).

Calibration, sensitivity analysis and model exploration. The methods Modelly implements are standard: approximate Bayesian computation [10], with sequential Monte Carlo [11, 109, 120], already applied to agent-based models [90, 121], elementary effects [79], variance-based indices [52, 101, 113] and the PAWN density-based index [88, 89]. Libraries implement them for generic simulators [45, 50, 62, 103], and workflow tools connect them to ABM platforms: the EMA Workbench [64], OpenMOLE [93] and nlrx for NetLogo [97]; reviews and tutorials give guidance on which to use [43, 66, 116, 119]. The same holds for history matching with emulators [5, 122], the bootstrap particle filter [34] and the ensemble Kalman filter [31], and gradient estimators for discrete choices [12, 51], and for the robust comparison of policies across sampled futures, by minimax regret [102], which the EMA Workbench also offers [64]. Integrators for ordinary and stochastic differential equations are standard [63, 123]; Agents.jl documents coupling its agents to DifferentialEquations.jl [20]. Data assimilation has been applied to agent-based models with particle filters, for calibration on line and for crowds in real time [72, 125]; differentiable agent-based models have been built for simulation-based optimisation and for epidemiology [4, 17], with tooling for gradient-based Bayesian inference on them [91]; and black-box Bayesian inference for agent-based models, including neural approaches, has been compared on economic models [28]. Modelly's contribution here is integration rather than new methods: the samplers share the simulation's seeding and executors, so a calibration or a Sobol analysis of a model is reproducible, resumable and parallel without extra code.

Documentation and reproducibility. The ODD protocol [36, 37, 39] and its extension to human decisions, ODD+D [80], are the standard description of an ABM in a publication, TRACE [38] documents its testing and analysis, and pattern-oriented modelling [35] guides its validation. Archiving model code remains uncommon [53], and replicating a published model is a study in itself [7, 128]. Modelly generates an ODD skeleton from the model's code and records each run in a manifest that a reviewer can rerun and verify. Model cards, short structured statements of a model's intended use, data, evaluation and limits, come from machine learning [78]; Modelly's cards tie each validation claim to a test and are reported stale when the model changes, and its model hub checks an archive's hash and signature before installing it, where model libraries for agent-based models archive code with documentation [53].

Table 1 compares the built-in capabilities of the Python frameworks and of two widely used platforms in other languages.

Table 1. Capabilities built in or documented by each project in October 2026 (Mesa 3.5.1, mesa-frames at development commit `ef92326`, version 0.1.1.dev0, as measured in Section 5.2 (its only release is the alpha 0.1.0a0), AgentPy 0.1.5, NetLogo 7, Repast4Py 1.3, Agents.jl 7.1). ✓: built in; ext.: through a companion tool that the project or the tool documents for it (EMA Workbench for Mesa, ipysimulate for AgentPy, pyNetLogo with SALib for NetLogo [54], EMEWS [87] for Repast4Py); –: not documented. Generic Python libraries such as SALib and pyabc can be called from any of the Python frameworks; the table records integration, not possibility (nlrx's ABC calibration supports NetLogo only up to 6.4, and EMEWS documents no sensitivity analysis or calibration for Repast4Py). The judgements are the author's, from each project's documentation and source; the evidence for every cell is in `paper/data/features_sources.md`. NetLogo's checkpoints are `export-world` and `import-world`; Agents.jl's do not cover its event-queue models. Parallel runs in Mesa, AgentPy, NetLogo and Agents.jl are across runs, not one run. Agents.jl's differential equations come from DifferentialEquations.jl, in a documented example; NetLogo's System Dynamics Modeler integrates stocks of the whole model, not one system per agent. Licences from the projects' licence files (mesa-frames' development branch; its 0.1.0a0 release is MIT).

	Modelly	Mesa 3	mesa-frames	AgentPy	NetLogo	Repast4Py	Agents.jl
Language	Python	Python	Python	Python	NetLogo	Python	Julia
First release	2026	2014	2024	2020	1999	2021	2018
Licence	Apache-2.0	Apache-2.0	Apache-2.0	BSD-3	GPL-2.0+	BSD-3	MIT
Per-agent objects	✓	✓	–	✓	✓	✓	✓
Array or table agent state	✓	–	✓	–	–	–	–
Counted cohorts	✓	–	–	–	–	–	–
One run across processes	✓	–	–	–	–	✓	–
shown at cluster scale	–	–	–	–	–	✓	–
Parameter sweeps	✓	✓	–	✓	✓	ext.	✓
Global sensitivity analysis	✓	ext.	–	✓	ext.	–	–
Bayesian calibration	✓	–	–	–	–	–	–
Checkpoint and resume	✓	–	–	–	✓	✓	✓
Run manifests and bundles	✓	–	–	–	–	–	–
Interactive visualisation	✓	✓	–	ext.	✓	–	✓
Equations in each agent	✓	–	–	–	–	–	ext.
Graphical model editor	–	–	–	–	✓	–	–

3 Design

3.1 The programming model

Modelly is an independent implementation: it contains no Mesa code and does not depend on Mesa. Its compatibility with Mesa is at the level of the programming model and the API's names, which is what lets a codemod migrate a Mesa model (Section 5.1). A Modelly model is a `Model` subclass that owns its agents, its spaces, its clock and its random generators; an agent is an `Agent` subclass with a `step` method or whatever behaviour the model calls. Parameters live in a typed `Scenario` that also carries the seed, so that a model's whole input is one object that can be stored, swept and hashed. Agents are kept in ordered `AgentSet`s with `do`, `shuffle_do`, `select`, `agg` and `groupby`; the model's registries are backed by dictionaries, so membership tests run in C and iteration order is deterministic. Time advances in ticks, through scheduled events, or both on one heap with lazy cancellation.

3.2 Three ways to store a population

The same model can be written at three levels of the speed/expressiveness trade-off, and Modelly supports all three in one framework:

Objects (the per-agent style of Mesa).
Each agent is a Python object with its own attributes, and each cell keeps the list of its occupants. This is the most direct translation of a model's description and the easiest to

read.

Columns.
An attribute declared as a `Column` is stored in a model-owned `numpy` array, one table per agent class. Each agent still reads and writes it as an attribute (a property reading its row), while the whole population is available as one array for vectorised phases (Listing 1). Phases that act on all agents at once (movement, infection, ageing, mortality) become array operations.

Cohorts.
When many agents are identical in every state variable that matters (stage, age, sex), a `CohortTable` stores one row per class with a count, and stochastic events become binomial draws on the counts (below), the super-individual approach of individual-based ecology [104]. A population of tens of thousands then costs at most about 190 rows in our runs.

When cohorts keep the individual model's law. The claim is conditional, and we state its conditions. Let a row hold n individuals in state x, and let each face, independently, the same probability $s(x)$ of surviving a phase. In the individual model the survivors are $\sum_{k=1}^{n} B_k$ with $B_k \sim \text{Bernoulli}(s(x))$ independent, which is $\text{Binomial}(n, s(x))$; the cohort draws that count directly:

$$\begin{aligned} &\text{survival } (\texttt{thin}): && n' \sim \text{Binomial}\big(n, s(x)\big), \\ &\text{transition } (\texttt{split}): && K \sim \text{Binomial}\big(n, p(x)\big) \text{ move to state } x', \text{ and } n-K \text{ stay}, \\ &\text{events } (\texttt{draw}): && E \sim \text{Binomial}\big(n, q(x)\big). \end{aligned} \tag{1}$$

The three operations cover survival, two-way transitions and counts of events that each individual triggers at most once per phase (an egg batch, an emergence).

Proposition.
Suppose that, in every phase, (A1) all the individuals of the population, within a row and across rows, are conditionally independent given the state vector at the start of the phase; (A2) every individual of a row faces the same probabilities, which are functions of that state vector only (homogeneity within the row and a synchronous update); and (A3) each individual's fate in the phase is one of the three operations above (one draw per individual, into at most two outcomes). Then, by induction over the phases, the vector of counts per state has the same distribution in the cohort model as in the individual model.

The proof is the binomial identity above applied row by row. Conditional on the state vector at the start of the phase, (A1) makes the fates of all individuals independent, so the fates of distinct rows are independent of one another and, within a row, (A2) and (A3) make them exchangeable Bernoulli variables with the row's probability: each row's count is therefore binomial, the rows' counts are conditionally independent, and their joint law given the state is the one the cohort model draws from (one independent binomial per row, with the same probabilities, since the cohort evaluates them from the same state vector). The induction over phases carries the equality of the joint law of the state vector from one phase to the next. Independence across rows is part of (A1), not a consequence of independence within rows: a phase in which two rows' fates are coupled (a shared resource drawn once and allocated to both, say) breaks (A1) even when each row's members are independent among themselves. The proposition says nothing when an assumption fails, and two cases show that the conclusion then fails too (20,000 replicates of one phase on $n = 200$ individuals with $s = 0.8$, by `paper/scripts/cohort_counterexample.py`, whose data file is next to it). First, an asynchronous update breaks (A2): when each individual's survival probability depends on the count alive at its own turn (rising by 0.002 per earlier death), the individual model keeps 167.0 survivors on average and the cohort, which evaluates the rate

Listing 1. A wealth-exchange model whose agents keep their wealth in a column; the step acts on the whole population at once. Run as shown with Modelly 1.5.0.

```
import numpy as np
from modelly import Agent, Column, Model
from modelly.scenarios import Scenario

class Economy(Scenario):
    n: int = 1000

class Trader(Agent):
    wealth = Column(int, default=1)

class Exchange(Model):
    def __init__(self, scenario: Economy | None = None):
        scenario = scenario or Economy()
        super().__init__(scenario=scenario)
        Trader.create_agents(self, scenario.n)

    def step(self):
        wealth = self.columns[Trader].wealth   # the whole population, one array
        givers = np.flatnonzero(wealth > 0)
        takers = self.rng.integers(0, wealth.size, size=givers.size)
        np.subtract.at(wealth, givers, 1)
        np.add.at(wealth, takers, 1)

model = Exchange(Economy(n=1000, rng=42))
model.run_for(100)       # same seed, same run
```

once at the start of the phase, 160.0, a difference of 7.1 (standard error 0.05). Second, hidden heterogeneity breaks (A2) in another way: when the individuals of a row carry a variable the row does not (each its own probability, uniform on 0.8 ± 0.2), the survivors are Poisson-binomial, with the same mean as the cohort's binomial (161.0 against 160.9) but a smaller variance (28.8 against 31.3; exact values 28.8 and 31.4; median-centred Levene $p = 10^{-6}$). Interactions within a phase between members of the same row (a pairwise contest, say) break (A1) in the same way. In the positive case, where the assumptions hold, the two distributions coincide (means 160.0 and 160.0, variances 31.9 and 32.2 for an exact 32.0, Kolmogorov–Smirnov $p = 0.997$). The practical rule is therefore: a cohort model reproduces the individual model that updates synchronously with rates evaluated once per phase, as the Mesa implementations of the insect playbook do (Section 5.4 checks the insect model's statistics), and any state variable that changes an individual's probabilities must be part of the row's key. Individual identities, and histories that differ within a state, are not represented; a model that needs them stays at the object or column level.

The columns view and the per-agent attribute are the same memory, so a model can mix per-agent code and vectorised phases freely: the benchmarks in Section 5 include variants of the same model at each level.

3.3 Seeded randomness

Every model owns two generators: `model.random`, a `FastRandom` that produces exactly the numbers of Python's `random.Random` for the same seed (the Mersenne Twister [75]), and `model.rng`, a `numpy Generator`. `FastRandom`'s hot methods (`randint`, `choice`, `shuffle`, `sample`) run in C on CPython's own generator state; at import it is checked against `random.Random`, and a pure-Python implementation with the same stream is used when the C module is absent. A model that seeds its own `random.Random(seed)` keeps that exact stream, which is what makes the identical results of the two migrated Mesa 2.3 models possible; `model.random` itself is seeded

from a value derived from the scenario's seed, so a migrated model that draws from it follows a different stream from Mesa's (Section 5.1). A model can instead give every agent generators of its own (`agent_streams = True`): each is seeded from the scenario's seed sequence and the agent's stream key, so what an agent draws no longer depends on the activation order or on the population, which is what lets a run split across workers reproduce the single-process run (Section 7.1). The flag is off by default; turning it on changes a model's numbers.

3.4 Compiled loops with pure-Python twins

Five small C modules implement the framework's inner loops: the random core; per-agent method calls (`do`, `shuffle_do`) and the breadth-first neighbourhood walk; the cell move; set filtering and attribute gathering (`select`, `agg`); and the event list's heap (Appendix G.4). Each C function is a line-for-line translation of a Modelly Python function that stays in the source as its fallback; tests run the two side by side, and `MODELLY_PURE_PYTHON=1` forces the Python path. Results are identical either way. The modules build with GCC, Clang and MSVC; the release workflow builds them on the tag for Linux (glibc and musl, x86-64 and ARM64), macOS and Windows (x86-64 and ARM64), where a module that fails to compile fails the build and the release workflow's import test checks that every wheel holds all five compiled modules. They also build for free-threaded CPython, where critical sections and cache generation counters replace the global interpreter lock, and the compiled modules' tests, except the event heap's, run on that build on the development machine.

3.5 Spaces

Discrete spaces (orthogonal Moore and von Neumann grids, hexagonal grids, networks, Voronoi meshes) share one cell abstraction with cached neighbourhoods, property layers backed by arrays and an empty-cell index; a `CellIndex` exposes any discrete space as integer rows and a sparse adjacency for vectorised counts and random-neighbour draws. Continuous space keeps positions in one array and answers neighbour queries with one KD-tree per step. A geographic space, comparable in scope to Mesa-Geo [124], projects coordinates once and supports regions, contiguity and road networks, and temporal and multilayer networks represent contact structures that change over time.

3.6 State that survives a pause

`model.save(path)` writes a checkpoint holding the whole model, both generators and the event scheduler; a run resumed from it, in a new process, is the run it would have been. Agents are written flat, so the cost of a checkpoint does not grow with the depth of the agent–cell graph. Checkpoints are the basis of time travel, resumable experiments and calibrations, and Studio's timeline (Section 4).

4 From a model to a published study

Modelly includes the steps between a working model and a published study. They share the framework's seeding: each run's seed is derived deterministically from the study's seed and the run's position in the design, so results never depend on the executor, the number of workers or the order in which runs finish.

4.1 Experiments

An `Experiment` combines a model, a `ScenarioSpace` (ranges, log and integer ranges, choices, fixed values), a design (full factorial, Latin hypercube [77], scrambled Sobol [74, 86, 96, 112]

Listing 2. A Latin-hypercube sweep over two parameters, three replicates per point, run on four processes; results are queried with SQL.

```
from modelly.examples import BoltzmannWealth
from modelly.experiments import Experiment, ScenarioSpace, designs

experiment = Experiment(
    model=BoltzmannWealth,
    space=ScenarioSpace(n=(50, 200), width=[10, 20], height=10),
    design=designs.latin_hypercube(8),
    replicates=3, steps=20, outputs=["gini"],
    store="runs/boltzmann", seed=42,
)
if __name__ == "__main__":   # process workers re-import the script
    experiment.run(executor="processes", max_parallel=4)
    sql = "SELECT width, avg(gini) FROM runs GROUP BY width"
    print(experiment.results().query(sql).df())
```

Listing 3. ABC-SMC calibration of two rates of an epidemic model to an observed series. `epidemic` is the model module of `docs/calibration.md`, the documented SIR model with its observed series `cases`; run as printed against that module, the listing gives the posterior quoted in Section 6.2 (`paper/scripts/check_listings.py`).

```
from modelly.calibrate import ABCSMC, Prior, Target
from epidemic import Epidemic, cases   # the model and the observed series

calibration = ABCSMC(
    model=Epidemic,
    priors={"beta": Prior.uniform(0.05, 1.0), "gamma": Prior.uniform(0.02, 0.5)},
    targets=[Target("infected", observed=cases)],
    steps=40, particles=100, seed=2027,
)
if __name__ == "__main__":
    posterior = calibration.run(generations=6, executor="processes")
    print(posterior.summary())   # means, medians, highest-density intervals
```

or Halton [40] sequences, or a custom list), a number of replicates and the outputs to record (Listing 2).

Results are appended to Parquet files with a DuckDB view; an interrupted sweep resumes without repeating finished runs. Executors cover serial runs, threads, processes, Dask, Ray and SLURM or PBS job arrays, and are pluggable. The test suite checks that serial, thread and process runs of the same experiment give identical results, the SLURM and PBS executors against stand-in `sbatch` and `qsub` commands (no cluster was available), and Dask runs when Dask is installed; the Ray executor is tested against a stand-in for its scheduler.

4.2 Calibration and sensitivity analysis

`modelly.calibrate` fits scenario parameters to observed data with ABC rejection [10], ABC-SMC, the simulated method of moments [76] and a Gaussian-process Bayesian optimiser [55], through the experiment executors; a stopped calibration resumes to the same posterior. `modelly.sensitivity` computes one-at-a-time effects, Morris elementary effects [79], Sobol indices and PAWN [89], on a model, a function or a Gaussian-process emulator [84]. Both are checked against known answers (Section 6.2); Listing 3 calibrates an epidemic model.

ABC-SMC. Let $\rho(\theta) = \sum_k w_k \, d_k(\bar{y}_k(\theta), y_k^{\mathrm{obs}})$ be the weighted distance between the simulated summaries (averaged over replicates) and the data, with the root-mean-square error as the

default d_k. Generation 0 draws N particles from the prior π with weights $1/N$. Generation $t \geq 1$ sets its tolerance to a quantile of the previous generation's distances,

$$\varepsilon_t = \max\big(Q_\alpha\{\rho_i^{(t-1)}\},\ \varepsilon_{\min}\big), \qquad \alpha = 0.5 \text{ by default}, \tag{2}$$

and proposes $\theta^* = \theta_j^{(t-1)} + \eta$, with the ancestor j drawn with probability $w_j^{(t-1)}$ and $\eta \sim \mathcal{N}(0, \Sigma_t)$, $\Sigma_t = 2\,\mathrm{Cov}_w(\theta^{(t-1)})$, twice the weighted (biased) covariance [11]; Modelly adds $10^{-9}\max(\bar{s}^2, 1)$ to its diagonal, $\bar{s}^2$ being the mean variance, to keep it invertible. A proposal outside the prior's support is discarded and both the ancestor and the move are drawn again, so the proposal density is the mixture truncated to the support; Sisson et al. [110] review the method. Choice parameters keep their value with probability 0.7 and otherwise jump uniformly to another value. A particle is accepted when $\rho(\theta^*) \leq \varepsilon_t$, and the accepted particles are weighted by

$$w_i^{(t)} \propto \frac{\pi(\theta_i^{(t)})}{\sum_j w_j^{(t-1)}\, K_t(\theta_i^{(t)} \mid \theta_j^{(t-1)})} \tag{3}$$

[120], where K_t is the kernel's density; the truncation's normalising constant is the same for every particle and cancels when the weights are normalised. That cancellation holds because the mixture is truncated as a whole (both the ancestor and the move are redrawn), not each ancestor's kernel on its own, which would leave a constant per ancestor in the denominator; Section 6.2 checks that the code does the former. The run stops after the requested generations, at $\varepsilon_{\min}$, when the acceptance rate falls below 1%, or before exceeding a budget of model runs. The posterior is summarised by weighted means and medians and by the shortest interval holding 95% of the weight, and its effective sample size is Kish's $1/\sum_i w_i^2$ [61]. Proposals are made in batches whose random streams are keyed by generation and batch, which makes the posterior independent of the executor and of a resume.

Sobol indices. For a model output $Y = f(X_1, \ldots, X_d)$ with independent inputs, the first-order and total indices are [47, 113]

$$S_i = \frac{\mathrm{Var}(\mathbb{E}[Y \mid X_i])}{\mathrm{Var}(Y)}, \qquad S_{Ti} = \frac{\mathbb{E}[\mathrm{Var}(Y \mid X_{\sim i})]}{\mathrm{Var}(Y)}. \tag{4}$$

Modelly draws A and B, two $N \times d$ matrices, from one scrambled Sobol sequence of dimension $2d$ [99, 100], forms $A_B^{(i)}$ as A with its i-th column taken from B, and evaluates the model $N(d+2)$ times. With V the variance of the $2N$ values of $f(A)$ and $f(B)$, the default estimators are Jansen's [52, 101]:

$$\hat{S}_i = \frac{V - \frac{1}{2N}\sum_{j=1}^{N}\big(f(B)_j - f(A_B^{(i)})_j\big)^2}{V}, \qquad \hat{S}_{Ti} = \frac{\frac{1}{2N}\sum_{j=1}^{N}\big(f(A)_j - f(A_B^{(i)})_j\big)^2}{V}, \tag{5}$$

with Saltelli's estimators as an option, and 95% intervals from 200 bootstrap resamples of the rows. For a stochastic model, f is the mean of the chosen statistic over a number of replicates per design point (one by default); the replicates' noise then adds to the variance the indices apportion, which inflates S_{Ti}, so a study of a noisy model should average replicates or account for the noise explicitly [42], as reviews of sensitivity analysis for agent-based models advise [116].

4.3 Decisions and causes

A study often ends in a choice between interventions, and a reader asks why the model chose it. `modelly.decide` runs named strategies (parameter values and a cost) on common random numbers: replicate r of every strategy gets the same seed, so the strategies meet the same chance events and their differences carry less noise. It reports each strategy's mean with an interval, the

paired differences, and the share of replicates in which each strategy is best; cost-effectiveness with dominance, extended dominance and incremental ratios, and net benefit; a ranking across futures sampled by Latin hypercube, by minimax regret [102] with the share of futures in which each strategy is satisfactory; and a seeded search of a policy's parameters with Optuna [2]. The bundled study gives 10,000 vaccine doses to the children, the adults or the seniors of the age-structured SEIR zoo model, or to nobody; over 20 paired replicates, vaccinating the seniors gives the fewest deaths (242, 95% interval 241 to 243, against 422 with no vaccine) in every replicate, and a second run gives an identical table. The fatality ratios are illustrative, which the model's card states.

`modelly.explain` answers the second question. `treatment_effect` estimates the effect of a change from paired replicates and measures what the pairing bought: the variance of the difference against independent seeds divided by its variance on common ones. On the Bass zoo model, raising imitation from 0.38 to 0.40 adds 2.05 percentage points of adoption after 60 steps (1.92 to 2.18, 30 pairs), and the pairing divides the variance by 24 to 62 over ten seeds of the study (median 42); for a change to 0.45, by 10 to 19. The gain is a property of the model, not of the method: vaccinating a group of the SEIR model adds a cohort row, which reorders the binomial draws from the first step, and the ratio there is 0.6 to 1.3, no gain (`paper/data/decisions.json`). The ratio is therefore measured for each study rather than assumed. `attribute` reruns a model once per candidate agent with that agent changed; in a sixty-person outbreak on a contact network it ranks the index case first on five seeds of five, moving the outbreak by 45 to 51 infections against 4 to 10 for the next agent. `counterfactual` continues a recorded run from a checkpoint with a change applied, and `shapley` computes exact Shapley values [107] of an emulator's prediction.

4.4 Provenance and publication

`modelly run` writes a run's outputs with a manifest: code hash, Modelly and Python versions, compiled or pure build, platform, scenario, seeds and hashes of inputs and outputs. `modelly verify` reruns a manifest and names the first output that differs; `modelly bundle` packs code, dependency lock, seeds and outputs into a byte-identical archive with Zenodo metadata; `modelly odd` writes the skeleton of an ODD description [39] from the model's classes, columns, scenario and schedule.

A model card [78], `card.toml` next to the model, states its intended and out-of-scope uses, its data, its known limits and its validation, each claim tied to a test; `modelly card stamp` runs those tests and records a digest of the model's source, and `modelly card check` reports a card that is incomplete or stale because the model changed after it was stamped. Every zoo model has a stamped card that the test suite checks. `modelly hub` shares models through a static index whose entries carry the model card, the Modelly versions the model supports, the SHA-256 of its source archive and, optionally, a Zenodo DOI and a Sigstore signature; `modelly hub install` checks the version, the hash and the signature before anything is installed. The eighteen zoo models are on the public index.

4.5 Tools for building and checking models

The package also includes, tested but not evaluated in this paper, a linter for reproducibility bugs and quadratic patterns (`modelly lint`), a profiler (`modelly profile`), time travel over checkpoints (rewind, replay with changed parameters, branch, and find the first step where two runs diverge), and a browser front end, Studio (`modelly studio`), with a timeline over a run's checkpoints, a tab that compares scenarios on paired replicates, a form that writes a model, HTML and PDF reports, a classroom mode, and its labels in five languages, laid out right to left for Arabic, Hebrew, Persian and Urdu. Studio and the documentation pass the WCAG 2.2 level A and AA rules of axe-core on the pages audited. The package also has language-model

agents with episodic memory, tool calls and conversations, whose cached replies replay offline (`modelly.llm`); a translator for a documented subset of NetLogo (`modelly netlogo`); and R and Julia packages that run, inspect and calibrate a model from those languages through the same Python functions, the agents' columns reaching Julia as arrays without a copy. Run from R and from Julia, the documentation's SIR calibration gives the posterior means of the Python run (0.423 and 0.098 for the two rates, to the three digits the tests compare), and the release workflow ran both packages' tests on Linux, macOS and Windows before this version was published.

4.6 A zoo of tested reference models

Eighteen reference models, each with a validation test against an analytical or well-known result, serve as starting points and as test cases: an age-structured susceptible–exposed–infected–recovered (SEIR) model checked against the final-size relation [59], Nagel–Schreckenberg traffic [81] against its flow–density relation, Bass diffusion [9] against the Bass equation, the Deffuant [21] and Hegselmann–Krause [44] bounded-confidence models, Axelrod's culture model [6], a kinetic wealth-exchange model [26] and the Bouchaud–Mézard model [14] against their stationary distributions, a zero-intelligence continuous double auction [32], a Drossel–Schwabl forest fire [27] around the percolation threshold [82], an energy-budget population [108] around its carrying capacity, and Ross–Macdonald vector-borne transmission [111] ordered by R_0; and six that couple agents to equations, fields and data: an epidemic between hosts whose infections are viral ODEs inside each host, checked against one `solve_ivp` call per host and an analytical clearance time, foragers on a diffusing nutrient field and an epidemic coupled to hospital capacity, and traffic assignment, habitat selection and Schelling segregation on real street, land-cover and census-tract samples (Section 7).

5 Performance

A framework can be fast in two senses: the same model code runs faster on it, or it lets a modeller write the same model in a faster form. We measure both, with methods chosen to make the comparison fair and repeatable, and check separately that the faster forms compute the same model. All the comparisons with Mesa are CPU time (`time.process_time`), measured in-process, pinned to one core with `taskset`, with the implementations alternating, on an Intel Core Ultra 7 356H laptop running Linux, CPython 3.15.2 (3.15.0 for mesa-frames) and numpy 2.5.4 in each implementation's environment, against Mesa 2.3.2 and Mesa 3.5.1, with Modelly at its tagged release v1.5.0; Table 3 comes from the second of two timing sessions, both run on the tag's `modelly` package (Appendix A). The core was held at its 1.9 GHz base frequency (the `performance` governor, turbo disabled) for every timing of this section. Following Kalibera and Jones [56], every timed run of the playbooks, the migrated models and Mesa's examples is a fresh interpreter (the GPU arms are fresh interpreters after one warm-up run), the rounds are paired and their order varies, and we report medians with their spread and bootstrap intervals rather than single runs.

Speed-up. For a workload with CPU times T_r over rounds $r = 1, \ldots, R$, we report

$$S_{\text{steps}} = \frac{\operatorname{median}_r T^{\text{Mesa}}_{\text{steps},r}}{\operatorname{median}_r T^{\text{Modelly}}_{\text{steps},r}}, \qquad S_{\text{total}} = \frac{\operatorname{median}_r T^{\text{Mesa}}_{\text{build},r} + \operatorname{median}_r T^{\text{Mesa}}_{\text{steps},r}}{\operatorname{median}_r T^{\text{Modelly}}_{\text{build},r} + \operatorname{median}_r T^{\text{Modelly}}_{\text{steps},r}}, \tag{6}$$

where *steps* is the simulation and *build* the construction of the model and its agents. S_{steps} is the gain per simulated step, which dominates long runs; S_{total} is the gain for one run of the benchmark's length. Round r runs every implementation on seed r, so the per-round ratios are paired. Where runs are paired chunk by chunk or round by round (Sections 5.1 and 7.2

and Table 2) we report instead the median of the per-round ratios with its range or quartiles; on the playbooks the two estimators differ by at most 1.9%.

5.1 The same model code on both frameworks

Modelly shares no code with Mesa; what the two share is a programming model, so a model written for Mesa can run on Modelly once its calls to the framework are rewritten. Modelly's codemod (`python -m modelly.migrate`) rewrites a Mesa model's imports, its agent constructors and the old scheduler calls (`schedule.agents`, `schedule.steps`, `schedule.step()`), leaving every line of model logic unchanged, and reports the steps it cannot make (the seeding keyword, `unique_id` parameters, schedulers, legacy grids). The model code is then the same on both frameworks, and the difference in time is the difference between the frameworks' own calls. We measured it in three settings, beginning with models we did not write.

Mesa's own examples. Mesa 3.5 ships ten example models written by its developers. We migrated copies of them with the codemod, kept in `benchmarks/mesa_examples/`: eight run on Modelly after the codemod and one more edit to each, the import of the example's own agents made relative (the codemod points it at Modelly's bundled example of the same name; the relative import makes each copy load the agents module next to it), plus, for the Epstein civil-violence model, the one step its report asks for (`self.steps` becomes `self.time`); Sugarscape G1MT uses Mesa's property layers and alliance formation its meta-agent factory, which Modelly lacks. The codemod maps Mesa 3.5's module paths, the experimental cell space, continuous space, meta-agents, signals, scenarios and data collection among them, and reports the other `mesa.experimental` modules and the property-layer module, which Modelly lacks; the timed code does not import the codemod. We ran each original on Mesa and its migrated copy on Modelly at Mesa's default parameters, except four whose defaults run too briefly to time reliably: Boltzmann wealth with 2,500 agents on a 50×50 grid, Schelling's model on 50×50, wolf–sheep on 50×50 with 625 sheep and 312 wolves, and the virus model on 1,000 nodes (`benchmarks/mesa_examples/run.py`), each for ten alternating rounds on the same fixed-frequency core, and compared every model reporter, at the last step and averaged over the run, over 100 shared seeds. Table 2 gives the results: on Modelly the examples take 0.60 (wolf–sheep) to 0.96 (boid flockers) of Mesa 3.5's CPU time, and all 35 statistics (34 once the virus model's final number infected, zero in every run, is set aside) are equivalent within $0.5\,s_p$, with no paired t-test or Kolmogorov–Smirnov test below 5% even before correction. The virus model's other seven statistics leaned the same way ($|d| = 0.22$ to 0.25, paired $p \geq 0.10$), and we traced it (`paper/scripts/virus_trace.py`, `data/virus_trace.json`, 10 seeds): the graph, the agents' order and neighbourhoods and the data collection are identical on both arms, and only the initial outbreak differs, because Mesa seeds `model.random` with the seed itself and Modelly with a 32-bit value derived from it. With Mesa's derivation forced on both arms the whole runs are bit-identical on every seed: the migrated copy is the same stochastic process, draw for draw, and the agreement study compares two independent streams of it. Exact, run-for-run reproduction of a Mesa model therefore requires aligned random streams: by default a migrated model draws `model.random` from Modelly's seed derivation and agrees with Mesa in distribution, not run by run. A replication on 1,000 new seeds (20,000 to 20,999, `paper/data/mesa_examples_virus1000_new.json`) gives every $|d| \leq 0.016$ (paired $p \geq 0.73$). Here, as for the grid epidemic's peak time and Schelling's model in Section 5.4, a replication on new seeds was run where a consistent pattern or an interval excluding zero appeared; the other comparisons were not replicated.

Two Mesa 2.3 models. We also migrated our own Mesa 2.3 implementations of the two playbooks of Section 5.2 that use no Mesa 2 grid (Modelly provides cell spaces instead, so a model on a Mesa 2 grid needs its space rewritten): the insect population and Boltzmann

Table 2. Mesa 3.5's own examples on Mesa and, migrated by the codemod, on Modelly: median CPU time of the steps over ten alternating rounds, Modelly's ratio and its per-round range, and the agreement of the model reporters over 100 shared seeds (statistics compared, equivalent within $0.5\,s_p$ by paired TOST, lowest Holm-corrected paired p). The script `paper/scripts/mesa_examples_table.py` generates it from `data/mesa_examples.json`.

		steps (s)		Modelly / Mesa		agreement		
Example	Steps	Mesa	Modelly	median	range	stats	equiv.	min p_{Holm}
Conway's Game of Life	100	0.443	0.406	0.92	0.90–0.95	1	1/1	1.00
Prisoner's dilemma grid	100	1.132	1.052	0.92	0.90–0.96	2	2/2	1.00
Virus on a network	200	0.161	0.119	0.74	0.70–0.75	8	8/8	1.00
Boid flockers	100	0.544	0.524	0.96	0.95–0.97	2	2/2	1.00
Boltzmann wealth	100	0.820	0.546	0.66	0.65–0.69	2	2/2	1.00
Schelling	50	0.248	0.175	0.71	0.68–0.73	8	8/8	1.00
Wolf–sheep	100	0.236	0.141	0.60	0.58–0.61	6	6/6	1.00
Epstein civil violence	100	3.013	2.637	0.87	0.84–1.01	6	6/6	1.00
Sugarscape G1MT	not migrated: a Mesa API Modelly lacks							
Alliance formation	not migrated: a Mesa API Modelly lacks							

wealth. The codemod rewrote imports and constructors; the three steps it reported were done by hand, as its report says (the agents' `unique_id`, the seeding keyword and, for Boltzmann wealth, the activation order). On Modelly (`benchmarks/migrated_mesa2/`) these models give the same statistics as on Mesa 2.3 on every seed tried (20 at the small size, 5 at the large), and over ten alternating rounds in fresh interpreters on one performance core they take 0.21 (insect population) and 0.38 (Boltzmann wealth) of Mesa 2.3's CPU time, every round within 0.20 to 0.22 and 0.37 to 0.38 (`paper/scripts/migrated.py`): these models spend most of their time in framework calls (Mesa 2.3 rebuilds its agent set each time `model.agents` is read and collects agent data in a Python loop), and on Modelly those calls are its compiled ones; upgrading the same models to Mesa 3.5 instead gives 1.9 and 1.8 times (Table 3). Models built on a Mesa 2 grid need their space rewritten before they run on Modelly, and their migration was not evaluated.

Five Mesa 3.5 models. Code written for current Mesa migrates with less effort: five of the eight Mesa 3.5 playbooks run on Modelly after the codemod alone, with no manual step (`benchmarks/migrated_mesa3/`). The other three do not: flocking uses Mesa's legacy `mesa.space.ContinuousSpace`, and predator-prey and Sugarscape use Mesa 3.5's `PropertyLayer` class, neither of whose APIs Modelly reproduces (its own continuous space has another API, and its property layers are plain `numpy` arrays). On Modelly the five take 0.45 (Boltzmann wealth), 0.50 (insect population), 0.76 (grid epidemic), 0.85 (network contagion) and 0.96 (Schelling's model) of Mesa 3.5's CPU time, medians of ten alternating rounds whose per-round ratios span 0.44–0.45, 0.45–0.53, 0.72–0.81, 0.80–0.89 and 0.94–0.99. Their outputs are not bit-identical to Mesa 3.5's, because Modelly derives `model.random` from the seed in a different way, so we compared them in distribution as in Section 5.4: over 100 shared seeds at the small size, 24 of the 26 statistics are equivalent within $0.5\,s_p$, two paired t-tests fall below 5% before correction (the network's peak of infections and the Boltzmann model's largest wealth, $|d| \leq 0.31$) and none after it, and no Kolmogorov–Smirnov test does.

The same model code therefore runs 1.04 to 1.7 times faster on Modelly for Mesa 3.5's own examples, 1.05 to 2.2 times for the Mesa 3.5 playbooks, and 2.6 and 4.8 times for the two Mesa 2.3 models, whose framework calls are slower; the larger gains come from the forms of Section 5.2.

5.2 Each framework's own idiom: eight playbooks

Eight workloads, chosen to exercise different parts of an ABM framework, are each implemented at least twice: a susceptible–infected–recovered (SIR) epidemic on a grid, predator-prey with grass, flocking [94], a contagion on a network, a stage-structured insect population, Schelling's segregation model [105], Boltzmann wealth exchange [26] and Sugarscape [29]. Each has two reference implementations written the way Mesa's API intends: one for Mesa 2.3 (per-agent methods on its grids, continuous space and network space) and one for the current Mesa 3.5 (its cell spaces, `CellAgent`, `AgentSet.shuffle_do` and its network space, as in Mesa's own examples; Appendix B notes the choices made), both plain per-agent Python. Four workloads (the grid epidemic, Schelling's model, Boltzmann wealth and Sugarscape) also have a *mesa-frames* implementation [3], vectorised over polars data frames the way that library's examples write models: it is the vectorised Python competitor to Modelly's columns. Three workloads (the grid epidemic, Schelling's model and Boltzmann wealth) have a *Mesa 3.5 + numpy* implementation, the hand-vectorised baseline: a `mesa.Model` that owns the population as numpy arrays, with no `Agent` objects, and keeps Mesa's grid for its neighbour table and property layers and Mesa's `DataCollector` for the records. It is what an expert gets by giving up agent objects altogether, and so a reference point for what a framework that keeps them can reach. Flocking also has a *Mesa 3.5 + KD-tree* implementation, Mesa 3.5's model with its neighbour search replaced by one periodic `scipy.spatial.cKDTree` per step, which separates the algorithm from the framework (Section 9). The Modelly implementation uses column state, `CellIndex`, KD-tree queries or cohorts, and four workloads have a further Modelly variant, *cell lists*, which keeps per-cell agent lists and vectorises only the obvious phases (Appendix B). The implementations agree within the margins that Section 5.4 measures and reports, apart from a small shift in our mesa-frames Schelling implementation (under 1% of the mean; Section 5.4), and three Mesa 3.5 implementations, which draw the same random numbers in the same order as Mesa 2.3, reproduce it run by run: flocking, the network contagion and the insect population. Each implementation is run twice from the same seed and must reproduce its own output exactly; all do. Table 15 in the appendix gives every workload's parameters. All variants of all workloads were timed in one session, ten rounds each, and again in a second session that agrees with it within 4% on every speed-up but those of Schelling's Mesa 3.5 + numpy baseline (6 and 7%; Section 9), on one performance core, the order of the variants rotating from round to round; each result file records the commit, the date, the load, the core and its power policy, and every package version. Section 9 gives the per-round spreads and the load during the session, and Table 16 in the appendix a bootstrap 95% interval for every speed-up, resampling the paired rounds.

Where Modelly is fast, and why. Modelly's column form runs every workload faster than Mesa 2.3 and Mesa 3.5, and faster than mesa-frames on the four it implements (Table 3 and Figure 1): its steps run 4.6 to 42 times faster than Mesa 2.3, 2.9 to 33 times faster than Mesa 3.5 and 3.0 to 9.5 times faster than mesa-frames, and its cohort form runs the insect population 242 times faster than Mesa 2.3 and 131 times faster than Mesa 3.5. Table 4 gives the reason workload by workload. In each, one Modelly mechanism replaces a Python loop over the agents with a few array operations over the whole population: a column read or write instead of an attribute per agent, one count over the grid's adjacency instead of a neighbourhood walk per agent, one KD-tree query instead of a distance test per pair, one sparse product over a network instead of a loop over each node's neighbours, one binomial draw per class instead of one draw per insect. The gain therefore grows with the share of a step that such a mechanism covers: largest where every phase becomes array work (flocking, the grid epidemic, Schelling's model, the network), smaller where a phase must stay sequential, as the foragers' walk to a free cell in Sugarscape, or where the step creates and removes many agents, as the births and deaths of predator-prey and of the individual insect model, or where a cost every implementation shares dominates the step, as Boltzmann wealth's data collector, which records the Gini coefficient and every agent's

Table 3. The playbooks at their large size: median CPU time of building the model and of running its steps over ten rounds, the speed-ups of Equation (6) over Mesa 2.3, and the speed-up of the steps over Mesa 3.5; Table 16 gives their 95% intervals. Agents: the population at the end of the last round. The GPU variants of the grid epidemic and the insect population, timed in the same session on the CPU, are left out: they draw from Philox streams and are reported with the GPU (Section 7.2); on the CPU the grid epidemic's GPU variant runs its steps in 0.109 s against the columns' 0.154 s. Generated from `benchmarks/playbooks/results/` by `paper/scripts/make_figures.py`.

			CPU time (s)		over Mesa 2.3		over Mesa 3.5
Workload	Variant	Agents	build	steps	steps	total	steps
Grid epidemic (SIR)	Mesa 2.3	20,000	0.034	6.497	1.0	1.0	–
	Mesa 3.5	20,000	0.340	3.904	1.7	1.5	–
	Mesa 3.5 + numpy	20,000	0.299	0.033	198.4	19.7	119.2
	mesa-frames	20,000	0.017	1.067	6.1	6.0	3.7
	Modelly, cell lists	20,000	0.156	2.724	2.4	2.3	1.4
	Modelly, columns	20,000	0.168	0.154	42.1	20.2	25.3
Predator-prey	Mesa 2.3	5,421	0.008	3.256	1.0	1.0	–
	Mesa 3.5	5,453	0.310	3.547	0.9	0.8	–
	Modelly, cell lists	5,567	0.127	2.066	1.6	1.5	1.7
	Modelly, columns	5,402	0.124	0.517	6.3	5.1	6.9
Flocking (boids)	Mesa 2.3	4,000	0.015	24.267	1.0	1.0	–
	Mesa 3.5	4,000	0.023	24.044	1.0	1.0	–
	Mesa 3.5 + KD-tree	4,000	0.024	9.233	2.6	2.6	2.6
	Modelly, columns	4,000	0.018	0.738	32.9	32.1	32.6
Network contagion	Mesa 2.3	30,000	0.164	7.466	1.0	1.0	–
	Mesa 3.5	30,000	0.417	2.378	3.1	2.7	–
	Modelly, columns	30,000	0.584	0.194	38.4	9.8	12.2
Insect population	Mesa 2.3	27,233	0.001	3.637	1.0	1.0	–
	Mesa 3.5	27,233	0.002	1.958	1.9	1.9	–
	Modelly, columns	27,042	0.003	0.671	5.4	5.4	2.9
	Modelly, cohorts	26,465	0.002	0.015	242.5	216.5	130.5
Schelling segregation	Mesa 2.3	31,951	0.043	4.402	1.0	1.0	–
	Mesa 3.5	32,068	0.383	3.386	1.3	1.2	–
	Mesa 3.5 + numpy	31,951	0.354	0.111	39.7	9.6	30.6
	mesa-frames	31,951	0.020	1.902	2.3	2.3	1.8
	Modelly, cell lists	31,951	0.256	0.310	14.2	7.9	10.9
	Modelly, columns	31,951	0.303	0.200	22.0	8.8	16.9
Boltzmann wealth	Mesa 2.3	10,000	0.009	2.055	1.0	1.0	–
	Mesa 3.5	10,000	0.024	1.115	1.8	1.8	–
	Mesa 3.5 + numpy	10,000	<0.001	0.120	17.1	17.1	9.3
	mesa-frames	10,000	0.005	0.973	2.1	2.1	1.1
	Modelly, columns	10,000	0.026	0.323	6.4	5.9	3.5
Sugarscape	Mesa 2.3	710	0.004	1.487	1.0	1.0	–
	Mesa 3.5	702	0.043	1.408	1.1	1.0	–
	mesa-frames	715	0.013	2.076	0.7	0.7	0.7
	Modelly, cell lists	703	0.028	1.088	1.4	1.3	1.3
	Modelly, columns	712	0.046	0.321	4.6	4.1	4.4

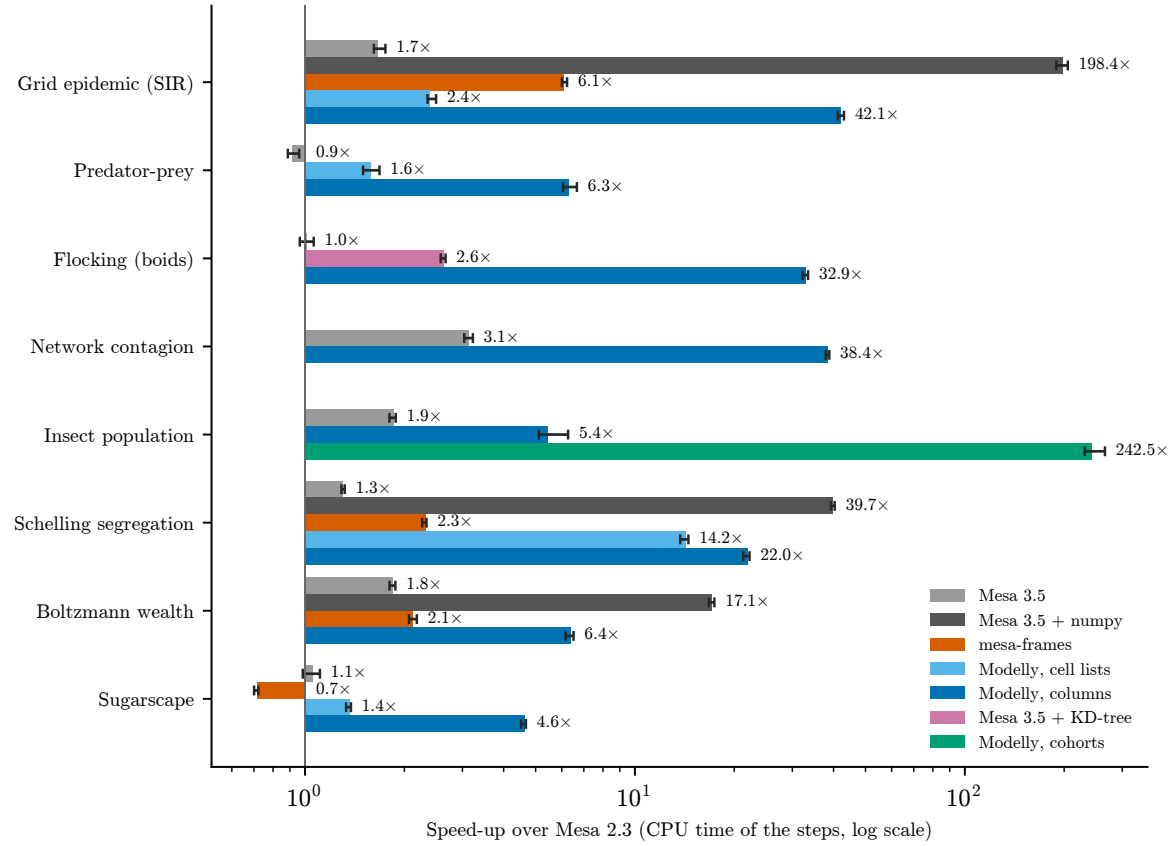


Figure 1. S_{steps} of each implementation over the Mesa 2.3 implementation of the same workload (Table 3); whiskers span the per-round ratios, and bars left of 1 are slower than Mesa 2.3. Columns and cohorts are Modelly's array-based storage; cell lists keep per-cell agent lists, as Mesa's grids do; mesa-frames is the vectorised data-frame library built on Mesa; Mesa 3.5 + numpy replaces Mesa's agents with hand-written arrays.

wealth after each step (Section 9). Code that gives up agent objects altogether goes further still: hand-written numpy is 1.8 to 4.7 times faster than the columns (below). The arguments that each vectorised phase has the law of its per-agent loop are given in the documentation (`docs/vectorised_phases.md`), and Section 5.4 checks that the outputs agree.

The other implementations. Mesa 3.5 is faster than Mesa 2.3 on five workloads (1.3 to 3.1 times), as fast on flocking and Sugarscape (1.01 and 1.06 times), and slower on predator-prey (0.92 times). mesa-frames is 2.1 to 6.1 times faster than Mesa 2.3 on three of its four workloads and slower on Sugarscape (0.72 times), where its per-forager search is a cross join and a sort over up to about 130,000 rows per step. Modelly's cell-list variants, which keep per-cell agent lists, give 1.4 to 14 times over Mesa 2.3 and 1.3 to 11 over Mesa 3.5: large for Schelling's model, whose residents mostly count their neighbours, small where movement and search stay per agent. Construction is where Modelly spends more than Mesa 2.3 (it allocates the columns and indexes the space; Mesa 3.5's cell spaces cost about as much, 0.31 to 0.42 s on the three large grids and the network), so the column form's gain including construction is 4.1 to 32 times, and lower than per step on short runs such as the network contagion (38 times per step, 9.8 in total).

Against hand-written numpy. The Mesa 3.5 + numpy baseline is faster than Modelly's columns on all three of its workloads: its steps take 0.033 s against 0.154 s on the grid epidemic (4.7 times faster, 95% interval 4.6 to 4.8), 0.111 s against 0.200 s on Schelling's model (1.8 times, 1.8 to 1.8) and 0.120 s against 0.323 s on Boltzmann wealth (2.7 times, 2.7 to 2.7; there

Table 4. Where Modelly's speed comes from: for each workload, from the largest step-time speed-up over Mesa 2.3 in Table 3 to the smallest (the insect population by its cohort form), the mechanism that replaces the per-agent Python loop, as the code of each implementation shows (the attribution comes from the code, not from per-workload profiles).

Workload	What Modelly does instead of a loop over agents
Insect population	cohorts: at most about 190 (stage, age, sex) classes thinned by binomial draws replace about 26,000 agents
Grid epidemic	positions are a column of cells: the walk is one `random_neighbor` call, the exposure one count of infected per cell
Network contagion	infected neighbours of every node in one sparse matrix–vector product; infection and recovery are Bernoulli vectors
Flocking	one periodic KD-tree query for the whole flock; the steering terms are pair sums scattered back with `bincount`; positions written in one call
Schelling	happiness is two sparse products (residents per cell, summed over neighbours); all moves in one pass with the sequential rule's law
Predator-prey	movement, energy, grazing and predation (`claim`) are array operations; deaths and births are one call per species
Boltzmann wealth	the exchange is a few array passes (a fixed-point iteration) with the law of the shuffled sequential order
Sugarscape	every forager's best visible cell found in one array pass; only the walk to a free cell stays sequential

partly because it records the wealth as one array per step rather than as per-agent rows). It pays for that with the model's structure: no agent has an identity, methods or attributes of its own, the population cannot hold agents of several kinds, every neighbourhood is a hand-built index table, and every phase has to be rewritten as array operations before the model runs at all, whereas Modelly keeps agent objects, agent sets and spaces next to the columns, and lets a model be vectorised one phase at a time. Its agreement with Mesa 2.3 is that of the other implementations (Table 17: 13 of 14 statistics equivalent within $0.5\,s_p$); the one paired t-test below 5% after the Holm correction is Schelling's segregation ($d = 0.24$, 0.6% of the mean). The baseline runs the column algorithm without the framework layer, and on these seeds its Schelling and Boltzmann wealth runs give exactly the column variant's statistics, so the two share this difference and any bias behind it; on 300 new seeds the column variant's segregation has $d = 0.07$ $[-0.02, 0.15]$ (Table 19). The benchmark harness's own six-seed check flags three of its six grid epidemic statistics, flags that vanish over 100 seeds (at the small size lowest paired p 0.13 and largest $|d|$ 0.21; at the timed size 0.19 and 0.20; all six equivalent in both; `paper/data/agreement_large_grid_mesa3np.json`). The gap is the price of the framework layer: Modelly sits between per-agent Mesa and hand-written arrays, 3.5 to 25 times above Mesa 3.5 and 1.8 to 4.7 times below the numpy baseline on these three workloads.

How the gain changes with population size. The playbooks are timed at one size; Table 5 times three of them, whose populations scale with one parameter, at the playbook's small size, its timed size and ten times that, five rounds each on the same fixed-frequency core (`paper/scripts/sizes.py`, `paper/data/sizes.json`). On the grid epidemic and Schelling's model the columns' lead grows with the population: on the grid epidemic it goes from 3.9 to 41 and 144 times over Mesa 2.3 and from 2.4 to 24 and 77 times over Mesa 3.5 as the population goes from 2,000 to 20,000 and 200,000 agents; on Schelling's model from 1.3 to 22 and 50 times over Mesa 2.3 and from 1.0 to 17 and 37 times over Mesa 3.5; on Boltzmann wealth, whose data collector records every agent's wealth each step, it grows over Mesa 2.3 (4.0, 6.4 and 9.9 times) but stays at 2.7 to 3.5 times over Mesa 3.5, and shrinks over mesa-frames (14, 3.0 and 2.0 times). At the smallest sizes the framework's fixed cost per step weighs most: on Schelling's model with 1,282 residents the columns are as fast as Mesa 3.5. Hand-written numpy's lead over

Table 5. The step-time speed-ups of Modelly's columns, and of hand-written numpy over the columns, at three population sizes: the playbook's small size, its timed size (as in Table 3) and ten times that at the same density. Medians of five rotated rounds on one fixed-frequency core; agents at the end of the run. Generated by `paper/scripts/sizes.py`.

				columns faster than			numpy faster
Workload	Size	Agents	Mesa 2.3, s	Mesa 2.3	Mesa 3.5	mesa-frames	than columns
Grid epidemic	small	2,000	0.39	3.9	2.4	2.5	17.0
	large	20,000	6.25	40.6	24.2	6.9	4.7
	huge	200,000	114.31	144	77.0	12.7	1.5
Schelling	small	1,282	0.13	1.3	1.0	2.5	13.4
	large	32,090	4.45	22.1	17.2	9.5	1.9
	huge	319,610	63.58	49.6	37.1	15.7	1.0
Boltzmann wealth	small	500	0.08	4.0	2.7	14.0	1.8
	large	10,000	2.06	6.4	3.5	3.0	2.7
	huge	100,000	39.96	9.9	3.4	2.0	2.8

the columns shrinks the other way on the grid epidemic (17, 4.7 and 1.5 times) and Schelling's model (13.4, 1.9 and 1.0 times), where at large populations the columns run at nearly the speed of code without agents while keeping them; on Boltzmann wealth it stays at 1.8 to 2.8 times.

On other interpreters. Every timing above ran on CPython 3.15. Since per-agent code gains from each interpreter release and array code does not, the ratios could depend on the version, so we timed the Mesa 3.5 and column implementations of all eight playbooks (and the insect population's cohorts) under CPython 3.12, 3.14 and 3.15, each in its own environment with the same package versions and Modelly's C modules built for it, five rotated rounds in fresh interpreters pinned to the same fixed-frequency core (`paper/scripts/python_versions.py`, Table 23 in the appendix). The speed-ups on 3.15 reproduce Table 3's within 7% (25.5, 6.9, 31.8, 13.1, 3.0, 17.6, 3.4 and 4.4 against 25.3, 6.9, 32.6, 12.2, 2.9, 16.9, 3.5 and 4.4; the cohorts 132 against 131), and on 3.12 and 3.14 they are similar or larger: 34 and 32 on the grid epidemic, 9.5 and 7.8 on predator-prey, 17 and 22 on Schelling's model, 4.2 and 4.2 on Boltzmann wealth, within 14% of the 3.15 values on the other four, with the cohorts at 143 and 158. Mesa's per-agent steps are 1 to 27% faster on 3.15 than on 3.12 (25% on the grid epidemic, 27% on predator-prey) while the columns' steps change by at most 9% on six playbooks (22% on Schelling's model and 12% on the insect population, whose column steps keep some per-agent work), so the newest interpreter is the least favourable of the three to Modelly on six of the eight workloads (the insect population's columns and Schelling's model 3% lower on 3.12), and the speed-ups reported in the paper are, within 4%, the conservative ones.

A sweep end to end. Step time dominates long runs; a sweep of short runs also pays for starting workers, importing and building the model, reading its outputs after every step and writing them to the store. We timed one small sweep end to end (`paper/scripts/sweep_e2e.py`, Table 24 in the appendix): 64 runs of the grid epidemic at the small size (a 16-point Latin hypercube over β, four replicates, 60 steps, one output per step), from the creation of the `Experiment` to its results queried from the Parquet store, driven by Modelly's experiment layer with each of the two engines, the column implementation and the Mesa 3.5 one (which takes its parameters by name and steps like any model, so the workflow's overhead is the same on both sides), medians of five rounds in fresh interpreters. On the serial executor pinned to one core the sweep takes 2.42 s with the columns and 19.22 s with Mesa 3.5, a factor of 7.9, against 12 for the simulation alone at this size (0.024 against 0.278 s per run): with runs this short, 37% of the column sweep is the workflow (0.014 s per run: the task's dispatch, the output read after each

of the 61 states, the Parquet write) and 7% of the Mesa one. On four processes the Mesa sweep is 2.8 times faster than serial and the column sweep barely faster (2.34 s): 64 runs of 0.02 s do not repay starting four workers and importing the model in each, so a sweep of runs this short on Modelly should stay serial, or batch many runs per task. The gain a study sees from the columns is therefore the step-time speed-up for long runs and, for short runs, a smaller factor that the workflow's fixed cost per run bounds; Table 3's S_{total} gives the bound from construction alone, and this measurement adds the experiment layer's share. The session ran on the same fixed-frequency core under a load average of 4 to 7.

mesa-frames with more threads. polars runs multi-threaded by default, while the timings above set `POLARS_MAX_THREADS=1` so that every implementation uses one core; pinned to the four performance cores, five alternating rounds in wall-clock time, mesa-frames with four threads was faster than with one on all four of its workloads, 1.04 to 1.21 times on three and 1.6 times on Schelling's model (1.28 s against 2.11 s; `paper/data/frames_threads_pinned.json`), so on those cores the one-thread comparison overstates the columns' lead over mesa-frames by up to that factor. With all 16 threads, unpinned, single runs gave Schelling's model 1.5 times faster and the other three slower (Appendix B): on the whole machine the one-thread comparison may overstate the columns' lead on Schelling's model by up to that factor.

5.3 Where the gain comes from: an ablation

Between per-agent Mesa 3.5 code and Modelly's column form several things change at once: the framework whose calls the same code makes, whether those calls run in C or in Python, the data structure that holds the population, the algorithm of a search, and whether a phase is written per agent or as array operations. To attribute the gain, we timed a ladder of implementations that changes one of these per rung on four workloads (`paper/scripts/ablation.py`, Table 6): ten rotated rounds in fresh interpreters pinned to one core, each Modelly rung run twice, once with `MODELLY_PURE_PYTHON=1` (the framework's inner loops in Python) and once with the C modules, so that the compiled runtime's share is read off within the rung. The session ran on the same fixed-frequency core as Table 3, and its times are within 8% of that table's; every ratio is paired within the session, and the speed-ups of the columns over Mesa 3.5 that it gives (24.3, 17.2, 33.0 and 4.3) agree with the main table's (25.3, 16.9, 32.6 and 4.4) within 4%.

What each rung contributes. The framework's own calls, with no phase rewritten, are worth little: the same code migrated to Modelly runs 1.01 (grid epidemic) and 1.04 (Schelling) times faster than on Mesa 3.5 with the inner loops in Python, and the C modules add a factor of 1.34 on the grid epidemic, whose step is dominated by per-agent moves and neighbourhood reads that the compiled loops serve, and 1.05 on Schelling's model, whose step is dominated by neighbour counting in model code. The data structure alone is worth about as little: on Sugarscape, where the cell-list variant runs Mesa's search algorithm on Modelly's cell lists, it gives 1.16 with the Python core and 1.28 with the compiled one. The algorithm is worth what it is worth: the KD-tree alone makes Mesa 3.5's flocking 2.6 times faster. Everything else is the array form of the phases. On the grid epidemic, moving from cell lists to columns, with no change of algorithm and with the C modules off on both sides, is a factor of 22; on Schelling's model vectorising the neighbour count alone (the cell-list rung, Python core) is a factor of 9.6 over the migrated code and vectorising the moves as well (columns) another 1.7; on flocking the steering terms as array operations are a factor of 12.6 over the KD-tree baseline; on Sugarscape the search as one array pass is a factor of 3.7 over the same search per forager. The compiled runtime contributes nothing to the column rungs (0.99 to 1.00): once a phase is an array operation, the framework's loops are no longer on the path, and the time is numpy's. In the ablation's session the column form sits 1.8 to 4.6 times above hand-written numpy without agent objects (1.8 to 4.7 in Table 3), which is the price of keeping agents, agent sets and spaces

Table 6. The ablation: one ladder of implementations per workload, each rung changing one thing from the rung above. Median CPU time of the steps over ten rotated rounds with the framework's inner loops in Python (`MODELLY_PURE_PYTHON=1`) and compiled; the compiled runtime's gain within the rung (median of the per-round ratios); and the speed-up of the compiled rung over Mesa 3.5 and over the rung above. The Mesa 3.5 and numpy rungs have no Python-core arm. Generated by `paper/scripts/ablation.py` from `paper/data/ablation.json`.

		steps (s)			speed-up (compiled)	
Workload	Rung	Python	compiled	C gain	Mesa 3.5	rung above
Grid epidemic	Mesa 3.5: per-agent code on Mesa's cell space	–	3.699	–	1.0	–
	migrated: same code, Modelly's calls (codemod)	3.662	2.772	1.34	1.4	1.35
	cell lists: Mesa's data structure; infection phase as arrays	3.282	2.528	1.30	1.5	1.09
	columns: every phase as arrays over columns	0.149	0.152	0.99	24.3	16.75
	Mesa 3.5 + numpy: arrays without agent objects or framework	–	0.033	–	112.2	4.59
Schelling	Mesa 3.5: per-agent code on Mesa's cell space	–	3.456	–	1.0	–
	migrated: same code, Modelly's calls (codemod)	3.314	3.137	1.05	1.1	1.10
	cell lists: Mesa's data structure; neighbour count as arrays	0.345	0.318	1.09	10.8	9.82
	columns: every phase as arrays over columns	0.199	0.201	0.99	17.2	1.59
	Mesa 3.5 + numpy: arrays without agent objects or framework	–	0.111	–	31.2	1.82
Flocking	Mesa 3.5: per-agent code, all-pairs neighbour scan	–	24.367	–	1.0	–
	Mesa 3.5 + KD-tree: same code, one KD-tree per step	–	9.304	–	2.6	2.62
	columns: KD-tree; steering as arrays over columns	0.742	0.741	1.00	33.0	12.59
Sugarscape	Mesa 3.5: per-agent code, one search per forager	–	1.431	–	1.0	–
	cell lists: Mesa's data structure, same search	1.218	1.105	1.10	1.3	1.28
	columns: search as one array pass over columns	0.324	0.322	1.00	4.3	3.40

next to the arrays. The attribution is therefore: of the grid epidemic's factor of 24, about 1.4 is the framework and its compiled loops and about 18 the array representation of the phases; of flocking's 33, 2.6 is the neighbour-search algorithm and 13 the array form of the steering; of Schelling's 17, 1.1 is the framework and 16 the array form; of Sugarscape's 4.3, 1.28 is the data structure and 3.4 the array form of the search.

5.4 Do the faster forms compute the same model?

The implementations draw their random numbers in different orders, so their dynamics cannot agree run by run; they must agree in distribution. Where two implementations draw the initial state the same way, a seed gives both the same start: strongly in Sugarscape (correlation between

the two implementations' outputs on a seed $r = 0.91$ to 0.99) and Schelling's model (0.42 to 0.70), completely in flocking, whose dynamics are deterministic once it has started ($r = 1$). The comparisons are therefore paired, seed by seed. We ran the Mesa 2.3 reference and every Modelly variant of every playbook on the same 100 seeds at the playbook's small size (2,100 runs), and compared each of the 67 pairs of a summary statistic and a Modelly variant (epidemic peak and final size, final populations, cluster counts, Gini coefficient, and so on) with the Mesa reference in three ways: a paired t-test on the seed-by-seed differences, the two-sample Kolmogorov–Smirnov test on the two marginal distributions (an unpaired test, conservative here because the shared seeds correlate the samples, and more so for the integer-valued statistics, whose values tie, so it serves as a coarse check only), and paired two one-sided tests (TOST) of equivalence [65, 106] with a margin of half a pooled standard deviation s_p, that is, against the null hypothesis $|\mu_{\text{Modelly}} - \mu_{\text{Mesa}}| \geq 0.5\, s_p$. We report the standardised difference $d = (\bar{x}_{\text{Modelly}} - \bar{x}_{\text{Mesa}})/s_p$ with its paired 95% interval, and correct for the 67 comparisons with Holm's procedure [46]. Table 27 in the appendix lists every comparison. Three kinds of statement follow from these tests, and we keep them apart throughout: a comparison is *shown equivalent* when its TOST rejects at 5% (nominally, one comparison at a time; or after the Holm correction, which controls the familywise error over the 67); a comparison is *shown different* when its paired interval excludes zero; and a comparison that is neither is *undecided*, not equal, since a nonsignificant difference test can reflect low power. The margin of $0.5\, s_p$ is a convention (a medium effect by Cohen's scale), not a scientific tolerance; we therefore also report how the counts change with the margin, give counts at margins relative to Mesa's mean, and, in Section 5.5, test science-specific margins prespecified in each quantity's own units.

Table 7. Agreement with Mesa 2.3 over 100 shared seeds per variant, at the small size: the largest standardised difference, the lowest paired t and Kolmogorov–Smirnov p-values (before correction), and the number of statistics equivalent within $0.5\, s_p$ (paired TOST at 5%, uncorrected). Generated by `paper/scripts/agreement.py`.

Workload	Modelly variant	Stats	max $\lvert d \rvert$	min p_t	min p_{KS}	Equivalent
Grid epidemic	cell lists	6	0.23	0.08	0.15	6/6
Grid epidemic	columns	6	0.13	0.39	0.58	6/6
Predator-prey	cell lists	5	0.30	0.02	0.01	4/5
Predator-prey	columns	5	0.13	0.34	0.47	5/5
Flocking	columns	3	0.00	0.90	1.00	3/3
Network contagion	columns	6	0.16	0.20	0.21	6/6
Insect population	columns	6	0.33	0.02	0.04	4/6
Insect population	cohorts	6	0.23	0.10	0.21	6/6
Schelling	cell lists	4	0.23	0.03	0.37	4/4
Schelling	columns	4	0.24	<0.01	0.28	4/4
Boltzmann wealth	columns	4	0.29	0.03	0.21	3/4
Sugarscape	cell lists	6	0.03	0.33	0.82	6/6
Sugarscape	columns	6	0.01	0.65	0.97	6/6

Table 7 and Figure 2 give the results. Within the margin of $0.5\, s_p$, 63 of the 67 comparisons are shown equivalent at the nominal 5% level (60 of 64 outside flocking, whose three agree trivially), and 29 of 67 after the Holm correction of the TOST p-values. The two counts answer different questions: the first says how many comparisons, each taken on its own, meet the margin; the second how many do so with the familywise error held at 5% over all 67, and it is the one to quote when the claim is about the set. The four comparisons not shown equivalent even nominally are the mean sheep population of predator-prey with cell lists, the final and the peak adult counts of the individual insect model, and the share of agents without wealth in the Boltzmann model ($|d| \leq 0.33$, at most 1.4% of the mean). The count depends on the margin: 50 at $0.4\, s_p$, 27 at $0.3\, s_p$ and 19 at $0.25\, s_p$; with a margin relative to Mesa's mean instead, 48 of 67

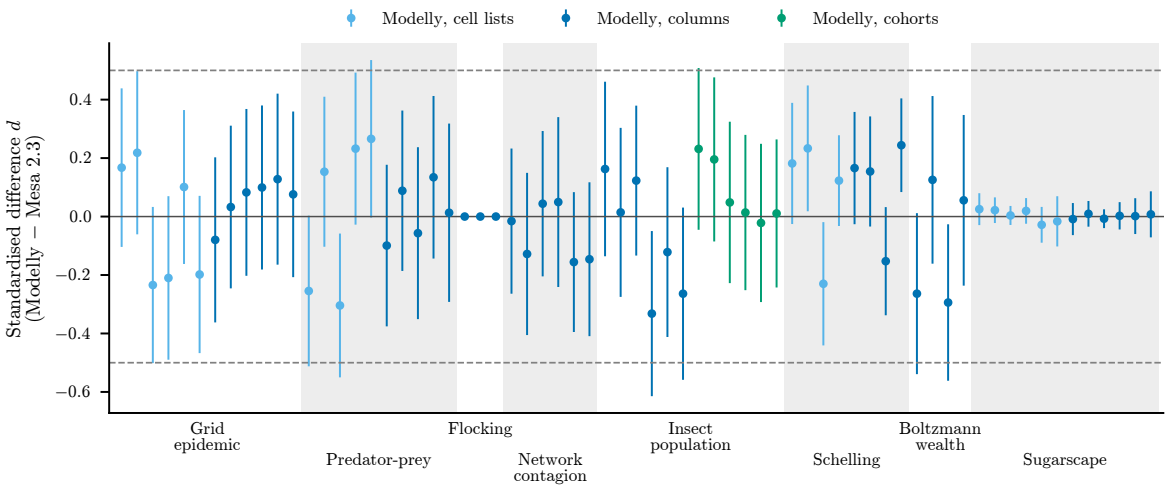


Figure 2. Standardised difference d between each Modelly variant and Mesa 2.3 for every summary statistic, with 95% intervals, over 100 seeds; dashed lines: the equivalence margin $\pm 0.5\, s_p$. Flocking is deterministic once its initial state is drawn, and both implementations draw it from the same seeded `numpy` stream: it agrees run by run to rounding error, hence $d = 0$.

are shown equivalent within ±5% and 56 within ±10%, the others being the noisiest statistics at this small size (the grid epidemic's counts with 2,000 agents, predator-prey's wolves), whose 100-seed intervals are wider than those margins. The intervals themselves are the most direct summary: every $|d|$ is at most 0.33, 58 of the 67 intervals lie within $[-0.5, 0.5]$ and all within $[-0.62, 0.62]$; every difference of means is at most 11% of Mesa's, and under 5% for 59 of 67. Six intervals exclude zero, so six differences are shown, taken one at a time: three in Schelling's model ($|d| \leq 0.24$, at most 4.3% of the mean, made detectable by the shared initial state), the mean sheep population of predator-prey with cell lists, the final adult count of the individual insect model and the share of agents without wealth in the Boltzmann model ($|d| \leq 0.33$, at most 1.4%). After the Holm correction no paired or Kolmogorov–Smirnov test falls below 5% (the smallest corrected paired p is 0.21); we do not read this as equality, since at the corrected level a paired test on 100 seeds detects $d = 0.33$ with probability 0.13 only (Section 9), but as consistent with the intervals above. Before correction, six paired t-tests and three Kolmogorov–Smirnov tests fall below 5%, against 3.2 expected by chance for each test among the 64 comparisons outside flocking if the tests were independent (they are not: the statistics of a workload are related, such as its final S, I and R). The spreads agree too: the ratio of the two implementations' standard deviations lies between 0.77 and 1.27, and one median-centred Levene test [16] of 67 falls below 5%, none after the Holm correction; within the power of a Levene test on 100 seeds (unpaired, and so conservative here) this is consistent with a vectorised phase leaving the variability of an outcome unchanged, and no more. The benchmark harness's own quick check, six seeds and an unpaired Welch test [126] at 1%, could not have shown any of this: its power to detect a difference of one standard deviation is 13%.

At the timed size. We repeated the study at the large size used for the timings, with 50 seeds per variant (1,050 runs; Table 8). 53 of the 67 comparisons are shown equivalent within the margin (nominally), fewer than at the small size because half as many seeds give wider intervals; one paired t-test (Sugarscape with cell lists, mean wealth, $d = 0.11$, 0.3% of the mean) and two Kolmogorov–Smirnov tests fall below 5% before the Holm correction and none after it, with the caveat on power above. The largest differences were in the grid epidemic's peak time, about 11% later in both Modelly variants ($d = 0.43$ and 0.40, paired $p = 0.05$ and 0.07). A replication on 200 new seeds, disjoint from these 50, does not reproduce them: no interval of

the twelve grid epidemic comparisons excludes zero, the largest $|d|$ is 0.13 (the peak time's is 0.03 and 0.04, under 1% of the mean), and all twelve are equivalent within $0.4\,s_p$, also after the Holm correction. The trajectory study at the small size (Section 5.5) finds the columns' peak 3.3 steps *earlier* on 100 other seeds; taken together, the three samples put the columns' peak timing within about ±12% of Mesa's, with the sign undetermined, which is what the tests' power at this margin allows.

The other baselines against Mesa 2.3. The same study on the Mesa 3.5 and mesa-frames implementations (100 seeds at the small size, 2,000 runs; Table 18) finds all 40 Mesa 3.5 statistics equivalent within $0.5\,s_p$ with nothing below 5% even before correction, and 18 of the 20 mesa-frames statistics equivalent with one difference that survives the Holm correction ($p = 0.013$): its Schelling implementation ends more segregated than Mesa 2.3's ($d = 0.31$, 0.7% of the mean). A replication of Schelling's model on 300 new seeds (Table 19) finds no difference after the Holm correction for any implementation; before it, mesa-frames' segregation still differs ($d = 0.10$ $[0.01, 0.18]$, 0.2% of the mean, $p = 0.02$), as do Mesa 3.5's share of happy residents and number of moves ($|d| \leq 0.23$, at most 3.8%), while neither Modelly variant does. The mesa-frames shift is small, its cause in our implementation unknown, and it does not change its timing's order of magnitude. Table 20 gives the grid epidemic over 200 new seeds.

Table 8. As Table 7, at the large size of the timings, over 50 shared seeds per variant.

Workload	Modelly variant	Stats	max $\lvert d\rvert$	min p_t	min p_{KS}	Equivalent
Grid epidemic	cell lists	6	0.43	0.05	0.04	2/6
Grid epidemic	columns	6	0.40	0.07	0.07	4/6
Predator-prey	cell lists	5	0.21	0.25	0.27	4/5
Predator-prey	columns	5	0.14	0.41	0.27	5/5
Flocking	columns	3	0.00	0.22	1.00	3/3
Network contagion	columns	6	0.27	0.14	0.18	5/6
Insect population	columns	6	0.33	0.12	0.02	3/6
Insect population	cohorts	6	0.20	0.31	0.18	4/6
Schelling	cell lists	4	0.09	0.46	0.72	4/4
Schelling	columns	4	0.07	0.59	0.72	4/4
Boltzmann wealth	columns	4	0.19	0.37	0.27	3/4
Sugarscape	cell lists	6	0.11	0.02	0.55	6/6
Sugarscape	columns	6	0.07	0.06	0.72	6/6

5.5 Do the faster forms follow the same dynamics?

Summary statistics at the end of a run can agree while the runs that produce them differ: in the shape of the epidemic curve, the spatial pattern it leaves, the speed at which a city segregates, the period of a predator-prey cycle, or how often a species dies out. We therefore compared trajectories and patterns on three playbooks, over 100 new shared seeds (20,000 to 20,099, disjoint from the playbook studies' seeds) at the small size, each Modelly variant against Mesa 2.3, and, as a reference for what two per-agent implementations of the same model differ by, Mesa 3.5 against Mesa 2.3 on the same seeds (`paper/scripts/trajectories.py`). The quantities and their margins were fixed before the runs (Table 22 lists them): for the grid epidemic, the whole infected curve, the peak's height and timing, the final size, and the clusters of cells with anyone ever infected at the end (8-connected on the torus: their number, the largest and the mean size); for Schelling's model, the curves of the happy share and of the moves, the final segregation and happy share, the composition of every resident's neighbourhood (the share of residents in each of ten bins of the same-kind share), and the time to converge (steps with moves above 2% of the residents); for predator-prey, run for 2,000 steps rather than the

playbook's 200 so that the cycle shows, the sheep, wolf and bare-grass curves, the probability that the wolves or the sheep die out within the run, and the amplitude and period of the sheep cycle (the first peak of its autocorrelation). The margins are in each quantity's own units: 2 steps for a timing, 0.02 for a share, 5 percentage points for a probability, ±5% of Mesa's mean for a count the primary study already covers and ±10% for a derived quantity. Each scalar is tested by a paired TOST at its margin, the 78 scalars Holm-corrected as one family; each curve by the largest standardised difference over its steps, $\sup_t |d_t|$, with a 95% upper bound from a bootstrap over seeds, and by a sign-flip permutation test of the same statistic.

Table 9. Trajectories and patterns over 100 shared seeds at the small size: per workload and implementation, the number of curves compared, the largest standardised difference over all steps of any curve and its 95% bootstrap upper bound, the number of scalar quantities, their largest $|d|$, and how many are shown equivalent at their prespecified margin (paired TOST at 5%; in brackets, after the Holm correction over all 78 scalars). Mesa 3.5 against Mesa 2.3 is the reference: two per-agent implementations of the same model. Generated by `paper/scripts/trajectories.py`; Tables 21 and 22 in the appendix give every curve and every scalar.

Workload	Implementation	Curves	$\sup_t \lvert d_t \rvert$	upper 95%	Scalars	max $\lvert d \rvert$	Equivalent
Grid epidemic	Mesa 3.5	1	0.15	0.46	6	0.18	2/6 (2)
Grid epidemic	cell lists	1	0.24	0.52	6	0.23	2/6 (2)
Grid epidemic	columns	1	0.34	0.54	6	0.28	2/6 (2)
Schelling	Mesa 3.5	2	0.13	0.46	14	0.17	13/14 (11)
Schelling	cell lists	2	0.28	0.48	14	0.21	13/14 (12)
Schelling	columns	2	0.07	0.29	14	0.18	14/14 (12)
Predator-prey	Mesa 3.5	3	0.39	0.67	6	0.15	5/6 (4)
Predator-prey	cell lists	3	0.37	0.66	6	0.19	5/6 (3)
Predator-prey	columns	3	0.35	0.65	6	0.14	5/6 (3)

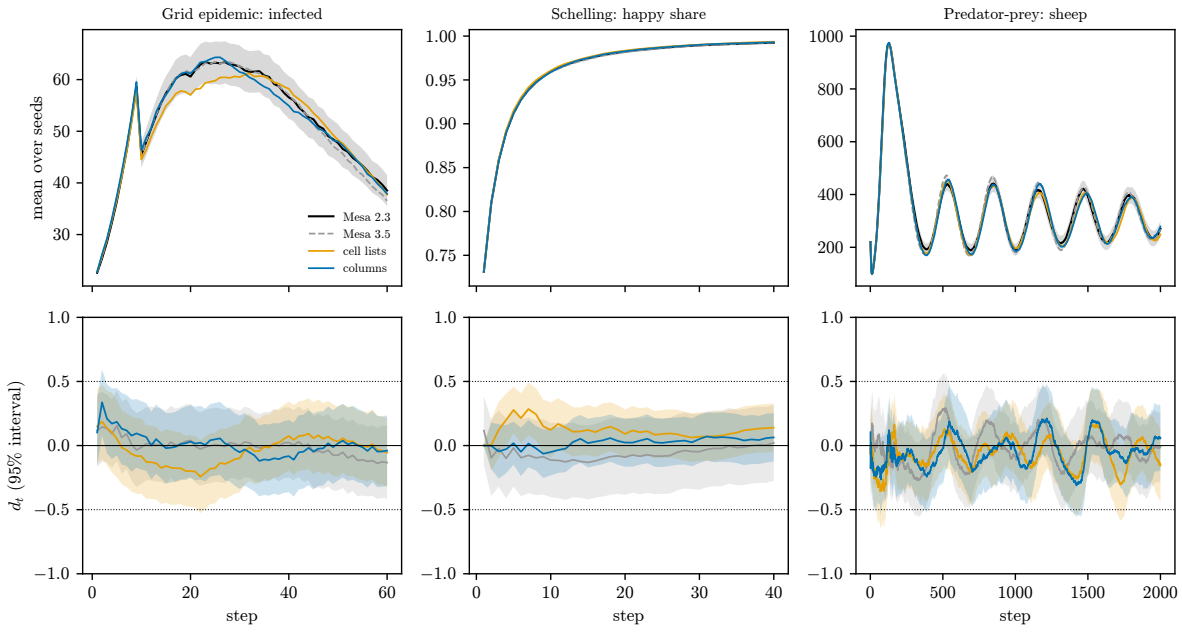


Figure 3. Top: the mean over 100 seeds of the infected count (grid epidemic), the happy share (Schelling) and the sheep count (predator-prey, 2,000 steps) for Mesa 2.3 (black, with a band of two standard errors), Mesa 3.5 (dashed) and the two Modelly variants. Bottom: the standardised paired difference d_t from Mesa 2.3 at every step, with its 95% interval; dotted lines at ±0.5.

Table 9 and Figure 3 give the results. On the curves, the Modelly variants are as close to

Mesa 2.3 as Mesa 3.5 is. On the grid epidemic's infected curve the largest $|d_t|$ is 0.34 for the columns, 0.24 for the cell lists and 0.15 for Mesa 3.5 (upper bounds 0.54, 0.52 and 0.46), the largest relative difference of the means over the steps where the epidemic is at least a tenth of its peak is 3.9, 7.4 and 5.3%, one step of 60 has an interval excluding zero (the columns, at step 2, where the first infections of the run are drawn), and no permutation test finds a difference ($p = 0.16$, 0.52, 0.92). On Schelling's model the columns follow Mesa 2.3 more closely than Mesa 3.5 does ($\sup_t |d_t| = 0.07$, upper bound 0.29, against 0.13 and 0.46; the means of the happy share differ by at most 0.1%) while the cell lists show a small difference at steps 5 to 8 ($\sup_t |d_t| = 0.28$, four steps with intervals excluding zero, permutation $p = 0.045$ for the happy share and 0.048 for the moves; at most 0.4% of the happy share and 4.8% of the moves). Over the 2,000 steps of predator-prey the largest $|d_t|$ is 0.29 to 0.39 for every implementation, Mesa 3.5 included (upper bounds 0.61 to 0.67; a maximum over 2,000 noisy steps), the permutation tests find nothing ($p \geq 0.24$), and the cycles of the four implementations are in phase for 2,000 steps (Figure 3): the same initial conditions set the phase, and the period, 345 steps in Mesa 2.3, differs by at most 18 steps in the others, with intervals of about ±33 steps.

On the scalars, 41 of the 52 Modelly comparisons are shown equivalent at their prespecified margin (34 after the Holm correction) against 20 of 26 for Mesa 3.5 (17); the study therefore does not establish equivalence for every quantity it tests, and we do not claim that it does: 18 of the 52 are undecided at the familywise level. The largest $|d|$ is 0.28 (Mesa 3.5: 0.18), and two paired t-tests of 52 fall below 5% before correction (none after; Mesa 3.5: none). The quantities not shown equivalent are the same for every implementation, Mesa 3.5 included, and for the same reason: their margins are narrower than what 100 seeds resolve. The peak's timing has a standard deviation of about 11 steps across seeds, so its paired interval is about ±3.2 steps against a margin of ±2; the peak's height (±5% = 3.8 against an interval of ±5.3), the final size (±14.6 against ±20), the largest cluster (±1.9 against ±2.7), the total moves (±99 against ±110, shown equivalent for the columns only) and the period (±35 against ±33) are the others. For all of these the point estimate lies inside the margin, except the columns' peak timing, which is the one scalar shown different: the columns' epidemic peaks 3.3 steps earlier than Mesa 2.3's ($[-6.6, -0.1]$, $d = -0.28$, 12% of a mean of 28 steps); the cell lists' and Mesa 3.5's peak 2.9 and 2.1 steps earlier with intervals that include zero. The second uncorrected difference is the cell lists' share of residents with no same-kind neighbour, 0.4% against 0.5% ($d = -0.21$). What the study does establish is this. Everything it was designed to catch beyond the summary statistics agrees: the cluster counts and mean cluster sizes are equivalent within ±10% for every implementation after the Holm correction, the ten composition bins are equivalent within 0.02 for both Modelly variants before and after it (nine of ten for Mesa 3.5 after it), the time to converge within 2 steps for every implementation before it, and the wolves die out in 1 of 100 runs on Mesa 2.3 and Mesa 3.5 and in 0 of 100 on either Modelly variant (sheep never), equivalent within 5 percentage points after it. The claim this supports is bounded: on these three workloads, at this size, the faster forms follow the same dynamics as Mesa 2.3 to within what 100 seeds resolve, and about as closely as Mesa 3.5 does; for the six quantities whose margins 100 seeds cannot decide, the point estimates lie within the margins but equivalence is not shown, and one timing difference is shown. The variants are therefore not interchangeable for every scientific question: one whose answer turns on the timing of an epidemic peak to within a few steps should treat the column form as a different implementation and test it at its own tolerance. A run of this study at the timed size, and on the other five playbooks, was not made; the three chosen are the ones with spatial structure, a convergence process and a cycle.

6 Correctness and reproducibility

6.1 Same seed, same digest

A determinism suite runs seventeen examples (the fourteen bundled examples and the three split-run reference models of Section 7.1, distinct from the eighteen zoo models of Section 4.6) for ten steps from a fixed seed and digests their collected data and final agent state (SHA-256 of a canonical serialisation). The digest must be identical between the compiled and the pure-Python builds, between a straight run and a run saved and resumed from a checkpoint at the midpoint (fifteen of the seventeen examples can be checkpointed), and against reference digests stored in the repository; the three split-run models must also give it split into 2, 4 and 16 partitions. Ten examples are compared exactly. Seven pass through functions whose last bit legitimately differs between platforms' maths libraries (`exp`, `log`, non-integer powers, `arctan2`, BLAS reductions) and are digested at ten significant digits; one of these depends on geographic libraries and is compared only on the platform that recorded it. Property-based tests check invariants of columns, cohorts, cell spaces, checkpoints and experiment designs on random inputs.

Three guarantees, by scope. Determinism, numerical reproducibility and bitwise reproducibility are different properties, and the suite tests each within a stated scope. *Determinism on one build*: the same code, inputs and seed give the same bytes, run straight or resumed, compiled or pure, in one process or split across workers; this is what `modelly verify` and the split-run contract of Section 7.1 promise, and it is exact. *Numerical reproducibility across platforms*: every example the suite compares off its recording platform (all but the geographic one) is meant to give the same digest at ten significant digits on every platform and interpreter, which is the guarantee a reader of a published study needs when rerunning it on another machine; for this version it was checked on one platform under three interpreters (below). *Bitwise reproducibility across platforms*: expected for the examples whose arithmetic is integer or exactly rounded, and not claimed in general, since the last bits of `exp`, `log`, powers, `arctan2` and BLAS reductions depend on the platform's libraries.

Where the checks ran. Table 10 states where each part of this is exercised for the version described. Every check ran on the development machine (Linux x86-64), under three interpreters: CPython 3.12, 3.14 and 3.15, each with the compiled modules built for it and in pure Python. Beyond the golden suite's comparison, each of these six builds wrote the exact, unrounded digest of every example (`python -m modelly.dev golden -json`; `paper/data/local_digests/`, compared by `paper/scripts/local_digests.py`): all seventeen examples give one digest on all six, bit for bit, the seven that the suite compares at ten significant digits included, so on one machine neither the interpreter version nor the compiled modules change a single bit. The determinism suite and the distributed suite (split runs of 2, 4 and 16 partitions, two Dask workers, and Open MPI with 2 and 4 ranks) passed under all three interpreters (134 tests each; three skipped: the two examples that cannot be checkpointed skip their resume test, and Ray was not installed). The release workflow that built the published wheels imported each one on its own runner, Linux, macOS and Windows on x86-64 and ARM64, and checked that all five compiled modules were active. Beyond that import, this version was not run on macOS, Windows or ARM, nor from the published wheels: the bitwise comparison across operating systems, architectures and maths libraries, and the verification on one platform of a bundle written on another, are untested for it, and a reader who reruns a study on such a platform should rely on the ten-digit guarantee, which is what the golden suite checks, and confirm it with `python -m modelly.dev golden`.

Table 10. Where the reproducibility checks ran for the version described.

Check	Where it ran
Golden digests, straight and resumed	Development machine (Linux x86-64) under CPython 3.12, 3.14 and 3.15; exact digests identical on all three, compiled and pure
Compiled core against pure-Python core, same digests	Development machine, three interpreters; the free-threaded build of the event heap untested
Split runs equal one process (2, 4, 16 partitions; two Dask workers)	Development machine, three interpreters
MPI ranks equal one process	Development machine: Open MPI, 2 and 4 ranks, three interpreters; MS-MPI and macOS untested
A bundle written on one platform verifies on another	Untested for this version
GPU backends give the `numpy` integers	Development machine's GPU environment (CuPy, CUDA JAX); Metal untested
Compiled wheels	Built by the release workflow for Linux, macOS and Windows on x86-64 and ARM64 and imported on each with its five compiled modules active; not otherwise run (no digests from macOS, Windows, ARM or the wheels)
Executor independence; calibration resumed from a checkpoint	Development machine

6.2 Validation of the scientific methods

Sobol indices on the Ishigami function. The Ishigami function [49],

$$f(x) = \sin x_1 + a \sin^2 x_2 + b\, x_3^4 \sin x_1, \qquad x_i \sim \mathcal{U}(-\pi, \pi), \quad a = 7,\ b = 0.1, \tag{7}$$

has known indices. With $V_1 = \frac{1}{2}(1 + b\pi^4/5)^2$, $V_2 = a^2/8$ and $V_{13} = b^2\pi^8(\frac{1}{18} - \frac{1}{50})$, the variance is $V = V_1 + V_2 + V_{13} = 13.845$, and

$$S = \left(\tfrac{V_1}{V}, \tfrac{V_2}{V}, 0\right) = (0.314, 0.442, 0), \qquad S_T = \left(\tfrac{V_1+V_{13}}{V}, \tfrac{V_2}{V}, \tfrac{V_{13}}{V}\right) = (0.558, 0.442, 0.244). \tag{8}$$

With $N = 4{,}096$ base samples (20,480 evaluations) and the Jansen estimators of Equation (5), Modelly estimates from seed 1 the first-order indices $\hat{S} = (0.313, 0.442, -0.003)$ and the total indices $\hat{S}_T = (0.557, 0.442, 0.244)$ (`paper/data/ishigami.json`, written by `paper/scripts/ishigami.py`), and every analytical value lies inside the bootstrap interval of its estimate. The bootstrap resamples the rows of a scrambled Sobol design as if they were independent, which they are not, so we measured the uncertainty the proper way as well: by repeating the estimate on 32 independent scramblings of the sequence (seeds 1 to 32), at $N = 1{,}024$ and 4,096 (`paper/scripts/sobol_scramblings.py`, Table 11). At $N = 4{,}096$ the standard deviation of an index across scramblings is 0.0017 to 0.0046, the 2.5–97.5% range of the 32 estimates contains the analytical value for all six indices, and the row bootstrap's half-width is 3.1 to 5.9 times 1.96 times that standard deviation (3.5 to 6.8 times at $N = 1{,}024$): the row bootstrap is conservative here, overstating the randomised-quasi-Monte-Carlo uncertainty several-fold rather than understating it, and all 32 of its intervals contain the analytical value for every index. The scramblings' spread, not the row bootstrap, is the uncertainty we recommend reporting; Modelly keeps the row bootstrap as the cheap default (one design evaluated once) and a study that needs a calibrated interval should repeat the analysis over seeds, as this table does.

Table 11. Sobol indices of the Ishigami function over 32 independent scramblings of the design at two sample sizes: the mean and standard deviation of the 32 estimates and their 2.5–97.5% range (the randomised-QMC uncertainty), the mean half-width of the row bootstrap's 95% interval, its ratio to 1.96 standard deviations across scramblings, and how many of the 32 bootstrap intervals contain the analytical value. Generated by `paper/scripts/sobol_scramblings.py`.

N	Index	Exact	Mean	SD across	2.5–97.5% range	bootstrap ±	ratio	covered
1024	S_1	0.314	0.313	0.0076	[0.299, 0.325]	0.0596	4.00	32/32
	S_2	0.442	0.443	0.0069	[0.427, 0.456]	0.0479	3.53	32/32
	S_3	0.000	0.003	0.0074	[-0.007, 0.018]	0.0982	6.80	32/32
	S_{T1}	0.558	0.558	0.0087	[0.544, 0.576]	0.0842	4.97	32/32
	S_{T2}	0.442	0.442	0.0036	[0.435, 0.448]	0.0398	5.61	32/32
	S_{T3}	0.244	0.243	0.0037	[0.235, 0.249]	0.0264	3.63	32/32
4096	S_1	0.314	0.314	0.0030	[0.309, 0.319]	0.0294	5.06	32/32
	S_2	0.442	0.442	0.0039	[0.435, 0.448]	0.0235	3.10	32/32
	S_3	0.000	0.001	0.0046	[-0.005, 0.012]	0.0497	5.54	32/32
	S_{T1}	0.558	0.557	0.0037	[0.550, 0.561]	0.0423	5.88	32/32
	S_{T2}	0.442	0.442	0.0025	[0.436, 0.443]	0.0202	4.08	32/32
	S_{T3}	0.244	0.244	0.0017	[0.240, 0.245]	0.0135	4.17	32/32

Parameter recovery and coverage of ABC-SMC. Synthetic data are generated from a stochastic SIR model with $\beta = 0.4$ and $\gamma = 0.1$ and fitted with uniform priors. In the documentation's worked example (100 particles, six generations, 2,200 model runs, final tolerance 83.6), the posterior means are $\hat{\beta} = 0.423$ and $\hat{\gamma} = 0.098$, with 95% highest-density intervals $[0.33, 0.55]$ and $[0.067, 0.139]$. One dataset and one run are an anecdote, so we measured coverage at three true points per model, spread across the priors: for each of 200 replicates per point we simulated a fresh dataset from the true parameters (its own data seed) and calibrated it with the test suite's settings (30 particles for the SIR model, 40 for a stochastic predator-prey model with a log-uniform prior on the predation rate, six generations). These are the small settings that keep the tests fast, not recommended ones: with so few particles the highest-density intervals are coarse, so the coverage measures the method under the tests' budget. Table 12 gives how often the true value lies in the posterior's 95% highest-density interval.

Table 12. ABC-SMC at the test suite's budget (30 and 40 particles, below the 100 a study should use; see the text for coverage at 100) over 200 synthetic datasets at each of three true points per model: mean relative error of the posterior mean (signed and absolute), mean absolute error of the posterior median, and how often the true value lies in the 95% highest-density interval (HDI), with its Clopper–Pearson 95% interval [18]. Generated by `paper/scripts/recovery.py`.

		Mean		Median	In 95% HDI	
Model	Parameter	error	|error|	|error|	share	95% CI
SIR	$\beta = 0.4$	+1.8%	8.9%	9.1%	192/200	[0.92, 0.98]
SIR	$\gamma = 0.1$	+4.4%	8.6%	9.6%	200/200	[0.98, 1.00]
SIR	$\beta = 0.25$	+24.0%	24.1%	23.4%	188/200	[0.90, 0.97]
SIR	$\gamma = 0.05$	+81.2%	81.2%	79.9%	191/200	[0.92, 0.98]
SIR	$\beta = 0.6$	+1.2%	6.7%	6.8%	190/200	[0.91, 0.98]
SIR	$\gamma = 0.2$	+3.7%	5.6%	5.8%	199/200	[0.97, 1.00]
Predator-prey	growth = 0.5	+9.6%	12.5%	12.7%	198/200	[0.96, 1.00]
Predator-prey	predation = 0.01	+16.1%	18.6%	14.9%	196/200	[0.95, 0.99]
Predator-prey	growth = 0.35	+7.4%	9.2%	9.1%	199/200	[0.97, 1.00]
Predator-prey	predation = 0.006	+5.3%	11.1%	10.5%	199/200	[0.97, 1.00]
Predator-prey	growth = 0.7	−3.3%	7.3%	8.0%	198/200	[0.96, 1.00]
Predator-prey	predation = 0.02	+4.8%	13.7%	14.2%	198/200	[0.96, 1.00]

Nominal coverage is guaranteed only on average over parameters drawn from the prior; at

a fixed true point a well-calibrated posterior can cover more or less often. Coverage is 94 to 100% for a nominal 95% at all six true points (the lowest Clopper–Pearson bound is 0.90), so the intervals are not too narrow and, for γ at 0.1 and 0.2 and five of the six predator-prey parameters, conservative (lower bound at least 0.96), as expected of ABC: a non-zero tolerance widens the posterior. With the true parameters drawn from the prior instead, as simulation-based calibration does [115], 500 datasets per model (`paper/scripts/prior_coverage.py`, `paper/data/prior_coverage.json`), coverage is 94.2% [91.8, 96.1] for γ, 98.4% and 96.8% for the two predator-prey rates, and 91.2% [88.4, 93.5] for β, which is below nominal at these small test settings. At the 100 particles of the documented example (Listing 3; `prior_coverage.py -particles 100`), coverage is nominal or conservative for all four parameters: 95.8% [93.7, 97.4] for β, 96.4% [94.4, 97.9] for γ, 99.0% and 97.2% for the predator-prey rates.

That 100 particles suffice for these two models says nothing about a third: the particles a calibration needs depend on the number of parameters, the model's stochasticity, the summaries, the tolerance schedule and how well the data identify the parameters. We therefore recommend a protocol rather than a number, and run it here (`paper/scripts/abc_sweep.py`; Table 25 in the appendix): calibrate the same 500 prior-drawn datasets per model at increasing particle counts, 30, 60, 100 and 200, and watch the diagnostics converge: the width of the 95% interval, the posterior's effective sample size, the last generation's acceptance rate, and, on synthetic data, the coverage. On the SIR model β's coverage rises from 91.2% at 30 particles to 93.6% at 60, 95.8% at 100 and 95.2% at 200, and γ's from 94.2 to 96.4, 96.4 and 98.0%; the interval widens from 0.265 to 0.285, 0.294 and 0.297 for β as more particles resolve its tails, which is where the under-coverage at 30 came from: a 30-particle highest-density interval is too short, not misplaced. On the predator-prey model coverage is 94.8 to 99.8% at every count, nominal at 30 and conservative from 60 on, as expected of ABC with a non-zero tolerance. The effective sample size stays at 0.89 to 0.95 of the particles and the last acceptance rate at 17 to 19% at every count, so the kernel adapts rather than starves, and the cost grows linearly (625 to 4,032 runs for the SIR model). The interval's width stops changing between 100 and 200 particles (by 1%) for every parameter, which is the stopping rule we propose: run the ladder on the model at hand with synthetic data from the prior, and stop at the count beyond which the interval's width and the coverage no longer move; for these two models that is 100, the documented example's setting, and the test suite's 30 and 40 are below it. Accuracy depends on how well 30 steps identify the parameters. At four of the six points the mean signed error of the posterior mean is within 10% of the truth (its mean absolute error is 6 to 14%); for the predation rate at 0.01 they overestimate it by 16% (its posterior is right-skewed under the log-uniform prior; the median errs by 15%), and for the slow epidemic ($\beta = 0.25$, $\gamma = 0.05$, $R_0 = 5$, whose infections peak at the last of the 30 observed steps in 85% of the datasets, as `recovery.py analyse` reports) by 24% and 81%: the data say little about these values, the intervals are wide, and they still cover the truth 94 and 96% of the time. The test suite asserts recovery on three fixed seeds per model, with a 25% criterion on the posterior mean and the truth inside the interval; for the skewed predation rate it uses the median.

Importance weights. A dedicated test checks Equation (3) directly: from a population crowding the lower bound of a uniform prior, 40,000 proposals reweighted by their importance weights must be uniform within 8% in every bin next to the bound. It guards against a subtle error: moving a proposal that leaves the support again from the same ancestor makes the effective proposal density differ from the one in the weights, and over-weights the bin next to the bound by 17% in this test; Modelly redraws the ancestor as well [120]. Because Equation (3) is right only if the proposal is the mixture truncated as a whole, we also verified that the code's sampling and its weights agree with that density and not with the mixture of individually truncated kernels (`paper/scripts/abc_weights_check.py`, `data/abc_weights_check.json`). First, 400,000 proposals from a kernel fitted to a population crowding the bound were binned

over the 0.4 next to it (40 bins): their histogram agrees with the whole-mixture density (largest $|z|$ over the bins 2.7, χ^2 $p = 0.11$, first bin within 0.1%) and not with the per-ancestor one (largest $|z|$ 24, first bin 14% off), and the code's log weights equal $\log \pi - \log \sum_j w_j K_t$ to 10^{-15}. Second, on a model whose ABC posterior at a tolerance is known in closed form (the mean of 20 normal draws of known spread, a uniform prior on $[0, 1]$, the truth at 0.03 next to the bound), five generations of 2,000 particles give a weighted population whose Kolmogorov–Smirnov distance from the exact ABC posterior at the final tolerance is 0.016 (the Monte Carlo scale for its effective sample size is 0.031; mean 0.0507 against 0.0505, 5, 50 and 95% quantiles within 0.001); the same particles reweighted as the per-ancestor truncation would weight them are 0.030 away. The script also reads, from the objects the coverage studies build, the settings that ran: quantile 0.5, kernel covariance twice the weighted covariance plus the 10^{-9} ridge, choice-kernel stay probability 0.7, six generations, one replicate, the priors of Table 12; they are the ones this section describes.

Executor independence and resumption. Posteriors and Sobol indices are identical on the serial, thread and process executors, and an ABC-SMC run gives the same posterior with the compiled or the pure-Python core. A calibration stopped after a generation, or by its run budget, and resumed from its checkpoint reaches the same final posterior as an uninterrupted one.

Reference models. Each of the eighteen zoo models (Section 4.6) has a test against an analytical or well-established result; the six coupled or real-data ones are described with their criteria in Appendices G.6 and G.8. For the age-structured SEIR and Bass diffusion models the tolerance is derived from the statistic's sampling variance; others use fixed tolerances (for example 3% for the energy-budget population around its carrying capacity) or a heuristic expectation (the number of opinion clusters in the Deffuant model, about $1/(2\varepsilon)$). Each test checks one point or one regime of its model's parameter space, the one where an analytical result exists; it is evidence that the model is implemented as described there, not a validation over the parameter ranges a study might sweep, and the zoo's role in this paper is as tested starting points, not as evidence for the framework's claims. A forest fire translated from NetLogo by `modelly netlogo` burns 8, 15, 56, 89 and 97% of its trees at tree densities of 50, 55, 59, 63 and 70% (a 101×101 grid, mean of five seeds), the transition at the square lattice's site-percolation threshold [82] (`docs/netlogo.md`).

7 Scaling out and connecting to data

A study often outgrows one process, or needs its model tied to data: a population too large for one core, millions of agents whose phases are array operations, parameters to fit by gradient, a model kept in step with arriving observations, agents on a diffusing resource, a differential equation inside every agent, a model too expensive to run thousands of times, a realistic starting population, or a model that colleagues should try without installing anything. Modelly covers each of these with the same model code. For each capability this section gives the mechanism, what was measured and the conditions under which it holds; Table 13 collects the numbers, and Appendix G has the designs, the measurement conditions and the file behind every one of them. The measurements were made on the laptop of Section 5 while it was shared with other work and without the fixed frequency of Section 5.2 (except the timings of equations inside agents), and Section 9 says which are repeated runs and which are single runs; the conclusions that matter (byte-identical outputs, order-of-magnitude speed-ups) do not turn on the last digit of a timing. Each capability is presented as validated functionality with the performance evidence stated and bounded, not as a general acceleration claim: the GPU's 28-fold step speed-up holds at ten

Table 13. Scaling out and connecting to data: what was measured, and the conditions under which it holds. Details and sources in Appendix G.

Capability	Measured	Conditions
Split runs	Byte-identical to one process at 1, 2, 4 and 16 partitions on three models; MPI with 2 and 4 ranks; Dask with 2 workers	Weak scaling 68, 45, 17 and 7% at 2, 4, 8 and 16 workers with 10,000 people per worker, 74, 50 and 33% at 2, 4 and 8 with 40,000 (single runs on a shared laptop whose copies without communication already fall to 25% at eight); 64 workers unmeasured; MPI with 2 and 4 ranks on Linux only (Open MPI); macOS, Windows and Ray untested
GPU backend	Steps 28 times faster than the CPU column variant on one core at 10 million agents (123 times the same GPU variant on `numpy`); the GPU variant's integer state identical on `numpy` and CuPy	Construction not faster (1.0 to 1.7 times in total); slower at 20,000 agents, the crossover unmeasured; cohort tables 230 times slower; Metal untested
Differentiable models	Two zoo models' parameters recovered within 2% and 7%; pathwise gradients within 10^{-6} and 10^{-4} of finite differences	Gradients through discrete choices are approximations; the straight-through estimator is biased
Data assimilation	Nowcast error 5.07 against 69.2 unfiltered; contact rate 0.451 for 0.45; bit-identical for a seed	One dataset and one seed; brokers tested with fake clients; replicates step in one process
Fields and coupling	Mass conserved to 10^{-12}; 1D heat kernel to about 10^{-6}; two zoo models with closed budgets	Orthogonal grids with uniform spacing only; upwind advection smears fronts
Equations inside agents	200 of 200 systems take `solve_ivp`'s accepted steps and end within 1.9×10^{-11} of it; a delay equation equal to its closed form to 10^{-14}; strong orders 0.53 and 0.95 for orders $\frac{1}{2}$ and 1; 131 to 217 times faster than a `solve_ivp` loop at 10,000 agents, 46 times in the within-host zoo model	Only RK45 and RK23 are batched; stiff systems share one step; one stacked call is faster on alike agents; timings best of three runs at the fixed frequency
Emulators	Parameter box reduced from 17.1 to 2.5% in four waves, truth inside	Half the runs of a hypercube only on the zoo model (factors 1.3 to 2.7); Branin 1.1 to 1.4, Ishigami 0.8 to 1.4
Synthetic populations	Totals and ages exact, sexes within 1%; the SEIR model's final size within 0.05; three real-data zoo models	Made-up marginals; no real joint distribution to test against
Browser	64 of 64 runs: 32 models, two wheels	First load 6.5 and 9.0 s at 50 Mbit/s without and with `scipy`, 19.4 and 33.4 s on Fast 4G

million agents and shrinks to 1.0 to 1.7 once the agents are built, and the split run's byte-identical results are a correctness property whose multi-node speed-up has not been shown.

7.1 Per-agent random streams and split runs

A model can give every agent random generators of its own, seeded from the scenario's seed and the agent's key, so that what an agent draws no longer depends on the activation order or on the population (the flag is off by default). One run can then be split across local, MPI, Dask or Ray workers (a grid into blocks with halos, a network by breadth-first banding, a continuous space by recursive bisection), with ghost agents exchanged before a phase that reads across a border and movers migrating after a phase that moves them. On three reference models every reporter and every agent's state is byte-identical to the single-process run at 1, 2, 4 and 16 partitions,

under MPI with 2 and 4 ranks and on two Dask workers (Appendix G.1). The weak scaling of Table 13 comes from single runs on a laptop shared with other jobs, whose hybrid processor is the main limit: eight copies of the same work with no communication at all already run at 25% of one, because beyond the four performance cores the copies land on efficiency cores, and with 10,000 people per worker the protocol keeps 40 to 61% of what the machine gives from four workers on, 80 to 90% with 40,000 (Table 26). A split run on one laptop is therefore a check of correctness more than a speed-up; scaling to the 64 workers of a cluster was not measured, and the Ray backend is untested.

7.2 A GPU backend

Column phases, cohort tables and the cell-space operators run on CuPy or JAX with a `numpy` fallback, from counter-based (Philox) random streams that draw the same numbers on every backend. At ten million agents the grid epidemic's 60 steps take 0.80 s on an RTX 5050 laptop GPU against 20.5 s for the CPU column variant on one core (medians of five rounds), 28 times faster (median of the rounds' ratios, 25 to 37), and 123 times faster than the same GPU variant run on `numpy`. The GPU variant draws from Philox streams rather than the model's generator, so it is a different variant of the model from the CPU column one, agreeing with it in distribution; run on `numpy` and on CuPy it gives the same infected series and the same digest of the final state (Appendix G.2). The comparison is with one CPU core: a sweep of independent replicates can instead run one per core, which on this 16-core laptop would narrow the GPU's lead in throughput by up to that factor (not measured). Building ten million agent objects takes 23 to 35 s on either side (83 s for the GPU's first, cold build), so build plus steps gains only 1.0 to 1.7 times; at 20,000 agents the GPU is slower than the CPU (0.15 to 0.16 s against 0.07 s) and the crossover was not measured; a cohort table of under 200 rows is about 230 times slower, because kernel launches cost more than its arithmetic; and Apple Metal was not measured.

7.3 Differentiable models

Column phases traced by JAX give gradients of a model's outputs with respect to its parameters, exact through normal and uniform draws and approximate through Bernoulli choices (straight-through, Gumbel-softmax or score-function estimators, each warning that it is one). Gradient calibration recovers two zoo models' parameters from runs of the agent-based originals, and an ODE inside every agent is differentiated through the steps of a fourth-order Runge–Kutta integrator, which recovers the two viral rates of a within-host model within 0.1% (Section 7.6); and pathwise gradients agree with finite differences (Table 13, Appendix G.3); the straight-through estimator is biased and fails that check, by design, so a gradient through a discrete choice is an approximation whose use the documentation delimits.

7.4 Live data and data assimilation

Observations arrive from a replayed table, a growing CSV file, an HTTP endpoint or a message broker; an ensemble of replicates advances in lock step, a bootstrap particle filter [34] weights and resamples it, and an ensemble Kalman filter [31] adjusts continuous parameters, both bit-identical for a seed. On the tutorial's synthetic town the filtered nowcast of the unobserved number infected has a root-mean-square error of 5.07 people against 69.2 for the unfiltered ensemble, and the Kalman filter recovers the contact rate as 0.451 for a truth of 0.45 (Appendix G.5); that is one dataset and one seed, not a coverage study. Message brokers were tested against fake clients only, and the replicates step in one process.

7.5 Fields and coupling

A field is a property layer with a solver (diffusion, upwind advection and decay, in explicit substeps that refuse an unstable time step, or backward Euler), coupled to differential equations and stock-flow components under a schedule that is checked when the model is built. Closed and periodic boundaries conserve mass to a relative 10^{-12} and a spreading Gaussian matches the one-dimensional heat kernel to about 10^{-6}; two zoo models, foragers on a diffusing nutrient and an epidemic coupled to hospital capacity, close their budgets to round-off and match their analytical checks (Appendix G.6). Fields need orthogonal grids with uniform spacing, and upwind advection smears sharp fronts.

7.6 Differential equations inside agents

When every agent carries an ODE (a viral load inside a host, a physiology, an energy budget), one `solve_ivp` call per agent costs a Python call per agent and step, and stacking the agents into one system forces them all onto the step size of the agent that needs the smallest. `solve_batch` integrates many independent systems at once: it runs SciPy's Dormand–Prince and Bogacki–Shampine pairs [13, 24, 123] with SciPy's step selection and error control applied to each system separately, and evaluates the right-hand side once per stage for all of them. On 200 random systems of a viral model, with both pairs at two tolerances, every system takes exactly the accepted steps of its own `solve_ivp` call and ends within a relative 1.9×10^{-11} of it at the looser tolerance and 6×10^{-15} at the tighter (`paper/data/odes.json`). `AgentODE` keeps the state in an agent class's columns, locates zero crossings per agent on each step's interpolant (an onset, a clearance), calls back into the model when one occurs, and hands stiff systems to BDF, Radau or LSODA as one stacked call with a block-diagonal Jacobian pattern. The same machinery integrates delay equations by the method of steps, keyed by agent so that newcomers keep their own past (equal to the closed form of $y'(t) = -a\,y(t-1)$ to a relative 6×10^{-15} at model steps of 0.5, 1 and 3), and stochastic equations by Euler–Maruyama or a derivative-free Milstein scheme with diagonal noise [63], drawn from the model's generator, whose strong orders on geometric Brownian motion measure 0.53 and 0.95 against the theoretical $\frac{1}{2}$ and 1. A mean-field companion runs a model's ODE limit beside it and fits how the gap shrinks with the population: as $N^{-0.47}$ on the Bass zoo model between 250 and 16,000 households, the $N^{-1/2}$ of sampling noise.

Timed against the two alternatives on the target-cell-limited model of influenza [8], three equations per agent over ten days with the Dormand–Prince pair at a relative tolerance of 10^{-6} (best of three runs, one for the loop beyond 1,000 agents, on one core at the fixed frequency of Section 5.2), `solve_batch` integrates 10,000 agents in 0.25 s where one `solve_ivp` call per agent takes 54 s, 217 times as long, and 131 times as long on a population whose rates span orders of magnitude, with the same worst error as the loop at every size (`paper/data/ode_compare.json`). One stacked `solve_ivp` call is faster when the agents are alike (0.17 s at 10,000 agents; 2.9 s against 3.4 s at 100,000), since it controls one step size instead of one per agent; on the diverse population it is level at 10,000 agents and 1.6 times slower at 100,000 (7.3 s against 4.6 s), since every agent takes the steps of the one that needs the smallest, which also makes it the more accurate there. In the within-host zoo model, 100,000 hosts at the epidemic's peak on day 20, `AgentODE` advances the hosts by one day, events included, in 1.07 s against 49.1 s for one `solve_ivp` call per exposed host, 46 times faster in one run of each, and the two final states agree to 4×10^{-9} of the tolerance (`paper/data/ode_within_host.json`).

7.7 Emulators, active learning and history matching

A Gaussian-process or neural emulator trained on an experiment's runs, active learning that picks the next runs by integrated variance reduction, and history matching in waves [5, 122]. On the age-structured SEIR model four waves of 20 runs reduce the parameter box to 2.5%,

with the true parameters inside after every wave (Appendix G.7). Active learning saves the runs it was meant to save on the zoo model and not on the Branin and Ishigami functions, where a well-spread design is already close to the best that a variance criterion blind to the outputs can do.

7.8 Synthetic populations and models on real data

Iterative proportional fitting, integerisation, households, places within a distance limit and a contact network with household, school and work layers build a starting population from aggregate tables; no personal data is bundled, and three small real samples (OpenStreetMap streets, ESA WorldCover land cover, census tracts) ship with their licences. The synthetic population reproduces its marginals and starts the SEIR model within its final-size check, and three zoo models run on the real samples against known answers (Appendix G.8); the marginals are made up, so no real joint distribution was tested against.

7.9 Modelly in the browser

`modelly export-web` writes a static site that runs a model with Pyodide in a browser tab, with Studio's parameter panel, a space view and a reporter plot. The 14 bundled examples and the 18 zoo models each ran from their export in headless Chromium with the pure-Python wheel and with the compiled one, 64 of 64 runs (Appendix G.9). A cold first visit is ready after 6.5 s on a 50 Mbit/s line for a model that needs no `scipy`, 9.0 s for one that does, and 19.4 and 33.4 s on a Fast 4G profile, so a first load under ten seconds is reached on the faster line, with or without `scipy`, and not on Fast 4G: Pyodide, `numpy` and `pandas` alone are 13.6 MB.

8 Studies that Modelly serves

Modelly is built for studies that need many runs, large populations, or results that others must be able to check. Table 14 lists these situations, the part of the framework that does the work, and the evidence for it in this paper.

A population too large for one process, millions of agents whose phases are array operations, a model kept in step with arriving data, agents on a diffusing resource, or an expensive model and few runs are the situations of Section 7.

Starting points by field. Each of the eighteen reference models comes with its validation test and can be copied into a new project with `python -m modelly.zoo copy`, so a study starts from a working, tested model rather than a blank file. An epidemiologist starts from the age-structured SEIR model, fits its rates to case counts with ABC-SMC, ranks the interventions by Sobol index and narrows the parameter space by history matching; or from the epidemic coupled to hospital capacity, Ross–Macdonald vector-borne transmission or the within-host model, whose hosts carry a viral ODE, and compares vaccination strategies on common random numbers. An ecologist starts from the energy-budget population, the foragers on a diffusing nutrient or habitat selection on real land cover, and runs a stage-structured population as cohorts over a factorial of scenarios. An economist starts from kinetic wealth exchange, the Bouchaud–Mézard model or the zero-intelligence order book, and sweeps exchange rules over a Latin hypercube on a cluster. A social scientist starts from Schelling's model on census tracts, Axelrod's culture model or the Deffuant and Hegselmann–Krause opinion models on a synthetic population's household, school and work networks. A transport researcher starts from Nagel–Schreckenberg traffic or from traffic assignment on a city's street graph. In each case the model stays ordinary Python, and the study's runs, fit and sensitivity analysis share one seed and one record that a reviewer can rerun with `modelly verify`.

Table 14. Research situations, the part of Modelly that serves each, and the evidence.

Situation	What helps
Large populations whose phases act on all agents alike (movement, infection, ageing, mortality)	Columns turn those phases into array operations: 2.9 to 33 times faster than Mesa 3.5 and 4.6 to 42 times faster than Mesa 2.3 on the playbooks (Section 5.2).
Populations with many identical individuals (insects, cohorts of patients, age classes)	Cohort tables with binomial draws keep the individual model's law under the assumptions of the proposition of Section 3.2 (independent, homogeneous, synchronous phases) at a fraction of the cost: on the insect model, 131 times faster than its individual-based version on Mesa 3.5 (a change of representation, which any framework could adopt; Modelly provides it built in).
Parameter sweeps, replicates and cluster runs	Experiments with resumable stores and executors from a laptop to SLURM, with results independent of the executor (Section 4).
Fitting a model to data; knowing which parameters matter	ABC-SMC, simulated moments, Morris, Sobol and PAWN on the same seeded engine, checked on known answers, with a particle-count protocol for ABC-SMC and scrambling-based uncertainty for Sobol (Section 6.2).
Choosing between interventions, and explaining the choice	Strategies compared on common random numbers with intervals, win shares, cost-effectiveness and minimax regret, the pairing's variance reduction measured for the study; attribution to agents and counterfactuals on recorded runs (Section 4.3).
Agents with internal dynamics (a viral load, a physiology, an energy budget)	An ODE, delay or stochastic equation in every agent, each agent taking the steps of its own `solve_ivp` call (Section 7.6).
A paper whose results a reviewer must be able to rerun	Run manifests, `modelly verify`, byte-identical bundles for archiving, ODD skeletons, model cards tied to tests, a hub that checks hashes, golden digests identical under three CPython versions, compiled and pure, on one Linux machine (Table 10).
An existing Mesa model that has become too slow	Migrate it with the codemod (the same statistics as Mesa 2.3 on every seed tried for the two Mesa 2.3 models migrated, equivalent ones from Mesa 3.5; 1.04 to 2.2 times faster than Mesa 3.5), then move the hot phases to columns one at a time.
A model others should try without installing anything	A static export that runs the model in a browser tab (Section 7.9).
A team that works in R or Julia	Packages that run, inspect and calibrate a model from either language (Section 4).

9 Threats to validity and limitations

9.1 Threats to the validity of the measurements

The baselines. The playbooks compare against Mesa 2.3.2 and Mesa 3.5.1 (3.5.2 followed on 28 October 2026); Mesa 2.3 runs here on CPython 3.15, which it does not officially support. The Mesa 3.5 and mesa-frames implementations were written for this paper, following each library's documentation and examples, and their developers might write them faster. mesa-frames was measured from its development branch (commit `ef92326`, reporting version 0.1.1.dev0, polars 2.0, on one thread), on four of the eight workloads; writing the others needs features it lacks (it has no continuous or network space), and working around three defects we met (neighbourhoods on a torus, the removal of agents, and neighbourhoods of radius above one) made its code longer than its examples. AgentPy, Agents.jl and NetLogo have not been measured, and the speed-ups

should not be read as applying to them.

Framework or algorithm. The playbook speed-ups measure what a modeller gains by writing each model in the form the framework offers, not the framework's overhead alone, and part of a gain can be algorithmic: the Mesa flocking implementation scans every agent for neighbours, as Mesa's continuous space does, while the Modelly one queries a KD-tree. Section 5.3 separates these by changing one thing per rung on four workloads; the KD-tree baseline there gives the same statistics as Mesa 3.5's own search on all 110 seeds compared (`paper/data/flocking_kdtree.json`). What the ablation does not separate is the vectorised rewrite of a phase from the column storage that makes it possible, since the one requires the other; the Mesa 3.5 + numpy baseline, which has the arrays without the framework, bounds that from the other side.

What the timings include. The timed region is the whole simulation, including the data collection and the summary statistics of each workload. In the Boltzmann wealth model the data collector (the Gini coefficient and the wealth of each of 10,000 agents, every step) is a large part of every implementation's run, so that workload measures data collection as much as the exchange; all implementations compute the Gini coefficient with `numpy`, the per-agent Mesa ones after gathering the agents' wealth in a Python loop. The summaries computed after the last step are a small part of each run (about 2% of Mesa's Schelling run).

Who wrote the implementations. Every implementation of every playbook, the Mesa ones included, was written by the author of Modelly, and no independent party has yet reviewed them or rerun the benchmarks. Each Mesa version follows Mesa's documented API and examples, and all of them are in the repository for inspection. One baseline invites a closer look: our Mesa 3.5 predator-prey, which follows Mesa's own examples (`CellAgent` on an `OrthogonalMooreGrid`, grass in a property layer), runs slower than Mesa 2.3's (0.92 times); we have not profiled why, so the columns' 6.9 times over Mesa 3.5 there may be larger than a tuned Mesa 3.5 model would allow. What a third party can check without rerunning the timings is that every table and figure regenerates from the published data and that every listing runs as printed: `paper/scripts/verify_artifacts.py` and `paper/scripts/check_listings.py` do both, and both pass at the commit of this version (Appendix A). The one comparison whose baselines we did not write is that of Mesa 3.5's own examples (Table 2), whose Mesa arm runs Mesa's code unmodified; it measures the same-code gain, not the gain of the faster forms, which no Mesa-written model can show.

Measurement. The timings of the playbooks, the migrated models and Mesa's examples come from one laptop with a hybrid processor, pinned to one performance core (CPU 2, one of the processor's four 4.7 GHz cores; the other twelve are efficiency cores), under the `intel_pstate` driver with the `performance` governor and turbo disabled, so the core ran at its 1.9 GHz base frequency. Absolute times are therefore longer than on a turbo-enabled machine; the ratios are what we report. The core was pinned but not isolated: other work shared the machine (one-minute load average 2.3 to 3.9 at the start of each workload of the main timings, 4.2 at the start of the size sweep), and `taskset` pins the benchmark, not the other processes, so under load they can run on the same core; CPU time excludes their time but not their cost in context switches, and they compete for the shared cache and memory bandwidth, which may weigh more on the memory-bound array code than on interpreter-bound per-agent code. The order of the implementations rotated from round to round: every per-round ratio over Mesa 2.3 lies within 17% of its median (over Mesa 3.5, within 17%), for 28 of the 29 non-reference variants within 10% (the exception is the insect population's column form, 16%), and the bootstrap intervals of Table 16 are a few percent wide (42.1 [41.7, 42.5] for the grid epidemic's columns

over Mesa 2.3). Those are within-session intervals; between sessions we timed all eight playbooks twice, in two sessions on the same core and settings (`paper/data/playbooks_session1/` and Table 3), and every one of the 50 speed-ups the two sessions share agrees within 7% (median 1.2%). The conclusions rest on that one machine: other processors, with other cache sizes and vector units, will give other ratios. The other measurements of Section 7 were made on the same laptop without the fixed frequency: the GPU comparison has five rounds and the browser's first load is a median of three, while the weak scaling (Table 26), the 100-million-row run and the nowcast are single runs; the claims that carry weight there (identical outputs, a 28-fold speed-up, shortfalls by a factor) do not turn on a few percent.

One machine, three interpreters, few sizes. Every timing of Table 3 ran on CPython 3.15 on one laptop; Section 5.2 repeats the Mesa 3.5 and column arms on CPython 3.12 and 3.14 on the same machine, where the speed-ups hold to within the session's noise, but no second processor was measured. The ablation, the interpreter comparison and the end-to-end sweep (Sections 5.2 to 5.3), and the differential equations of Section 7.6, ran in the same session as the main timings, at the same fixed 1.9 GHz on the same core, but each in its own script with its own rounds, so their ratios are paired within each script. The playbooks are timed at one population size each, and three of them at three sizes (Table 5), where the columns' lead grows with the population; the other five were not timed at other sizes, and the GPU measurement shows how strongly a speed-up can depend on the size (Section 7.2).

Agreement. The agreement study (Section 5.4) compares chosen summary statistics; the trajectory study (Section 5.5) adds whole curves, spatial clusters, neighbourhood composition, time to converge, extinction probability and cycle period on three playbooks, not on all eight and not at the timed size. Six small differences are individually detectable at the small size, and one more (the columns' epidemic peak, 3.3 steps earlier) in the trajectory study; that none survives the correction is weak evidence on its own, since at the corrected level a paired test on 100 seeds detects $d = 0.33$ with probability 0.13 and $d = 0.5$ with probability 0.52 for uncorrelated pairs (more where the shared initial state correlates them), which is why the paper reports intervals and equivalence tests rather than the absence of significant differences. At the timed size the study uses 50 seeds (200 for the grid epidemic), whose intervals are wider. An equivalence margin of half a standard deviation is a convention, not a scientific tolerance: it is a medium effect by Cohen's scale, the margin is estimated from the same data, which makes the test slightly liberal, stricter margins give fewer equivalences (27 of 67 at $0.3\,s_p$, 19 at $0.25\,s_p$), and the Holm-corrected count (29 of 67) is the one that controls the familywise error. The science-specific margins of the trajectory study are prespecified but ours: another modeller may need a tighter tolerance on a peak's timing than ± 2 steps, and at that margin 100 seeds do not decide the question for any implementation, Mesa 3.5 included.

Reproducibility. For this version the determinism suite, the split-run checks, MPI and the exact digests ran on one machine, Linux x86-64, under CPython 3.12, 3.14 and 3.15, compiled and pure (Table 10); on it every example gives one exact digest under all six builds. macOS, Windows and ARM were not run beyond the release workflow's import of each wheel, nor were the published wheels, so the bitwise and even the ten-digit agreement across operating systems and architectures is untested for this version, as are MS-MPI, bundles verified across platforms, the free-threaded wheels and the free-threaded build of the event heap, the Ray backend and Apple Metal; the golden digests cover ten steps of seventeen examples (the fourteen bundled examples and three split-run models, distinct from the zoo). Seven of these pass through transcendental functions or BLAS reductions whose last bits depend on the platform's libraries, so their exact digests can be expected to differ on other platforms: a study that needs bitwise agreement across machines must avoid these operations, or state its guarantee at ten significant

digits, as the suite does. The suite has not been run for more than ten steps.

A young project. Modelly was first released on 24 October 2026, by one developer, who is so far the only maintainer; the other projects of Table 1 have been in use for between two and twenty-seven years. Every public name carries a stability level, stable or provisional (`docs/api_stability.toml`); the research workflow and the capabilities of Section 7 are recent, and the latter have been used by their tests and tutorials, not yet by a study outside the repository. The project's governance is set out in its repository: issues and pull requests on GitHub, with a contribution guide (`CONTRIBUTING.md`), a code of conduct and a security policy; a stable name keeps working for at least one minor release after it is deprecated, with a warning naming its replacement, and is removed only in a major release (`docs/deprecations.md`), which no release has yet been, so the notice before a removal is counted in major releases; releases are tagged after lint, type checks and the test suite pass. The plugin interface (its entry-point groups, the `Plugin` class and its loader) is frozen for the 1.x series, and `modelly migrate -check-2.0` lists, with the line, every change a project must make before 2.0.

9.2 Limitations of the software

- **Speed comes from the model's form.** A model written agent by agent gains only where the framework's own calls take its time (1.05 to 2.2 times over Mesa 3.5, 2.6 and 4.8 over Mesa 2.3, Section 5.1); the large gains of Section 5.2 require phases that can be expressed over the whole population. Sequential decisions where each agent depends on the previous one's move remain per-agent loops.
- **Scaling out has conditions.** A split run needs per-agent streams and a model that meets the contract of Section 7.1 (phases with a declared radius, no model-level draws, no column or dataset agents, no events of its own), and on one laptop it gains little; the GPU backend is slower at 20,000 agents and faster at ten million, the crossover unmeasured, and does not pay for cohort tables or for construction; gradients through discrete choices are approximations; fields need orthogonal grids with uniform spacing; the Ray backend and Apple Metal are untested.
- **Equations and pairing.** Only the RK45 and RK23 pairs integrate each agent with its own steps; stiff systems go through one stacked call whose step all agents share, and the stochastic schemes take diagonal noise only. Common random numbers reduce the variance of a comparison only while the strategies consume the random numbers in the same number and order; an intervention that changes either (a new cohort row, a different branch) loses the gain, which `treatment_effect` measures rather than assumes. Language-model agents reproduce a run only from their cache of replies.
- **Checkpoints and closures.** Checkpoints pickle the model; models that hold lambdas, nested functions, classes created at run time or open files cannot be saved: lambdas and nested functions are refused with an error naming them, other unpicklable objects fail with pickle's own error. Checkpoints load only under the Modelly and the Python minor versions that wrote them.

10 Conclusion

Modelly gives Python modellers the agent-based programming model they know and reduces what it costs to do research with it. A model keeps its agents as objects and has its population-wide phases written as array operations, one at a time; on eight workloads those phases run 2.9 to 33 times faster than per-agent Mesa 3.5 code, the lead growing with the population on two of the three workloads timed at three sizes, within 1.8 to 4.7 times of hand-written numpy

that gives agents up, and counted populations two orders of magnitude faster; an ablation attributes the gain to the array form of the phases. Whether the faster forms are the same model is tested: against the author's Mesa 2.3 reference implementations, 29 of 67 statistic–variant comparisons are shown equivalent within half a standard deviation over 100 paired seeds after Holm correction (63 at nominal significance; 48 shown equivalent within ±5% of Mesa's mean), and on three workloads the whole curves, the spatial patterns and the times to converge of the faster forms are as close to Mesa 2.3 as Mesa 3.5's are. What this establishes is close agreement on the dynamics tested, at the tolerances stated for each quantity, not a general scientific equivalence across all behaviours: 34 of 52 scalar quantities are shown equivalent after Holm correction, 17 are undecided at 100 seeds, and one difference is shown, the column form's epidemic peak 3.3 steps earlier than Mesa 2.3's ($[-6.6, -0.1]$), so a study for which a peak's timing matters to that precision should set its own tolerance and test it as Section 5.5 does; and Mesa's own examples, migrated with the codemod, give all 35 of their statistics within the same margin, and the virus example's migrated copy is the same process as Mesa's, draw for draw, once the random streams are aligned (Section 5.1). Around that model, the steps of a study (sweeps, calibration, sensitivity analysis, emulation, the comparison of interventions on common random numbers, provenance, model cards and archiving) are part of the framework, usable from Python, R and Julia, built on one seeding discipline and checked against problems whose answers are known, and the reproducibility guarantees are stated by scope: deterministic on one build, and bit-identical for all seventeen examples across CPython 3.12, 3.14 and 3.15, compiled or pure, on the one machine (Linux x86-64) where this version was checked; other operating systems and architectures are untested for it. The same model splits across processes with byte-identical results, runs its array phases on a GPU (with a bounded gain once construction is counted), connects to live data and continuous fields, carries an ordinary, delay or stochastic differential equation in every agent with each agent's own solver steps, and can be shared as a web page (Table 13); these capabilities are validated functionality with the performance evidence of Table 13, not general acceleration claims. For a researcher who needs many runs, large populations or results that others can check, Modelly brings the tools usually assembled by hand around a simulation library into one package, one seed and one record. The conditions under which these results hold, and the evidence still to gather, are set out in Section 9.

11 Availability

`pip install modelly-abm` installs Modelly (Python 3.12 or later), which is imported as `modelly` and is free software under the Apache 2.0 licence. The release workflow builds compiled wheels for Linux, macOS and Windows (x86-64 and ARM64) on the tag and a pure-Python wheel for everything else (Table 10). Source, issue tracker, the tests and the benchmark harness are at github.com/RemiPelloux/Modelly; documentation, tutorials and the roadmap at remipelloux.github.io/Modelly; the R and Julia packages in the repository's `r/modelly` and `julia/Modelly.jl` folders. At this version, the suite runs 3,274 tests, of which 3,215 pass and 59 skip without an optional library (CuPy, Ray, optuna, xarray, a broker) or a browser, with 97.2% line coverage (`paper/data/test_suite.json`). The version described here is 1.5.0; the provenance of every measurement (commits, machine, package versions and the Software Heritage identifier of the archived code) is in Appendix A, and the data and scripts behind every table and figure of this paper are in the repository's `paper/` folder. To cite Modelly, use the metadata in the repository's `CITATION.cff`.

Declarations

Software availability. Name: Modelly (`modelly-abm` on PyPI). Developer: Rémi Pelloux, OpenPro (rp@openpro.ai). First available: 2026. Version described: 1.5.0. Language: Python,

with five C extension modules. Requirements: CPython 3.12 or later, `numpy`, `pandas`, `scipy` and `tqdm` (and `pythran` on macOS); optional extras for visualisation, geography, networks, experiment storage, Dask and Ray executors, MPI (`mpi4py`), GPUs (CuPy), differentiable models (JAX), message brokers, an outside optimiser, a policy search (Optuna) and xarray; R and Julia packages that call it. Hardware: any machine that runs CPython; the release workflow builds compiled wheels for Linux, macOS and Windows on x86-64 and ARM64 and a pure-Python wheel for everything else and for the browser. Size: 1.32 MB for the Linux x86-64 wheel and 1.27 MB for the pure one. Licence: Apache 2.0. Source: github.com/RemiPelloux/Modelly.

Data availability. The data and scripts behind every measured table and figure are in the repository's `paper/` and `benchmarks/` folders (Appendix A).

Funding. The author received no specific grant for this work.

Competing interests. The author develops Modelly, the software evaluated here.

References

[1] Sameera Abar, Georgios K. Theodoropoulos, Pierre Lemarinier, and Gregory M. P. O'Hare. Agent based modelling and simulation tools: A review of the state-of-art software. *Computer Science Review*, 24:13–33, 2017. doi: 10.1016/j.cosrev.2017.03.001.

[2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.

[3] Adam Amer and mesa-frames contributors. mesa-frames. Software repository, 2026. URL https://github.com/mesa/mesa-frames. Development commit ef92326 of 6 October 2026 (version 0.1.1.dev0), accessed 2026-10-25.

[4] Philipp Andelfinger. Differentiable agent-based simulation for gradient-guided simulation-based optimization. In *Proceedings of the 2021 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, pages 27–38, 2021. doi: 10.1145/3437959.3459261.

[5] Ioannis Andrianakis, Ian R. Vernon, Nicky McCreesh, Trevelyan J. McKinley, Jeremy E. Oakley, Rebecca N. Nsubuga, Michael Goldstein, and Richard G. White. Bayesian history matching of complex infectious disease models using emulation: a tutorial and a case study on HIV in Uganda. *PLoS Computational Biology*, 11(1):e1003968, 2015. doi: 10.1371/journal.pcbi.1003968.

[6] Robert Axelrod. The dissemination of culture: A model with local convergence and global polarization. *Journal of Conflict Resolution*, 41(2):203–226, 1997. doi: 10.1177/0022002797041002001.

[7] Robert Axtell, Robert Axelrod, Joshua M. Epstein, and Michael D. Cohen. Aligning simulation models: A case study and results. *Computational and Mathematical Organization Theory*, 1(2):123–141, 1996. doi: 10.1007/BF01299065.

[8] Prasith Baccam, Catherine Beauchemin, Catherine A. Macken, Frederick G. Hayden, and Alan S. Perelson. Kinetics of influenza A virus infection in humans. *Journal of Virology*, 80(15):7590–7599, 2006. doi: 10.1128/JVI.01623-05.

[9] Frank M. Bass. A new product growth for model consumer durables. *Management Science*, 15(5):215–227, 1969. doi: 10.1287/mnsc.15.5.215.

[10] Mark A. Beaumont, Wenyang Zhang, and David J. Balding. Approximate Bayesian computation in population genetics. *Genetics*, 162(4):2025–2035, 2002. doi: 10.1093/genetics/162.4.2025.

[11] Mark A. Beaumont, Jean-Marie Cornuet, Jean-Michel Marin, and Christian P. Robert. Adaptive approximate Bayesian computation. *Biometrika*, 96(4):983–990, 2009. doi: 10.1093/biomet/asp052.

[12] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. arXiv preprint, 2013. URL https://arxiv.org/abs/1308.3432. arXiv:1308.3432.

[13] P. Bogacki and L. F. Shampine. A 3(2) pair of Runge–Kutta formulas. *Applied Mathematics Letters*, 2(4):321–325, 1989. doi: 10.1016/0893-9659(89)90079-7.

[14] Jean-Philippe Bouchaud and Marc Mézard. Wealth condensation in a simple model of economy. *Physica A: Statistical Mechanics and its Applications*, 282(3–4):536–545, 2000. doi: 10.1016/S0378-4371(00)00205-3.

[15] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs. Software repository, 2018. URL https://github.com/jax-ml/jax. Version 0.12.0 used.

[16] Morton B. Brown and Alan B. Forsythe. Robust tests for the equality of variances. *Journal of the American Statistical Association*, 69(346):364–367, 1974. doi: 10.1080/01621459.1974.10482955.

[17] Ayush Chopra, Alexander Rodríguez, Jayakumar Subramanian, Arnau Quera-Bofarull, Balaji Krishnamurthy, B. Aditya Prakash, and Ramesh Raskar. Differentiable agent-based epidemiology. In *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 1848–1857, 2023.

[18] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404.

[19] Nicholson T. Collier, Jonathan Ozik, and Eric R. Tatara. Experiences in developing a distributed agent-based modeling toolkit with Python. In *2020 IEEE/ACM 9th Workshop on Python for High-Performance and Scientific Computing (PyHPC)*, pages 1–12, 2020. doi: 10.1109/PyHPC51966.2020.00006.

[20] George Datseris, Ali R. Vahdati, and Timothy C. DuBois. Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity. *Simulation*, 100 (10):1019–1031, 2024. doi: 10.1177/00375497211068820.

[21] Guillaume Deffuant, David Neau, Frédéric Amblard, and Gérard Weisbuch. Mixing beliefs among interacting agents. *Advances in Complex Systems*, 3(01n04):87–98, 2000. doi: 10.1142/S0219525900000078.

[22] W. Edwards Deming and Frederick F. Stephan. On a least squares adjustment of a sampled frequency table when the expected marginal totals are known. *The Annals of Mathematical Statistics*, 11(4):427–444, 1940. doi: 10.1214/aoms/1177731829.

[23] L. C. W. Dixon and G. P. Szegő, editors. *Towards Global Optimisation 2*. North-Holland, Amsterdam, 1978.

[24] J. R. Dormand and P. J. Prince. A family of embedded Runge–Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980. doi: 10.1016/0771-050X(80)90013-3.

[25] Randal Douc and Olivier Cappé. Comparison of resampling schemes for particle filtering. In *Proceedings of the 4th International Symposium on Image and Signal Processing and Analysis (ISPA)*, pages 64–69, 2005. doi: 10.1109/ISPA.2005.195385.

[26] Adrian Drăgulescu and Victor M. Yakovenko. Statistical mechanics of money. *The European Physical Journal B*, 17(4):723–729, 2000. doi: 10.1007/s100510070114.

[27] B. Drossel and F. Schwabl. Self-organized critical forest-fire model. *Physical Review Letters*, 69(11):1629–1632, 1992. doi: 10.1103/PhysRevLett.69.1629.

[28] Joel Dyer, Patrick Cannon, J. Doyne Farmer, and Sebastian M. Schmon. Black-box Bayesian inference for agent-based models. *Journal of Economic Dynamics and Control*, 161:104827, 2024. doi: 10.1016/j.jedc.2024.104827.

[29] Joshua M. Epstein and Robert L. Axtell. *Growing Artificial Societies: Social Science from the Bottom Up*. Brookings Institution Press and MIT Press, Washington, DC and Cambridge, MA, 1996. doi: 10.7551/mitpress/3374.001.0001.

[30] Geir Evensen. Sequential data assimilation with a nonlinear quasi-geostrophic model using Monte Carlo methods to forecast error statistics. *Journal of Geophysical Research: Oceans*, 99(C5):10143–10162, 1994. doi: 10.1029/94JC00572.

[31] Geir Evensen. The ensemble Kalman filter: theoretical formulation and practical implementation. *Ocean Dynamics*, 53(4):343–367, 2003. doi: 10.1007/s10236-003-0036-9.

[32] J. Doyne Farmer, Paolo Patelli, and Ilija I. Zovko. The predictive power of zero intelligence in financial markets. *Proceedings of the National Academy of Sciences*, 102 (6):2254–2259, 2005. doi: 10.1073/pnas.0409157102.

[33] Joël Foramitti. AgentPy: A package for agent-based modeling in Python. *Journal of Open Source Software*, 6(62):3065, 2021. doi: 10.21105/joss.03065.

[34] N. J. Gordon, D. J. Salmond, and A. F. M. Smith. Novel approach to nonlinear/non-Gaussian Bayesian state estimation. *IEE Proceedings F: Radar and Signal Processing*, 140(2):107–113, 1993. doi: 10.1049/ip-f-2.1993.0015.

[35] Volker Grimm, Eloy Revilla, Uta Berger, Florian Jeltsch, Wolf M. Mooij, Steven F. Railsback, Hans-Hermann Thulke, Jacob Weiner, Thorsten Wiegand, and Donald L. DeAngelis. Pattern-oriented modeling of agent-based complex systems: Lessons from ecology. *Science*, 310(5750):987–991, 2005. doi: 10.1126/science.1116681.

[36] Volker Grimm, Uta Berger, Finn Bastiansen, Sigrunn Eliassen, Vincent Ginot, Jarl Giske, John Goss-Custard, Tamara Grand, Simone K. Heinz, Geir Huse, et al. A standard protocol for describing individual-based and agent-based models. *Ecological Modelling*, 198(1–2):115–126, 2006. doi: 10.1016/j.ecolmodel.2006.04.023.

[37] Volker Grimm, Uta Berger, Donald L. DeAngelis, J. Gary Polhill, Jarl Giske, and Steven F. Railsback. The ODD protocol: A review and first update. *Ecological Modelling*, 221(23):2760–2768, 2010. doi: 10.1016/j.ecolmodel.2010.08.019.

[38] Volker Grimm, Jacqueline Augusiak, Andreas Focks, Béatrice M. Frank, Faten Gabsi, Alice S. A. Johnston, Chun Liu, Benjamin T. Martin, Mattia Meli, Viktoriia Radchuk, Pernille Thorbek, and Steven F. Railsback. Towards better modelling and decision support: Documenting model development, testing, and analysis using TRACE. *Ecological Modelling*, 280:129–139, 2014. doi: 10.1016/j.ecolmodel.2014.01.018.

[39] Volker Grimm, Steven F. Railsback, Christian E. Vincenot, Uta Berger, Cara Gallagher, Donald L. DeAngelis, Bruce Edmonds, Jiaqi Ge, Jarl Giske, Jürgen Groeneveld, et al. The ODD protocol for describing agent-based and other simulation models: A second update to improve clarity, replication, and structural realism. *Journal of Artificial Societies and Social Simulation*, 23(2):7, 2020. doi: 10.18564/jasss.4259.

[40] J. H. Halton. On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerische Mathematik*, 2(1):84–90, 1960. doi: 10.1007/BF01386213.

[41] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[42] J. L. Hart, A. Alexanderian, and P. A. Gremaud. Efficient computation of Sobol' indices for stochastic models. *SIAM Journal on Scientific Computing*, 39(4):A1514–A1530, 2017. doi: 10.1137/16M106193X.

[43] Florian Hartig, Justin M. Calabrese, Björn Reineking, Thorsten Wiegand, and Andreas Huth. Statistical inference for stochastic simulation models – theory and application. *Ecology Letters*, 14(8):816–827, 2011. doi: 10.1111/j.1461-0248.2011.01640.x.

[44] Rainer Hegselmann and Ulrich Krause. Opinion dynamics and bounded confidence: Models, analysis and simulation. *Journal of Artificial Societies and Social Simulation*, 5 (3):2, 2002. URL https://www.jasss.org/5/3/2.html.

[45] Jon Herman and Will Usher. SALib: An open-source Python library for sensitivity analysis. *Journal of Open Source Software*, 2(9):97, 2017. doi: 10.21105/joss.00097.

[46] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL https://www.jstor.org/stable/4615733.

[47] Toshimitsu Homma and Andrea Saltelli. Importance measures in global sensitivity analysis of nonlinear models. *Reliability Engineering & System Safety*, 52(1):1–17, 1996. doi: 10.1016/0951-8320(96)00002-6.

[48] Wolfgang Hörmann. The generation of binomial random variates. *Journal of Statistical Computation and Simulation*, 46(1–2):101–110, 1993. doi: 10.1080/00949659308811496.

[49] T. Ishigami and T. Homma. An importance quantification technique in uncertainty analysis for computer models. In *Proceedings of the First International Symposium on Uncertainty Modeling and Analysis*, pages 398–403. IEEE Computer Society Press, 1990. doi: 10.1109/ISUMA.1990.151285.

[50] Takuya Iwanaga, William Usher, and Jonathan Herman. Toward SALib 2.0: Advancing the accessibility and interpretability of global sensitivity analyses. *Socio-Environmental Systems Modelling*, 4:18155, 2022. doi: 10.18174/sesmo.18155.

[51] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-softmax. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1611.01144. arXiv:1611.01144.

[52] Michiel J. W. Jansen. Analysis of variance designs for model output. *Computer Physics Communications*, 117(1–2):35–43, 1999. doi: 10.1016/S0010-4655(98)00154-4.

[53] Marco A. Janssen. The practice of archiving model code of agent-based models. *Journal of Artificial Societies and Social Simulation*, 20(1):2, 2017. doi: 10.18564/jasss.3317.

[54] Marc Jaxa-Rozen and Jan H. Kwakkel. PyNetLogo: Linking NetLogo with Python. *Journal of Artificial Societies and Social Simulation*, 21(2):4, 2018. doi: 10.18564/jasss.3668.

[55] Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4): 455–492, 1998. doi: 10.1023/A:1008306431147.

[56] Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, pages 63–74, 2013. doi: 10.1145/2464157.2464160.

[57] George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998. doi: 10.1137/S1064827595287997.

[58] Jackie Kazil, David Masad, and Andrew Crooks. Utilizing Python for agent-based modeling: The Mesa framework. In Robert Thomson, Halil Bisgin, Christopher Dancy, Ayaz Hyder, and Muhammad Hussain, editors, *Social, Cultural, and Behavioral Modeling (SBP-BRiMS 2020)*, volume 12268 of *Lecture Notes in Computer Science*, pages 308–317, Cham, 2020. Springer. doi: 10.1007/978-3-030-61255-9_30.

[59] William Ogilvy Kermack and A. G. McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A*, 115(772): 700–721, 1927. doi: 10.1098/rspa.1927.0118.

[60] Diederik P. Kingma and Jimmy Ba. Adam: a method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. URL https://arxiv.org/abs/1412.6980. arXiv:1412.6980.

[61] Leslie Kish. *Survey Sampling*. John Wiley & Sons, New York, 1965.

[62] Emmanuel Klinger, Dennis Rickert, and Jan Hasenauer. pyABC: distributed, likelihood-free inference. *Bioinformatics*, 34(20):3591–3593, 2018. doi: 10.1093/bioinformatics/bty361.

[63] Peter E. Kloeden and Eckhard Platen. *Numerical Solution of Stochastic Differential Equations*. Springer, Berlin, Heidelberg, 1992. doi: 10.1007/978-3-662-12616-5.

[64] Jan H. Kwakkel. The Exploratory Modeling Workbench: An open source toolkit for exploratory modeling, scenario discovery, and (multi-objective) robust decision making. *Environmental Modelling & Software*, 96:239–250, 2017. doi: 10.1016/j.envsoft.2017.06.054.

[65] Daniël Lakens. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4):355–362, 2017. doi: 10.1177/1948550617697177.

[66] Ju-Sung Lee, Tatiana Filatova, Arika Ligmann-Zielinska, Behrooz Hassani-Mahmooei, Forrest Stonedahl, Iris Lorscheid, Alexey Voinov, J. Gary Polhill, Zhanli Sun, and Dawn C. Parker. The complexities of agent-based modeling output analysis. *Journal of Artificial Societies and Social Simulation*, 18(4):4, 2015. doi: 10.18564/jasss.2897.

[67] Robin Lovelace and Dimitris Ballas. 'truncate, replicate, sample': a method for creating integer weights for spatial microsimulation. *Computers, Environment and Urban Systems*, 41:1–11, 2013. doi: 10.1016/j.compenvurbsys.2013.03.004.

[68] Sean Luke, Claudio Cioffi-Revilla, Liviu Panait, Keith Sullivan, and Gabriel Balan. MASON: A multiagent simulation environment. *Simulation*, 81(7):517–527, 2005. doi: 10.1177/0037549705058073.

[69] Charles M. Macal and Michael J. North. Tutorial on agent-based modelling and simulation. *Journal of Simulation*, 4(3):151–162, 2010. doi: 10.1057/jos.2010.3.

[70] David J. C. MacKay. Information-based objective functions for active data selection. *Neural Computation*, 4(4):590–604, 1992. doi: 10.1162/neco.1992.4.4.590.

[71] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: a continuous relaxation of discrete random variables. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1611.00712. arXiv:1611.00712.

[72] Nick Malleson, Kevin Minors, Le-Minh Kieu, Jonathan A. Ward, Andrew West, and Alison Heppenstall. Simulating crowds in real time with agent-based modelling and a particle filter. *Journal of Artificial Societies and Social Simulation*, 23(3):3, 2020. doi: 10.18564/jasss.4266.

[73] David Masad and Jacqueline Kazil. Mesa: An agent-based modeling framework. In *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, pages 51–58, 2015. doi: 10.25080/Majora-7b98e3ed-009.

[74] Jiří Matoušek. On the L_2-discrepancy for anchored boxes. *Journal of Complexity*, 14(4): 527–556, 1998. doi: 10.1006/jcom.1998.0489.

[75] Makoto Matsumoto and Takuji Nishimura. Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1):3–30, 1998. doi: 10.1145/272991.272995.

[76] Daniel McFadden. A method of simulated moments for estimation of discrete response models without numerical integration. *Econometrica*, 57(5):995–1026, 1989. doi: 10.2307/1913621.

[77] M. D. McKay, R. J. Beckman, and W. J. Conover. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 21(2):239–245, 1979. doi: 10.1080/00401706.1979.10489755.

[78] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596.

[79] Max D. Morris. Factorial sampling plans for preliminary computational experiments. *Technometrics*, 33(2):161–174, 1991. doi: 10.1080/00401706.1991.10484804.

[80] Birgit Müller, Friedrich Bohn, Gunnar Dreßler, Jürgen Groeneveld, Christian Klassert, Romina Martin, Maja Schlüter, Jule Schulze, Hanna Weise, and Nina Schwarz. Describing human decisions in agent-based models – ODD+D, an extension of the ODD protocol. *Environmental Modelling & Software*, 48:37–48, 2013. doi: 10.1016/j.envsoft.2013.06.003.

[81] Kai Nagel and Michael Schreckenberg. A cellular automaton model for freeway traffic. *Journal de Physique I*, 2(12):2221–2229, 1992. doi: 10.1051/jp1:1992277.

[82] M. E. J. Newman and R. M. Ziff. Efficient Monte Carlo algorithm and high-precision results for percolation. *Physical Review Letters*, 85(19):4104–4107, 2000. doi: 10.1103/PhysRevLett.85.4104.

[83] Michael J. North, Nicholson T. Collier, Jonathan Ozik, Eric R. Tatara, Charles M. Macal, Mark Bragen, and Pam Sydelko. Complex adaptive systems modeling with Repast Simphony. *Complex Adaptive Systems Modeling*, 1(1):3, 2013. doi: 10.1186/2194-3206-1-3.

[84] Jeremy E. Oakley and Anthony O'Hagan. Probabilistic sensitivity analysis of complex models: A Bayesian approach. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 66(3):751–769, 2004. doi: 10.1111/j.1467-9868.2004.05304.x.

[85] Ryosuke Okuta, Yuya Unno, Daisuke Nishino, Shohei Hido, and Crissman Loomis. CuPy: a NumPy-compatible library for NVIDIA GPU calculations. In *Proceedings of the Workshop on Machine Learning Systems (LearningSys) at the 31st Conference on Neural Information Processing Systems (NIPS)*, 2017. URL https://github.com/cupy/cupy. Version 14.2.0 used.

[86] Art B. Owen. Scrambling Sobol' and Niederreiter–Xing points. *Journal of Complexity*, 14(4):466–489, 1998. doi: 10.1006/jcom.1998.0487.

[87] Jonathan Ozik, Nicholson T. Collier, Justin M. Wozniak, and Carmine Spagnuolo. From desktop to large-scale model exploration with Swift/T. In *2016 Winter Simulation Conference (WSC)*, pages 206–220, 2016. doi: 10.1109/WSC.2016.7822090.

[88] Francesca Pianosi and Thorsten Wagener. A simple and efficient method for global sensitivity analysis based on cumulative distribution functions. *Environmental Modelling & Software*, 67:1–11, 2015. doi: 10.1016/j.envsoft.2015.01.004.

[89] Francesca Pianosi and Thorsten Wagener. Distribution-based sensitivity analysis from a generic input-output sample. *Environmental Modelling & Software*, 108:197–207, 2018. doi: 10.1016/j.envsoft.2018.07.019.

[90] Donovan Platt. A comparison of economic agent-based model calibration methods. *Journal of Economic Dynamics and Control*, 113:103859, 2020. doi: 10.1016/j.jedc.2020.103859.

[91] Arnau Quera-Bofarull, Joel Dyer, Anisoara Calinescu, J. Doyne Farmer, and Michael Wooldridge. BlackBIRDS: Black-box inference for differentiable simulators. *Journal of Open Source Software*, 8(89):5776, 2023. doi: 10.21105/joss.05776.

[92] Steven F. Railsback, Steven L. Lytinen, and Stephen K. Jackson. Agent-based simulation platforms: Review and development recommendations. *Simulation*, 82(9): 609–623, 2006. doi: 10.1177/0037549706073695.

[93] Romain Reuillon, Mathieu Leclaire, and Sebastien Rey-Coyrehourcq. OpenMOLE, a workflow engine specifically tailored for the distributed exploration of simulation models. *Future Generation Computer Systems*, 29(8):1981–1990, 2013. doi: 10.1016/j.future.2013.05.003.

[94] Craig W. Reynolds. Flocks, herds and schools: A distributed behavioral model. *ACM SIGGRAPH Computer Graphics*, 21(4):25–34, 1987. doi: 10.1145/37402.37406.

[95] Paul Richmond, Robert Chisholm, Peter Heywood, Mozhgan Kabiri Chimeh, and Matthew Leach. FLAME GPU 2: A framework for flexible and performant agent based simulation on GPUs. *Software: Practice and Experience*, 53(8):1659–1680, 2023. doi: 10.1002/spe.3207.

[96] Pamphile T. Roy, Art B. Owen, Maximilian Balandat, and Matt Haberland. Quasi-Monte Carlo methods in Python. *Journal of Open Source Software*, 8(84):5309, 2023. doi: 10.21105/joss.05309.

[97] Jan Salecker, Marco Sciaini, Katrin M. Meyer, and Kerstin Wiegand. The nlrx R package: A next-generation framework for reproducible NetLogo model analyses. *Methods in Ecology and Evolution*, 10(11):1854–1863, 2019. doi: 10.1111/2041-210X.13286.

[98] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: as easy as 1, 2, 3. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, pages 1–12, 2011. doi: 10.1145/2063384.2063405.

[99] Andrea Saltelli. Making best use of model evaluations to compute sensitivity indices. *Computer Physics Communications*, 145(2):280–297, 2002. doi: 10.1016/S0010-4655(02)00280-1.

[100] Andrea Saltelli, Marco Ratto, Terry Andres, Francesca Campolongo, Jessica Cariboni, Debora Gatelli, Michaela Saisana, and Stefano Tarantola. *Global Sensitivity Analysis: The Primer*. John Wiley & Sons, Chichester, 2008. doi: 10.1002/9780470725184.

[101] Andrea Saltelli, Paola Annoni, Ivano Azzini, Francesca Campolongo, Marco Ratto, and Stefano Tarantola. Variance based sensitivity analysis of model output. Design and estimator for the total sensitivity index. *Computer Physics Communications*, 181(2): 259–270, 2010. doi: 10.1016/j.cpc.2009.09.018.

[102] L. J. Savage. The theory of statistical decision. *Journal of the American Statistical Association*, 46(253):55–67, 1951. doi: 10.1080/01621459.1951.10500768.

[103] Yannik Schälte, Emmanuel Klinger, Emad Alamoudi, and Jan Hasenauer. pyABC: Efficient and robust easy-to-use approximate Bayesian computation. *Journal of Open Source Software*, 7(74):4304, 2022. doi: 10.21105/joss.04304.

[104] M. Scheffer, J. M. Baveco, D. L. DeAngelis, K. A. Rose, and E. H. van Nes. Super-individuals a simple solution for modelling large populations on an individual basis. *Ecological Modelling*, 80(2–3):161–170, 1995. doi: 10.1016/0304-3800(94)00055-M.

[105] Thomas C. Schelling. Dynamic models of segregation. *The Journal of Mathematical Sociology*, 1(2):143–186, 1971. doi: 10.1080/0022250X.1971.9989794.

[106] Donald J. Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657–680, 1987. doi: 10.1007/BF01068419.

[107] L. S. Shapley. A value for n-person games. In H. W. Kuhn and A. W. Tucker, editors, *Contributions to the Theory of Games, Volume II*, number 28 in Annals of Mathematics Studies, pages 307–318. Princeton University Press, 1953. doi: 10.1515/9781400881970-018.

[108] Richard M. Sibly, Volker Grimm, Benjamin T. Martin, Alice S. A. Johnston, Katarzyna Kułakowska, Christopher J. Topping, Peter Calow, Jacob Nabe-Nielsen, Pernille Thorbek, and Donald L. DeAngelis. Representing the acquisition and use of energy by individuals in agent-based models of animal populations. *Methods in Ecology and Evolution*, 4(2):151–161, 2013. doi: 10.1111/2041-210x.12002.

[109] S. A. Sisson, Y. Fan, and Mark M. Tanaka. Sequential Monte Carlo without likelihoods. *Proceedings of the National Academy of Sciences*, 104(6):1760–1765, 2007. doi: 10.1073/pnas.0607208104.

[110] S. A. Sisson, Y. Fan, and M. A. Beaumont, editors. *Handbook of Approximate Bayesian Computation*. Chapman and Hall/CRC, Boca Raton, FL, 2018. doi: 10.1201/9781315117195.

[111] David L. Smith, Katherine E. Battle, Simon I. Hay, Christopher M. Barker, Thomas W. Scott, and F. Ellis McKenzie. Ross, Macdonald, and a theory for the dynamics and control of mosquito-transmitted pathogens. *PLoS Pathogens*, 8(4):e1002588, 2012. doi: 10.1371/journal.ppat.1002588.

[112] I. M. Sobol'. On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics*, 7(4):86–112, 1967. doi: 10.1016/0041-5553(67)90144-9.

[113] Ilya M. Sobol'. Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates. *Mathematics and Computers in Simulation*, 55(1–3):271–280, 2001. doi: 10.1016/S0378-4754(00)00270-6.

[114] Patrick Taillandier, Benoit Gaudou, Arnaud Grignard, Quang-Nghi Huynh, Nicolas Marilleau, Philippe Caillou, Damien Philippon, and Alexis Drogoul. Building, composing and experimenting complex spatial models with the GAMA platform. *GeoInformatica*, 23(2):299–322, 2019. doi: 10.1007/s10707-018-00339-6.

[115] Sean Talts, Michael Betancourt, Daniel Simpson, Aki Vehtari, and Andrew Gelman. Validating Bayesian inference algorithms with simulation-based calibration. arXiv preprint arXiv:1804.06788, 2018. URL https://arxiv.org/abs/1804.06788.

[116] Guus ten Broeke, George van Voorn, and Arend Ligtenberg. Which sensitivity analysis method should I use for my agent-based model? *Journal of Artificial Societies and Social Simulation*, 19(1):5, 2016. doi: 10.18564/jasss.2857.

[117] Ewout ter Hoeven, Jan Kwakkel, Vincent Hess, Thomas Pike, Boyu Wang, rht, and Jackie Kazil. Mesa 3: Agent-based modeling with Python in 2025. *Journal of Open Source Software*, 10(107):7668, 2025. doi: 10.21105/joss.07668.

[118] The Pyodide developers. Pyodide: Python with the scientific stack, compiled to WebAssembly. https://pyodide.org, 2026. Version 314.0.7, as loaded by `modelly export-web`.

[119] Jan C. Thiele, Winfried Kurth, and Volker Grimm. Facilitating parameter estimation and sensitivity analysis of agent-based models: A cookbook using NetLogo and 'R'. *Journal of Artificial Societies and Social Simulation*, 17(3):11, 2014. doi: 10.18564/jasss.2503.

[120] Tina Toni, David Welch, Natalja Strelkowa, Andreas Ipsen, and Michael P. H. Stumpf. Approximate Bayesian computation scheme for parameter inference and model selection in dynamical systems. *Journal of the Royal Society Interface*, 6(31):187–202, 2009. doi: 10.1098/rsif.2008.0172.

[121] Elske van der Vaart, Mark A. Beaumont, Alice S. A. Johnston, and Richard M. Sibly. Calibration and evaluation of individual-based models using approximate Bayesian computation. *Ecological Modelling*, 312:182–190, 2015. doi: 10.1016/j.ecolmodel.2015.05.020.

[122] Ian Vernon, Michael Goldstein, and Richard G. Bower. Galaxy formation: a Bayesian uncertainty analysis. *Bayesian Analysis*, 5(4):619–669, 2010. doi: 10.1214/10-BA524.

[123] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[124] Boyu Wang, Vincent Hess, and Andrew Crooks. Mesa-Geo: A GIS extension for the Mesa agent-based modeling framework in Python. In *Proceedings of the 5th ACM SIGSPATIAL International Workshop on GeoSpatial Simulation*, pages 1–10, 2022. doi: 10.1145/3557989.3566157.

[125] Jonathan A. Ward, Andrew J. Evans, and Nicolas S. Malleson. Dynamic calibration of agent-based models using data assimilation. *Royal Society Open Science*, 3(4):150703, 2016. doi: 10.1098/rsos.150703.

[126] Bernard L. Welch. The generalization of 'Student's' problem when several different population variances are involved. *Biometrika*, 34(1–2):28–35, 1947. doi: 10.1093/biomet/34.1-2.28.

[127] Uri Wilensky. NetLogo. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL, 1999. URL https://ccl.northwestern.edu/netlogo/.

[128] Uri Wilensky and William Rand. Making models match: Replicating an agent-based model. *Journal of Artificial Societies and Social Simulation*, 10(4):2, 2007. URL https://www.jasss.org/10/4/2.html.

[129] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992. doi: 10.1007/BF00992696.

[130] Songmin Yu and Zhanyi Hou. Melodie: Agent-based modeling in Python. *Journal of Open Source Software*, 8(83):5100, 2023. doi: 10.21105/joss.05100.

[131] Daniele Zanaga, Ruben Van De Kerchove, Dirk Daems, Wanda De Keersmaecker, Carsten Brockmann, Grit Kirches, Jan Wevers, Oliver Cartus, Maurizio Santoro, Steffen Fritz, Myroslava Lesiv, Martin Herold, Nandin-Erdene Tsendbazar, Panpan Xu, Fabrizio Ramoino, and Olivier Arino. ESA WorldCover 10 m 2021 v200. Zenodo, 2022. URL https://doi.org/10.5281/zenodo.7254221.

A Reproducing the measurements

Every measured table and figure of this paper is generated from data and scripts in the repository. The raw output of the agreement study, the GPU comparison, the weak-scaling runs, the nowcasting tutorial and the Ishigami estimate is in the paper's `data/` folder, and the playbook timings are in `benchmarks/playbooks/results/`.

```
git clone https://github.com/RemiPelloux/Modelly && cd Modelly
python3.15 -m venv .venv && .venv/bin/pip install -e ".[dev]"   # CPython 3.15
PINS="numpy==2.5.4 pandas==3.0.6 networkx==3.7"     # the versions of the measurements
.venv/bin/pip install "mesa==3.5.1" $PINS scipy==1.18.1  # Mesa 3.5 runs next to Modelly
python3.15 -m venv .venv-mesa2 && .venv-mesa2/bin/pip install "mesa==2.3.2" $PINS
python3.15 -m venv .venv-frames && .venv-frames/bin/pip install beartype $PINS \
    "mesa-frames @ git+https://github.com/mesa/mesa-frames@ef92326"

# the eight playbooks (Mesa 2.3, Mesa 3.5, + numpy, mesa-frames, Modelly), tables, figure
.venv/bin/python benchmarks/playbooks/compare.py --core 2 --rounds 10  # a P-core
.venv/bin/python paper/scripts/make_figures.py
.venv/bin/python paper/scripts/bootstrap_ci.py   # intervals of the speed-ups
# the agreement study at the small size: 2,100 runs, then its tables and figure
.venv/bin/python paper/scripts/agreement.py run --size small --seeds 100 --jobs 8
.venv/bin/python paper/scripts/agreement.py analyse --size small
# the same at the timed size, 50 seeds
.venv/bin/python paper/scripts/agreement.py run --size large --seeds 50 --jobs 8
.venv/bin/python paper/scripts/agreement.py analyse --size large
# Mesa 3.5 and mesa-frames against Mesa 2.3
.venv/bin/python paper/scripts/agreement.py run --variants mesa3 frames --tag small_baselines
.venv/bin/python paper/scripts/agreement.py analyse --tag small_baselines
# replications on new seeds: the grid epidemic at the timed size, Schelling's model
NEW="--seed-base 10000"
.venv/bin/python paper/scripts/agreement.py run $NEW --size large --seeds 200 \
    --playbooks grid_epidemic --tag large_grid200_new
.venv/bin/python paper/scripts/agreement.py run $NEW --seeds 300 --playbooks schelling \
    --variants mesa3 frames modelly modelly-cells --tag schelling300_new
.venv/bin/python paper/scripts/agreement.py analyse --tag large_grid200_new
.venv/bin/python paper/scripts/agreement.py analyse --tag schelling300_new
# Mesa 3.5 + numpy against Mesa 2.3, at the small size and the timed size
NP="--variants mesa3np --playbooks grid_epidemic schelling boltzmann_wealth"
.venv/bin/python paper/scripts/agreement.py run $NP --tag small_mesa3np
.venv/bin/python paper/scripts/agreement.py run --size large --variants mesa3np \
    --playbooks grid_epidemic --tag large_grid_mesa3np
.venv/bin/python paper/scripts/agreement.py analyse --tag small_mesa3np
.venv/bin/python paper/scripts/agreement.py analyse --tag large_grid_mesa3np
# the same model code on both frameworks (Mesa 2.3 and Mesa 3.5); mesa-frames threads
.venv/bin/python paper/scripts/migrated.py --rounds 10 --core 2
.venv/bin/python paper/scripts/frames_threads.py
# Mesa's own examples on Mesa and, migrated by the codemod, on Modelly; then their table
.venv/bin/python benchmarks/mesa_examples/run.py --rounds 10 --seeds 100 --core 2 \
    --agree-cores 8-15
.venv/bin/python paper/scripts/mesa_examples_table.py
# ABC-SMC coverage over 200 datasets per model
.venv/bin/python paper/scripts/recovery.py run --replicates 200 --jobs 8
.venv/bin/python paper/scripts/recovery.py analyse
# coverage with the true parameters drawn from the prior; speed-ups at three sizes
.venv/bin/python paper/scripts/prior_coverage.py run --replicates 500 --jobs 8
.venv/bin/python paper/scripts/prior_coverage.py run --replicates 500 --particles 100
# the KD-tree flocking against Mesa 3.5 run by run; the virus example on 300 new seeds
.venv/bin/python paper/scripts/kdtree_equality.py --small 100 --large 10
.venv/bin/python benchmarks/mesa_examples/run.py --only virus_on_network --rounds 1 \
    --seeds 300 --seed-base 10000 --output paper/data/mesa_examples_virus300_new.json
.venv/bin/python paper/scripts/sizes.py run --rounds 5 --core 2
# further studies: trajectories and patterns (1,200 runs), the ablation ladder,
# three CPython versions (.venv-py312 and .venv-py314 built like .venv), a sweep end to end
```

```
.venv/bin/python paper/scripts/trajectories.py run --seeds 100 --jobs 8
.venv/bin/python paper/scripts/trajectories.py analyse
.venv/bin/python paper/scripts/ablation.py run --rounds 10 --core 2
.venv/bin/python paper/scripts/python_versions.py run --rounds 5 --core 2
.venv/bin/python paper/scripts/sweep_e2e.py run --rounds 5 --core 2
# ABC-SMC coverage and diagnostics at four particle counts; Sobol across 32 scramblings;
# the cohort proposition's positive case and counterexamples
.venv/bin/python paper/scripts/abc_sweep.py run --particles 30 60 100 200 --replicates 500
.venv/bin/python paper/scripts/sobol_scramblings.py
.venv/bin/python paper/scripts/cohort_counterexample.py
.venv/bin/python paper/scripts/abc_weights_check.py   # the SMC weights vs their equation
.venv/bin/python paper/scripts/virus_trace.py --seeds 10  # the virus example, level by level
.venv/bin/python benchmarks/mesa_examples/run.py --only virus_on_network --rounds 1 \
    --seeds 1000 --seed-base 20000 --output paper/data/mesa_examples_virus1000_new.json
# exact digests under CPython 3.12, 3.14 and 3.15, compiled and pure, and the
# determinism and split-run suites under each (.venv-py312 and .venv-py314 built like .venv)
bash paper/measure/measure_150b.sh
.venv/bin/python paper/scripts/local_digests.py
# the artifact checks: every table and figure from the published data; every listing
.venv/bin/python paper/scripts/verify_artifacts.py
.venv/bin/python paper/scripts/check_listings.py
# determinism, calibration and sensitivity checks
.venv/bin/python -m pytest tests/determinism tests/calibrate tests/sensitivity
# scaling out and connecting to data: split runs, assimilation, coupling, emulators, ...
.venv/bin/python -m pytest tests/distributed tests/stream tests/couple tests/emulate \
    tests/population tests/netlogo tests/diff    # tests/diff needs jax
.venv/bin/python benchmarks/distributed/weak_scaling.py      # weak scaling (MPI, Dask)
.venv/bin/python benchmarks/emulate/active_vs_lhs.py         # active learning vs LHS
.venv/bin/python benchmarks/large_columns.py --rows 100000000  # under a memory limit
# the GPU run needs a CUDA environment (docs/gpu.md): .gpu-venv with cupy-cuda13x
.venv/bin/python benchmarks/playbooks/gpu_speedup.py --equality --n 10000000 --side 1000
.venv/bin/python benchmarks/playbooks/gpu_speedup.py --n 10000000 --side 1000 --rounds 3
# the browser: every model from its export, then the first-load measurement
MODELLY_BROWSER_TESTS=1 .venv/bin/python -m pytest tests/web/test_browser.py
.venv/bin/python tools/web/measure_load.py site/
```

Artifact checks. Two scripts let a reader check the paper's artifacts without rerunning its measurements. `verify_artifacts.py` copies `tables/` and `figures/` aside, regenerates every table and figure from the data files with the scripts above (the `analyse` commands and the table scripts), reruns the five scripts that recompute a data file from the code (the Ishigami estimate, the scramblings, the cohort counterexample, the graph cut, the nowcast), and compares: the 28 tables of `tables/` byte for byte (21 of them in the paper, the other seven the full agreement tables kept as data), the 9 figures pixel for pixel, the 5 data files on their numbers. `check_listings.py` extracts the three Python listings of the paper and runs each as printed (the ABC listing against the model module of `docs/calibration.md`, which it writes from the documentation's own code blocks); the ABC listing's output is the posterior quoted in Section 6.2. Both pass at the commit of this version, in the development environment and from a fresh clone in a fresh environment whose dependencies were resolved from PyPI at the time (CPython 3.14.8, numpy 2.5.4, scipy 1.18.1, pandas 3.1.1, matplotlib 3.11.2; `data/clean_checkout_check.json`), where every table, figure and data file regenerated identically and the ABC listing gave the same posterior to four decimals. These are the author's runs, not an independent party's. One pitfall for a reader: the editable install of the listing above compiles the C modules into the checkout; a plain `pip install .` run from the repository root imports the checkout's uncompiled sources rather than the installed wheel, so install editable, or run from another directory. What the two scripts do not do is rerun a timing, an agreement study or the digests under several interpreters, which the commands above do.

The playbook runner starts each run in a fresh interpreter, pins it to one core with `taskset`,

alternates the implementations over rounds, checks determinism, and writes JSON results and a Markdown report to `benchmarks/playbooks/results/`. The methods, and the rules a fair comparison must follow, are described in `docs/performance.md`.

Provenance of the measurements. Every measurement of this paper ran on 4 November 2026 from commit `6cd4ae6`, whose `modelly` package is identical to the tagged release v1.5.0 (the commit differs from the tag only in documentation and the documentation site's test), compiled on the measurement machine with GCC 16.2, by `paper/measure/measure_150.sh`. One session ran, in this order: at the fixed frequency, the first timing session of every playbook (`paper/data/playbooks_session1/`), the migrated models, Mesa's own examples, the mesa-frames threads, the three sizes, the ablation, the end-to-end sweep, the three CPython versions, the ODE timings and the second timing session of every playbook, which gives Table 3; then, at the machine's default frequency policy, the GPU comparison over five rounds, the weak scaling, the agreement and trajectory studies, the run-by-run comparison of the KD-tree implementation with Mesa 3.5, the virus example's trace and replications on new seeds, the calibration studies, the data scripts, the 100-million-row run, the active-learning comparison, the GPU and differentiable tests, the browser runs with the pure and the WebAssembly wheel, the browser's first load and the test suite. A second session the same day, `paper/measure/measure_150b.sh`, wrote the exact digests and ran the determinism and distributed suites under three interpreters (Section 6). Each result file records what it ran with: the playbook, migrated-model, example and agreement files the commit, the machine, the load and the package versions; `assimilation.json`, `ishigami.json`, `graph_cut.json` and the other data scripts' files the commit, date, interpreter and package versions, their `+dirty` marking the benchmark results written earlier in the session, not code; the `weak_*.json` files the interpreter and the platform; and `gpu_speedup.json` the package versions of both arms. The tagged release v1.5.0 is archived by Software Heritage as `swh:1:rel:7b7b49f3d26e8411797bb8a29e3c83eff3b98222`.

B The workloads

Choices in the implementations. For flocking the Mesa 3.5 implementation uses the legacy continuous space of `mesa.space`, which Mesa 3.5 still provides and which runs this workload 1.6 times faster than the experimental space of Mesa's own example. Modelly's cell-list variants keep the data structure of Mesa's grids, each cell holding the list of its agents, and vectorise only the obvious phases. Where a phase searches cells one agent at a time, as Sugarscape's foragers do, the Mesa implementations and the cell-list variant use the same single-pass algorithm, so that their comparison is between data structures; the column and mesa-frames variants find every forager's best visible cell in one array pass and keep only the walk to a free cell sequential. The four mesa-frames workloads were also timed in wall-clock time with one polars thread and with all 16 (`paper/scripts/frames_threads.py`, `paper/data/frames_threads.json`): with all threads the grid epidemic takes 1.68 s against 1.36 s, Boltzmann wealth 1.54 s against 1.35 s and Sugarscape 3.61 s against 2.88 s, while Schelling's model is 1.5 times faster (1.81 s against 2.63 s); these are single runs, unpinned, while other jobs loaded the machine (one-minute load average 7.6 at their start), which penalises the multi-threaded arm. Pinned to the four performance cores, five alternating rounds each (`frames_threads.py -cores 0-3`, `paper/data/frames_threads_pinned.json`), four threads were faster than one on every workload: 1.15 s against 1.19 s on the grid epidemic, 1.28 s against 2.11 s on Schelling's model, 0.93 s against 1.13 s on Boltzmann wealth and 2.00 s against 2.41 s on Sugarscape. Threads therefore pay on Schelling's model and, on cores of their own, modestly on the other three, whose data frames gain little (4% on the grid epidemic); under load they cost more than they gain on those three.

Table 15. The eight playbooks: steps per run, the size parameters at the small size (agreement study) and the large size (timings), and the model parameters shared by every implementation. Generated from `benchmarks/playbooks/registry.py` by `paper/scripts/make_figures.py`.

Workload	Steps	Small / large size	Parameters
Grid epidemic (SIR)	60	`n` = 2,000, `side` = 60 / `n` = 20,000, `side` = 180	`beta` = 0.25, `recovery` = 10, `initially_infected` = 20
Predator-prey	200	`side` = 80, `sheep` = 240, `wolves` = 16 / `side` = 200, `sheep` = 1,500, `wolves` = 100	`sheep_reproduce` = 0.04, `wolf_reproduce` = 0.03, `sheep_gain` = 4, `wolf_gain` = 40, `regrowth` = 20
Flocking (boids)	50	`n` = 500, `side` = 100 / `n` = 4,000, `side` = 300	`vision` = 10, `speed` = 1, `separation` = 2, `cohere` = 0.03, `separate` = 0.015, `match` = 0.05
Network contagion	100	`n` = 2,000 / `n` = 30,000	`avg_degree` = 6, `beta` = 0.15, `gamma` = 0.1, `resistance` = 0.5, `outbreak` = 0.01
Insect population	120	`initial_adults` = 100, `capacity` = 5,000 / `initial_adults` = 500, `capacity` = 50,000	`egg_days` = 3, `larva_days` = 8, `pupa_days` = 10, `maturity_days` = 5, `survival` = {egg 0.9, larva 0.95, pupa 0.97, adult 0.95}, `lay_prob` = 0.5, `clutch` = 8, `sterile_ratio` = 1
Schelling segregation	40	`side` = 40 / `side` = 200	`density` = 0.8, `minority` = 0.3, `homophily` = 3
Boltzmann wealth	100	`n` = 500 / `n` = 10,000	`initial_wealth` = 1
Sugarscape	100	`side` = 50, `n` = 400 / `side` = 100, `n` = 1,600	`max_sugar` = 4, `growback` = 1, `peaks` = 0.3–0.7; 0.7–0.3, `spread` = 0.2, `vision` = 1–6, `metabolism` = 1–4, `initial_sugar` = 5–25

C Intervals of the speed-ups

Table 16. Step-time speed-ups of Table 3 with paired bootstrap 95% intervals: the ten rounds of a workload are resampled with replacement, keeping each round's runs together, and the ratio of median step times is recomputed 10,000 times. With ten rounds the percentile intervals are coarse, and a bound can coincide with the point estimate. Generated by `paper/scripts/bootstrap_ci.py`.

Workload	Variant	steps over Mesa 2.3	steps over Mesa 3.5
Grid epidemic (SIR)	Mesa 3.5	1.7 [1.6, 1.7]	–
	Mesa 3.5 + numpy	198 [194, 201]	119 [117, 121]
	mesa-frames	6.1 [6.0, 6.1]	3.7 [3.6, 3.7]
	Modelly, cell lists	2.4 [2.4, 2.4]	1.4 [1.4, 1.5]
	Modelly, columns	42.1 [41.7, 42.5]	25.3 [25.1, 25.7]
Predator-prey	Mesa 3.5	0.9 [0.9, 0.9]	–
	Modelly, cell lists	1.6 [1.5, 1.6]	1.7 [1.7, 1.7]
	Modelly, columns	6.3 [6.1, 6.5]	6.9 [6.8, 6.9]
Flocking (boids)	Mesa 3.5	1.0 [1.0, 1.0]	–
	Mesa 3.5 + KD-tree	2.6 [2.6, 2.6]	2.6 [2.5, 2.7]
	Modelly, columns	32.9 [32.7, 33.1]	32.6 [31.5, 33.5]
Network contagion	Mesa 3.5	3.1 [3.1, 3.2]	–
	Modelly, columns	38.4 [38.2, 38.7]	12.2 [12.1, 12.3]
Insect population	Mesa 3.5	1.9 [1.8, 1.9]	–
	Modelly, columns	5.4 [5.3, 5.6]	2.9 [2.8, 3.0]
	Modelly, cohorts	242 [237, 248]	131 [129, 135]
Schelling segregation	Mesa 3.5	1.3 [1.3, 1.3]	–
	Mesa 3.5 + numpy	39.7 [39.6, 40.1]	30.6 [30.4, 30.8]
	mesa-frames	2.3 [2.3, 2.3]	1.8 [1.8, 1.8]
	Modelly, cell lists	14.2 [14.0, 14.3]	10.9 [10.8, 11.0]
	Modelly, columns	22.0 [21.8, 22.2]	16.9 [16.8, 17.0]
Boltzmann wealth	Mesa 3.5	1.8 [1.8, 1.9]	–
	Mesa 3.5 + numpy	17.1 [16.9, 17.2]	9.3 [9.2, 9.3]
	mesa-frames	2.1 [2.1, 2.1]	1.1 [1.1, 1.2]
	Modelly, columns	6.4 [6.3, 6.4]	3.5 [3.4, 3.5]
Sugarscape	Mesa 3.5	1.1 [1.0, 1.1]	–
	mesa-frames	0.7 [0.7, 0.7]	0.7 [0.7, 0.7]
	Modelly, cell lists	1.4 [1.3, 1.4]	1.3 [1.3, 1.3]
	Modelly, columns	4.6 [4.5, 4.7]	4.4 [4.3, 4.5]

D Further agreement tables

Table 17. As Table 7, for the Mesa 3.5 + numpy implementations against Mesa 2.3, over 100 shared seeds at the small size.

Workload	Implementation	Stats	max $\lvert d \rvert$	min p_t	min p_{KS}	Equivalent
Grid epidemic	Mesa 3.5 + numpy	6	0.21	0.13	0.21	6/6
Schelling	Mesa 3.5 + numpy	4	0.24	<0.01	0.28	4/4
Boltzmann wealth	Mesa 3.5 + numpy	4	0.29	0.03	0.21	3/4

Table 18. As Table 7, for the Mesa 3.5 and mesa-frames implementations against Mesa 2.3, over 100 shared seeds at the small size.

Workload	Implementation	Stats	max $\lvert d \rvert$	min p_t	min p_{KS}	Equivalent
Grid epidemic	Mesa 3.5	6	0.23	0.09	0.11	6/6
Grid epidemic	mesa-frames	6	0.20	0.17	0.28	6/6
Predator-prey	Mesa 3.5	5	0.14	0.36	0.58	5/5
Flocking	Mesa 3.5	3	0.00	1.00	1.00	3/3
Network contagion	Mesa 3.5	6	0.00	1.00	1.00	6/6
Insect population	Mesa 3.5	6	0.00	1.00	1.00	6/6
Schelling	Mesa 3.5	4	0.21	0.12	0.15	4/4
Schelling	mesa-frames	4	0.31	<0.01	0.05	4/4
Boltzmann wealth	Mesa 3.5	4	0.24	0.06	0.21	4/4
Boltzmann wealth	mesa-frames	4	0.31	0.03	0.04	2/4
Sugarscape	Mesa 3.5	6	0.20	0.14	0.21	6/6
Sugarscape	mesa-frames	6	0.02	0.55	0.97	6/6

Table 19. As Table 7, for Schelling's model at the small size: a replication on 300 new seeds (10,000 to 10,299, disjoint from the primary study's), every implementation against Mesa 2.3.

Workload	Implementation	Stats	max $\lvert d \rvert$	min p_t	min p_{KS}	Equivalent
Schelling	Mesa 3.5	4	0.23	<0.01	0.12	4/4
Schelling	mesa-frames	4	0.10	0.02	0.52	4/4
Schelling	cell lists	4	0.09	0.07	0.72	4/4
Schelling	columns	4	0.08	0.11	0.72	4/4

Table 20. As Table 8, for the grid epidemic at the large size: a replication on 200 new seeds (10,000 to 10,199, disjoint from the primary study's).

Workload	Modelly variant	Stats	max $\lvert d \rvert$	min p_t	min p_{KS}	Equivalent
Grid epidemic	cell lists	6	0.13	0.18	0.22	6/6
Grid epidemic	columns	6	0.09	0.36	0.33	6/6

E The trajectory study in full

Table 22. Every scalar comparison of the trajectory study over 100 shared seeds at the small size (seeds 20000 to 20099): Mesa 2.3's mean, the mean paired difference (implementation minus Mesa 2.3) with its 95% interval, the standardised difference d, the prespecified margin, and the paired TOST p-value at that margin, uncorrected and Holm-corrected over all the scalars of the study (the period is compared on the seeds where both arms define it). Generated by `paper/scripts/trajectories.py`.

Workload	Impl.	Quantity	Mesa	diff. [95%]	d	margin	p_{TOST}	Holm
Grid epidemic	Mesa 3.5	peak timing (step)	28.2	−2.07 [−5.27, +1.13]	−0.18	±2	0.517	1.000
Grid epidemic	Mesa 3.5	peak infected	75.8	−0.84 [−6.15, +4.47]	−0.04	±5%	0.137	1.000
Grid epidemic	Mesa 3.5	final size	292.9	−0.24 [−20.8, +20.3]	−0.00	±5%	0.084	1.000
Grid epidemic	Mesa 3.5	clusters of infected cells	154.2	−0.78 [−8.16, +6.6]	−0.03	±10%	<0.001	0.003
Grid epidemic	Mesa 3.5	largest cluster	19.4	+0.57 [−2.11, +3.25]	+0.06	±10%	0.157	1.000
Grid epidemic	Mesa 3.5	mean cluster size	1.935	+0.00708 [−0.0562, +0.0704]	+0.03	±10%	<0.001	<0.001
Grid epidemic	cell lists	peak timing (step)	28.2	−2.86 [−6.18, +0.465]	−0.23	±2	0.696	1.000
Grid epidemic	cell lists	peak infected	75.8	−0.5 [−5.99, +4.99]	−0.03	±5%	0.119	1.000
Grid epidemic	cell lists	final size	292.9	−4.27 [−24.7, +16.2]	−0.06	±5%	0.159	1.000
Grid epidemic	cell lists	clusters of infected cells	154.2	−2.92 [−9.83, +3.99]	−0.11	±10%	<0.001	0.008
Grid epidemic	cell lists	largest cluster	19.4	+0.38 [−2.4, +3.16]	+0.04	±10%	0.134	1.000

Workload	Impl.	Quantity	Mesa	diff. [95%]	d	margin	p_{TOST}	Holm
Grid epidemic	cell lists	mean cluster size	1.935	+0.00264 [−0.0682, +0.0734]	+0.01	±10%	<0.001	<0.001
Grid epidemic	columns	peak timing (step)	28.2	−3.33 [−6.55, −0.115]	−0.28	±2	0.793	1.000
Grid epidemic	columns	peak infected	75.8	−0.82 [−5.99, +4.35]	−0.04	±5%	0.129	1.000
Grid epidemic	columns	final size	292.9	−0.9 [−21.7, +19.9]	−0.01	±5%	0.096	1.000
Grid epidemic	columns	clusters of infected cells	154.2	−0.09 [−7.18, +7]	−0.00	±10%	<0.001	0.001
Grid epidemic	columns	largest cluster	19.4	−0.16 [−2.77, +2.45]	−0.02	±10%	0.089	1.000
Grid epidemic	columns	mean cluster size	1.935	−0.00395 [−0.0719, +0.064]	−0.02	±10%	<0.001	<0.001
Schelling	Mesa 3.5	segregation (final)	0.822	−0.000624 [−0.00628, +0.00503]	−0.03	±0.02	<0.001	<0.001
Schelling	Mesa 3.5	happy share (final)	0.992	+0.000147 [−0.00202, +0.00231]	+0.02	±0.02	<0.001	<0.001
Schelling	Mesa 3.5	moves (total)	1973.4	+26.7 [−85.6, +139]	+0.07	±5%	0.103	1.000
Schelling	Mesa 3.5	steps to converge	19.2	+0.13 [−2.07, +2.33]	+0.02	±2	0.048	0.856
Schelling	Mesa 3.5	residents with same-kind share 0.0–0.1	0.005	$-9.62e-05$ [−0.00154, +0.00134]	−0.02	±0.02	<0.001	<0.001
Schelling	Mesa 3.5	residents with same-kind share 0.1–0.2	0.001	+0.000125 [−0.000248, +0.000498]	+0.09	±0.02	<0.001	<0.001
Schelling	Mesa 3.5	residents with same-kind share 0.2–0.3	0.001	$+2.19e-06$ [−0.000384, +0.000389]	+0.00	±0.02	<0.001	<0.001
Schelling	Mesa 3.5	residents with same-kind share 0.3–0.4	0.025	+0.000748 [−0.000797, +0.00229]	+0.13	±0.02	<0.001	<0.001
Schelling	Mesa 3.5	residents with same-kind share 0.4–0.5	0.032	+0.000966 [−0.000771, +0.0027]	+0.17	±0.02	<0.001	<0.001
Schelling	Mesa 3.5	residents with same-kind share 0.5–0.6	0.122	−0.00142 [−0.00514, +0.0023]	−0.11	±0.02	<0.001	<0.001
Schelling	Mesa 3.5	residents with same-kind share 0.6–0.7	0.081	−0.000242 [−0.00326, +0.00278]	−0.02	±0.02	<0.001	<0.001
Schelling	Mesa 3.5	residents with same-kind share 0.7–0.8	0.112	+0.000815 [−0.00247, +0.0041]	+0.07	±0.02	<0.001	<0.001
Schelling	Mesa 3.5	residents with same-kind share 0.8–0.9	0.155	−0.000152 [−0.00598, +0.00568]	−0.01	±0.02	<0.001	<0.001
Schelling	Mesa 3.5	residents with same-kind share 0.9–1.0	0.468	−0.000748 [−0.015, +0.0135]	−0.02	±0.02	0.004	0.114
Schelling	cell lists	segregation (final)	0.822	+0.00136 [−0.00161, +0.00433]	+0.08	±0.02	<0.001	<0.001
Schelling	cell lists	happy share (final)	0.992	+0.00107 [−0.000362, +0.0025]	+0.14	±0.02	<0.001	<0.001
Schelling	cell lists	moves (total)	1973.4	−57.5 [−131, +16.3]	−0.14	±5%	0.135	1.000
Schelling	cell lists	steps to converge	19.2	−0.44 [−2, +1.12]	−0.06	±2	0.025	0.556
Schelling	cell lists	residents with same-kind share 0.0–0.1	0.005	−0.000978 [−0.00189, $-6.18e-05$]	−0.21	±0.02	<0.001	<0.001
Schelling	cell lists	residents with same-kind share 0.1–0.2	0.001	+0.000161 [−0.000144, +0.000465]	+0.10	±0.02	<0.001	<0.001
Schelling	cell lists	residents with same-kind share 0.2–0.3	0.001	$-4.76e-05$ [−0.000359, +0.000264]	−0.03	±0.02	<0.001	<0.001

Workload	Impl.	Quantity	Mesa	diff. [95%]	d	margin	p_{TOST}	Holm
Schelling	cell lists	residents with same-kind share 0.3–0.4	0.025	−0.000161 [−0.00132, +0.001]	−0.03	±0.02	<0.001	<0.001
Schelling	cell lists	residents with same-kind share 0.4–0.5	0.032	+0.000336 [−0.00109, +0.00176]	+0.06	±0.02	<0.001	<0.001
Schelling	cell lists	residents with same-kind share 0.5–0.6	0.122	$-8.01e-05$ [−0.00247, +0.00231]	−0.01	±0.02	<0.001	<0.001
Schelling	cell lists	residents with same-kind share 0.6–0.7	0.081	−0.000302 [−0.00241, +0.0018]	−0.03	±0.02	<0.001	<0.001
Schelling	cell lists	residents with same-kind share 0.7–0.8	0.112	+0.000274 [−0.0022, +0.00275]	+0.02	±0.02	<0.001	<0.001
Schelling	cell lists	residents with same-kind share 0.8–0.9	0.155	−0.00396 [−0.00864, +0.000718]	−0.20	±0.02	<0.001	<0.001
Schelling	cell lists	residents with same-kind share 0.9–1.0	0.468	+0.00476 [−0.00333, +0.0128]	+0.11	±0.02	<0.001	0.005
Schelling	columns	segregation (final)	0.822	+0.00226 [−0.000676, +0.00521]	+0.13	±0.02	<0.001	<0.001
Schelling	columns	happy share (final)	0.992	+0.000481 [−0.000942, +0.0019]	+0.06	±0.02	<0.001	<0.001
Schelling	columns	moves (total)	1973.4	−9.27 [−79.1, +60.6]	−0.02	±5%	0.006	0.165
Schelling	columns	steps to converge	19.2	+0.08 [−1.53, +1.69]	+0.01	±2	0.010	0.245
Schelling	columns	residents with same-kind share 0.0–0.1	0.005	−0.000641 [−0.00156, +0.000273]	−0.14	±0.02	<0.001	<0.001
Schelling	columns	residents with same-kind share 0.1–0.2	0.001	+0.000269 [$-6.59e-05$, +0.000603]	+0.17	±0.02	<0.001	<0.001
Schelling	columns	residents with same-kind share 0.2–0.3	0.001	$-7.94e-06$ [−0.0003, +0.000284]	−0.01	±0.02	<0.001	<0.001
Schelling	columns	residents with same-kind share 0.3–0.4	0.025	−0.000333 [−0.00156, +0.000894]	−0.06	±0.02	<0.001	<0.001
Schelling	columns	residents with same-kind share 0.4–0.5	0.032	−0.000616 [−0.00194, +0.000711]	−0.10	±0.02	<0.001	<0.001
Schelling	columns	residents with same-kind share 0.5–0.6	0.122	−0.00213 [−0.00468, +0.000416]	−0.18	±0.02	<0.001	<0.001
Schelling	columns	residents with same-kind share 0.6–0.7	0.081	$+4.08e-05$ [−0.00241, +0.00249]	+0.00	±0.02	<0.001	<0.001
Schelling	columns	residents with same-kind share 0.7–0.8	0.112	−0.00028 [−0.00269, +0.00213]	−0.03	±0.02	<0.001	<0.001
Schelling	columns	residents with same-kind share 0.8–0.9	0.155	−0.00213 [−0.00645, +0.00218]	−0.11	±0.02	<0.001	<0.001
Schelling	columns	residents with same-kind share 0.9–1.0	0.468	+0.00583 [−0.00187, +0.0135]	+0.14	±0.02	<0.001	0.007
Predator-prey	Mesa 3.5	wolves die out (prob.)	0.010	+0 [−0.0282, +0.0282]	+0.00	±5 pp	<0.001	0.010
Predator-prey	Mesa 3.5	sheep die out (prob.)	0.000	+0 [+0, +0]	+0.00	±5 pp	<0.001	<0.001
Predator-prey	Mesa 3.5	mean sheep	348.7	−1.68 [−18.3, +14.9]	−0.03	±5%	0.031	0.655

Workload	Impl.	Quantity	Mesa	diff. [95%]	d	margin	p_{TOST}	Holm
Predator-prey	Mesa 3.5	mean wolves	236.5	+0.306 [−6.83, +7.44]	+0.01	±5%	0.001	0.025
Predator-prey	Mesa 3.5	sheep amplitude	551.6	−7.24 [−36.3, +21.9]	−0.06	±10%	0.001	0.022
Predator-prey	Mesa 3.5	sheep period (steps)	345.0	−17.9 [−50.1, +14.3]	−0.15	±10%	0.154	1.000
Predator-prey	cell lists	wolves die out (prob.)	0.010	−0.01 [−0.0298, +0.00984]	−0.14	±5 pp	<0.001	0.002
Predator-prey	cell lists	sheep die out (prob.)	0.000	+0 [+0, +0]	+0.00	±5 pp	<0.001	<0.001
Predator-prey	cell lists	mean sheep	348.7	−6.84 [−18, +4.32]	−0.16	±5%	0.031	0.655
Predator-prey	cell lists	mean wolves	236.5	+3.18 [−1.73, +8.1]	+0.17	±5%	<0.001	0.011
Predator-prey	cell lists	sheep amplitude	551.6	−24.1 [−59.4, +11.2]	−0.19	±10%	0.042	0.798
Predator-prey	cell lists	sheep period (steps)	345.0	−11.6 [−44.8, +21.6]	−0.10	±10%	0.087	1.000
Predator-prey	columns	wolves die out (prob.)	0.010	−0.01 [−0.0298, +0.00984]	−0.14	±5 pp	<0.001	0.002
Predator-prey	columns	sheep die out (prob.)	0.000	+0 [+0, +0]	+0.00	±5 pp	<0.001	<0.001
Predator-prey	columns	mean sheep	348.7	−5.27 [−17.2, +6.67]	−0.12	±5%	0.023	0.526
Predator-prey	columns	mean wolves	236.5	+1.6 [−3.7, +6.9]	+0.09	±5%	<0.001	0.004
Predator-prey	columns	sheep amplitude	551.6	+14.2 [−24, +52.4]	+0.11	±10%	0.018	0.433
Predator-prey	columns	sheep period (steps)	345.0	−7.18 [−42.2, +27.8]	−0.06	±10%	0.062	1.000

Table 21. Every curve of the trajectory study (Section 5.5) over 100 shared seeds: the number of steps, the largest standardised paired difference over them, its 95% upper bound from 2,000 bootstrap resamples of the seeds, the number of steps whose 95% interval excludes zero, the largest relative difference of the means over the steps where Mesa 2.3's mean is at least a tenth of its maximum, and the p-value of a sign-flip permutation test (2,000 permutations) of the largest $|d_t|$. Generated by `paper/scripts/trajectories.py`.

Workload	Impl.	Series	Steps	$\sup_t \lvert d_t \rvert$	upper 95%	steps ∋ 0	rel.	p_{perm}
Grid epidemic	Mesa 3.5	infected	60	0.15	0.46	0	5.3%	0.921
Grid epidemic	cell lists	infected	60	0.24	0.52	0	7.4%	0.524
Grid epidemic	columns	infected	60	0.34	0.54	1	3.9%	0.158
Schelling	Mesa 3.5	happy share	40	0.13	0.46	0	0.2%	0.931
Schelling	cell lists	happy share	40	0.28	0.47	4	0.4%	0.045
Schelling	columns	happy share	40	0.07	0.29	0	0.1%	0.991
Schelling	Mesa 3.5	moves	40	0.13	0.46	0	3.9%	0.931
Schelling	cell lists	moves	40	0.28	0.48	4	4.8%	0.048
Schelling	columns	moves	40	0.07	0.29	0	1.8%	0.990
Predator-prey	Mesa 3.5	sheep	2000	0.29	0.61	22	15.1%	0.732
Predator-prey	cell lists	sheep	2000	0.36	0.66	71	13.8%	0.382
Predator-prey	columns	sheep	2000	0.31	0.61	60	13.0%	0.629
Predator-prey	Mesa 3.5	wolves	2000	0.39	0.67	135	14.6%	0.245
Predator-prey	cell lists	wolves	2000	0.34	0.65	27	13.7%	0.457
Predator-prey	columns	wolves	2000	0.35	0.65	71	14.1%	0.407
Predator-prey	Mesa 3.5	bare grass	2000	0.32	0.62	96	4.7%	0.534
Predator-prey	cell lists	bare grass	2000	0.37	0.65	45	4.7%	0.236
Predator-prey	columns	bare grass	2000	0.30	0.61	63	5.1%	0.622

F Further measurements

Table 23. The step-time speed-up of Modelly's columns (and the insect population's cohorts) over Mesa 3.5 on three CPython versions, each in its own environment with the same package versions and the compiled modules built for it: median CPU time of the steps over five rotated rounds and the median of the per-round ratios (Section 5.2). Generated by `paper/scripts/python_versions.py`.

		CPython 3.12			CPython 3.14			CPython 3.15		
Workload	Form	Mesa	Modelly	S_{steps}	Mesa	Modelly	S_{steps}	Mesa	Modelly	S_{steps}
Grid epidemic	columns	4.899	0.143	33.5	4.494	0.141	31.6	3.904	0.153	25.5
Predator-prey	columns	4.511	0.473	9.5	4.026	0.517	7.8	3.554	0.518	6.9
Flocking	columns	24.107	0.747	32.2	25.072	0.735	34.2	23.888	0.748	31.8
Network contagion	columns	2.722	0.194	14.0	2.932	0.196	14.9	2.544	0.195	13.1
Insect population	columns	2.184	0.740	2.9	2.264	0.715	3.2	2.012	0.662	3.0
	cohorts	2.184	0.015	142.7	2.264	0.014	158.3	2.012	0.015	132.4
Schelling	columns	4.172	0.246	17.0	4.171	0.192	21.9	3.553	0.202	17.6
Boltzmann wealth	columns	1.317	0.311	4.2	1.355	0.325	4.2	1.123	0.328	3.4
Sugarscape	columns	1.505	0.332	4.5	1.569	0.332	4.6	1.424	0.323	4.4

Table 24. A sweep end to end (Section 5.2): 64 runs of the grid epidemic at the small size (a 16-point Latin hypercube over β, four replicates, 60 steps) from the creation of the `Experiment` to its results queried from the Parquet store, driven by Modelly's experiment layer with each engine, on the serial executor pinned to one core and on four processes; the median build-plus-steps CPU time of one run of the same engine in the same process; and the share of the sweep that is simulation. Medians of five rounds. Generated by `paper/scripts/sweep_e2e.py`.

Engine	Executor	Sweep (s)	per run (s)	simulation per run (s)	share / gain
columns	serial, one core	2.42	0.038	0.024	63% simulation
	4 processes	2.34	0.037	0.024	1.0× serial
Mesa 3.5	serial, one core	19.22	0.300	0.278	93% simulation
	4 processes	6.86	0.107	0.278	2.8× serial

Table 25. ABC-SMC at four particle counts, the true parameters drawn from the prior, 500 datasets per model and count (the same datasets at every count): coverage of the 95% highest-density interval with its Clopper–Pearson interval, the interval's mean width, the posterior's effective sample size as a share of the particles, the last generation's acceptance rate and the model runs spent (Section 6.2). Generated by `paper/scripts/abc_sweep.py`.

Particles	Model	Parameter	Coverage	95% interval	HDI width	ESS/N	Acceptance	Runs
30	SIR	β	91.2%	[88.4, 93.5]	0.265	0.93	19%	625
		γ	94.2%	[91.8, 96.1]	0.181	0.93	19%	625
30	Predator-prey	growth	96.8%	[94.9, 98.2]	0.265	0.89	18%	719
		predation	94.8%	[92.5, 96.6]	0.014	0.89	18%	719
60	SIR	β	93.6%	[91.1, 95.6]	0.285	0.95	19%	1,216
		γ	96.4%	[94.4, 97.9]	0.191	0.95	19%	1,216
60	Predator-prey	growth	98.2%	[96.6, 99.2]	0.279	0.90	17%	1,437
		predation	97.2%	[95.3, 98.5]	0.015	0.90	17%	1,437
100	SIR	β	95.8%	[93.7, 97.4]	0.294	0.95	19%	2,018
		γ	96.4%	[94.4, 97.9]	0.194	0.95	19%	2,018
100	Predator-prey	growth	99.0%	[97.7, 99.7]	0.287	0.91	17%	2,425
		predation	97.2%	[95.3, 98.5]	0.015	0.91	17%	2,425
200	SIR	β	95.2%	[92.9, 96.9]	0.297	0.95	18%	4,032
		γ	98.0%	[96.4, 99.0]	0.197	0.95	18%	4,032
200	Predator-prey	growth	99.8%	[98.9, 100.0]	0.291	0.91	17%	4,820
		predation	98.4%	[96.9, 99.3]	0.015	0.91	17%	4,820

G Scaling out and connecting to data in detail

This appendix gives the design, the measurement conditions and the evidence behind each capability of Section 7, with the tests and result files that reproduce every number. The measurements were made on the laptop of Section 5, with the machine shared and without the fixed frequency of Section 5.2, except the timings of equations inside agents (Appendix G.6).

G.1 Per-agent random streams and split runs

Streams. A model class that sets `agent_streams = True` gives every agent generators of its own: `agent.random` and `agent.rng` are seeded from the scenario's seed sequence and the agent's `stream_key()` (its `unique_id` unless overridden), in a spawn-key domain that `SeedSequence.spawn` never reaches. An agent then draws the same numbers whatever the activation order and the population, and a checkpoint continues every stream. The flag is off by default, and a model without it draws from the model's two generators (`tests/core/test_streams.py`).

Partitions and the contract. `modelly.distributed` cuts a grid into balanced blocks with their halos (torus-aware), a network by breadth-first banding with greedy boundary refinement (not METIS [57]: on a 30×30 lattice cut in four it cuts 134 edges against the 60 of the best split, 2.2 times it, and a tenth of a random assignment's 1,315, `paper/data/graph_cut.json`; the test asserts at most three times the best), and a continuous space by recursive bisection at the population's quantile. A model declares `partitionable = Partitionable(space, phases, reporters)`: each phase names how far it reads (`radius`) and whether it moves agents. Before a phase that reads across a border the workers exchange ghost copies of their border agents; after a phase that moves agents, the movers migrate to their new partition with their streams, in a deterministic order. The build refuses, with the reason, a model that draws from the model's generators inside a phase, holds `Column` or dataset agents, schedules events of its own, moves into cells with a capacity, keeps references to other agents in attributes, or creates agents keyed on `unique_id` during a phase. Backends are `local`, `mpi` (`mpi4py`), `dask` and `ray`; `Sum`, `Min` and `Max` reporters reduce across partitions, and a checkpoint writes one file per partition with an index of digests that `Cluster.restore` continues draw for draw.

Measured. On three reference models in `modelly/examples/distributed` (a grid epidemic on blocks, a network SIR on graph parts, a continuous SIR on KD-tree boxes), every reporter row and every agent's attributes, location and stream state are byte-identical to a plain single-process run at 1, 2, 4 and 16 partitions on two seeds; the test is `test_independence.py` in `tests/distributed/`, also in the determinism suite, whose single-process digests are the same with the compiled and the pure-Python core. The same holds under `mpirun` with 2 and 4 ranks on the grid and network models (`test_mpi.py`) and on two Dask workers. Weak scaling of the grid epidemic is in Table 26; the script is `weak_scaling.py` in `benchmarks/distributed/`, and the raw numbers are in `paper/data/weak_*.json`. Next to each split run of K workers the script times K independent copies of one block with no communication, which separates what the machine gives from what the protocol keeps of it.

Table 26. Weak scaling of the split grid epidemic under MPI, one run per worker count, 10,000 people per worker, 20 steps, Open MPI 5.0.11 and mpi4py 4.1.2, wall-clock seconds per step after a warm-up step, build excluded, on the 16-core laptop of Section 5 shared with other jobs. "Load" is the one-minute load average at the start of each run; "machine" is the efficiency of K copies with no communication, timed by the slowest copy; "protocol" is the split run's time relative to the copies'.

Workers	People	Split, s/step	Copies, s/step	Efficiency	Machine	Protocol	Load
1	10,000	0.0114	0.0101	100%	100%	89%	1.5
2	20,000	0.0169	0.0119	68%	85%	71%	1.5
4	40,000	0.0252	0.0153	45%	66%	61%	1.5
8	80,000	0.0667	0.0402	17%	25%	60%	1.9
12	120,000	0.0884	0.0492	13%	21%	56%	3.0
16	160,000	0.1602	0.0648	7%	16%	40%	4.7

With 10,000 people per worker the split run takes 1.1 to 2.5 times as long per step as its slowest copy, so the protocol keeps 40 to 61% of what the machine gives from four workers on. Under MPI the efficiency falls from 68% at two workers to 45% at four and 7% at sixteen, with the machine itself, the copies, at 66% and 16%. With 40,000 people per worker (10 steps, loads of 6 to 7), the efficiency is 74, 50 and 33% at 2, 4 and 8 workers against 72, 55 and 36% for the copies, so the protocol keeps 80 to 90%: a larger block has more work per exchanged border. The Dask backend, which routes every exchange through its scheduler (eight per step), measured 65, 32 and 22% at 2, 4 and 8 workers (loads of 9 to 11), each split step 3.0 to 4.4 times its copies: the scheduler's round trips, not the model, take its time. Every efficiency here is a single run per worker count, not a median of repeats, on a shared machine, so the table supports the shape of the curve and the byte-identity claims, not the last digit of an efficiency.

Not met. Efficiency at 70% or more from 1 to 64 workers, the level a cluster code would aim for, cannot be measured on one laptop, and the laptop's hybrid processor is the main limit: eight copies of the same work with no communication at all already run at 25% of one (36% with 40,000 people per worker), because beyond the four performance cores the copies land on efficiency cores (at eight workers their step times split into two groups of four, 0.021 to 0.022 s and 0.033 to 0.040 s). The Ray backend is written but untested (Ray does not install on CPython 3.15); and the split-run suite runs on the three reference models, not on the zoo, whose models would change their numbers if moved to per-agent streams.

G.2 A GPU backend for column phases

`modelly.gpu` runs column phases (`DeviceTable`), cohort tables (`DeviceCohortTable`, binomial thinning included) and the cell-space operators (`DeviceCellIndex`: counts, neighbour sums, random-neighbour moves) on `numpy`, CuPy [85] or JAX [15], chosen by `get_backend` or `MODELLY_BACKEND`; a backend that does not load falls back to `numpy` with a warning, and `import modelly` loads neither library. Random numbers come from `PhiloxStream`, a stateless Philox4x32-10 [98] keyed by seed and stream, checked against the Random123 known-answer vectors and identical on every backend; binomial draws use inversion below a mean of 10 and Hörmann's BTRS rejection sampler above it [48], both with the exact law (chi-square tests), inversion identical across backends and BTRS equal in every test and run (it uses `log`, so a last-bit difference is possible in principle). Without 64-bit types JAX draws single-precision uniforms, so its runs differ from `numpy`'s in value, not in law. Populations larger than the device run in batches, bit for bit the same as one batch.

Measured. The script is `gpu_speedup.py` in `benchmarks/playbooks/`, and the raw numbers are in `paper/data/gpu_speedup.json`. The grid epidemic at 10 million agents on a 1000 × 1000 torus, 60 steps, seed 1, in fresh interpreters, wall clock around the steps with the device synchronised, the build timed apart, one untimed run first to fill CuPy's kernel cache: over five rounds (with the `modelly` package of the tag v1.5.0), the CPU column variant (CPython 3.15, one core) takes 52.15, 20.51, 20.16, 22.11 and 19.86 s for the steps (the first run includes a cold start) and the CuPy variant (CPython 3.14, the GPU environment's interpreter; the two interpreters differ, but the steps are dominated by numpy and CuPy kernels) 1.413, 0.803, 0.708, 0.884 and 0.701 s, per-round ratios of 36.9, 25.6, 28.5, 25.0 and 28.3 and a median of 28.3 on an NVIDIA RTX 5050 laptop GPU (8 GB, CUDA runtime 13.2 from the `cupy-cuda13x` 14.2.0 wheel). Run on `numpy` and on CuPy with the same seed, the variant gives the same infected series for all 60 steps and the same SHA-256 of the final state, days and row columns of all 10 million agents; on `numpy` its steps take 108.4 s, more than the column variant's because Philox draws cost more than `numpy`'s generator, and 123 times its 0.88 s on CuPy. Building 10 million agent objects takes 23 to 30 s on the CPU and 26 to 35 s for the GPU variant (83 s in the first, cold round), so build plus steps is only 1.0 to 1.7 times faster: a GPU an order of magnitude faster than the column form at ten million agents is what the steps show, not what a whole run shows.

Where it does not pay. At the playbook's own 20,000 agents the steps take 0.15 to 0.16 s on CuPy against 0.07 s on the CPU (three seeds), and the insect cohorts, a table of under 200 rows, take 1.60 to 1.64 s on the GPU for 120 days against 7 ms on the CPU: kernel launches cost more than the arithmetic. JAX recompiles whenever a cohort table changes length. Apple Metal was not measured (no Apple GPU was available), and against the original CPU column variant, which draws from `model.rng`, the GPU variant agrees in distribution only, as every playbook variant does.

G.3 Differentiable models

`modelly.diff` simulates column phases that JAX traces (a `DiffModel` of phase functions, each with a `differentiable` flag) and differentiates their outputs with respect to the scenario's parameters. Normal and uniform draws are pathwise and exact; a Bernoulli choice goes through the straight-through estimator [12], Gumbel-softmax [51, 71] or the score function [129], and each warns that the gradient is an approximation. `check_gradients` compares a gradient with central finite differences in double precision, and `GradientCalibration` runs Adam [60] over `modelly.calibrate`'s priors and targets, deterministic for a seed.

Measured (`tests/diff`, `docs/differentiable.md`), against runs of the agent-based zoo models at known parameters: the differentiable twin of Bass diffusion (20,000 households, 100 steps, straight-through) recovers an innovation rate of 0.0305 and an imitation rate of 0.379 for a truth of 0.03 and 0.38, within 2% (the test allows 5%); the Bouchaud–Mézard twin (1,000 agents, 300 steps, two replicates, pathwise) recovers a volatility of 1.448 and an exchange rate of 0.937 for 1.5 and 1.0, within 7% (the test allows 10%), and 1.54 and 1.09 with four replicates, so the error is the observed Gini series' noise, not the gradient's. A pathwise gradient agrees with finite differences to a relative error below 10^{-6} on a one-parameter test model and below 10^{-4} on the Bouchaud–Mézard twin, and the Gumbel-softmax Bass model within 10^{-3}; the straight-through estimator fails the same check, by design, since it is biased. The tests skip without JAX; on the GPU environment all 108 GPU and differentiable tests pass on CuPy and CUDA JAX.

G.4 Compiled event heap, column gather and memory-mapped columns

A fifth C module, `fastevents`, holds the event list's heap: the same sift steps as `heapq`, on keys (time, priority, insertion order) read once at push and compared as C numbers, with a line-for-line Python twin that gives the same pops and the same heap array on random schedules, and checkpoints that load in either (`tests/native/test_event_heap*.py`). The data collector's attribute reporters, `AgentSet.get` and `agg` read a `Column` attribute from its table in C instead of calling the Python getter per agent, with identical rows (`test_column_gather.py`). `MemoryMapped(dir)` keeps a table's columns in `numpy.memmap` files, grown in place, with checkpoints that carry the rows. A cohort table of 100 million rows (19 bytes a row) ran three life-cycle steps inside a 16 GB `systemd` memory limit with a peak of 3.61 GB memory-mapped (4.26 GB in memory); a step took 11.2 to 14.0 s memory-mapped and 9.9 to 10.9 s in memory (one run of three steps each); and under a 3 GB limit the memory-mapped table finished, its steps taking 11.1 to 12.4 s, while the in-memory one was killed (`benchmarks/large_columns.py`; the four runs in `paper/data/large_columns/`); 100 million *agent objects* cannot fit in 16 GB, since the objects alone need over 22 GB.

G.5 Live data and data assimilation

`modelly.stream` hands observations to a running model as they arrive: a replayed table, a CSV file that grows between polls, an HTTP endpoint, and MQTT or Kafka topics through optional extras (tested against fake clients only; no broker was available). An `ObservationOperator` reads the observed quantities from a model and scores an observation under independent Gaussian errors. An `Ensemble` advances n replicates of one scenario in lock step in one process, with spawned seeds and starting values drawn from a prior; a `ParticleFilter` is the bootstrap filter of Gordon et al. [34]: it weights the replicates by the likelihood, tracks the effective sample size, and resamples systematically [25] below a threshold, duplicating replicates through checkpoint copies reseeded from derived keys; an `EnsembleKalmanFilter` is the stochastic filter with perturbed observations [30, 31] on named model attributes, with optional bounds. Both give the ensemble's mean and quantiles, now and ahead, as a long-format table.

Measured (the tutorial `docs/assimilation.md`, which a test in `tests/stream/` runs as a module): a town of 1,000 people with a true contact rate of 0.45 produces 50 days of noisy case counts; 40 replicates with contact rates drawn from a uniform prior track it. The root-mean-square error of the filtered nowcast of the unobserved number infected is 5.07 people, against 69.2 for the unfiltered ensemble (ratio 0.073; the test requires below 0.2), with 24 resamplings; the ensemble Kalman filter recovers a contact rate of 0.451 (spread 0.030 across replicates) for the true 0.45 (the test allows ±0.05). On a linear-Gaussian case with 2,000 replicates the Kalman update reproduces the exact posterior. The tutorial uses one synthetic dataset (truth seed 1, ensemble seeds 2 and 3); a coverage study over datasets was not done. Two runs from one seed give identical weights, histories and replicate states, in one process and in a fresh interpreter, with the compiled and the pure-Python core. Stepping stays in one process (shipping every replicate to a worker for each observation would cost more than it saves), and in a model with per-agent streams a duplicate's agents keep the streams of the agents they were copied from, which the documentation states. The numbers of this paragraph are recorded in `paper/data/assimilation.json` by `paper/scripts/assimilation.py`.

G.6 Fields, differential equations and a coupling schedule

`modelly.couple` couples agents to continuous processes. A `Field` owns or adopts a float property layer of an orthogonal grid and updates it in place with diffusion, first-order upwind advection and exact decay, in explicit sub-steps that refuse a time step above the stability limit (pointing to the implicit scheme, backward Euler with a sparse LU solve), with closed, absorbing or periodic boundaries and a budget of what was added, flowed out and decayed. `ODE` and `StockFlow` components integrate with `solve_ivp`, and `AgentODE`, `AgentSDE` and delays integrate one system per agent with `solve_batch` (Section 7.6). A `Coupling` orders components phase by phase, writers before readers, and `build()` rejects a read nobody writes (naming the closest quantity), two writers of one quantity, a cycle inside a phase, and a lagged read without an initial value.

Measured (`tests/couple`): closed and periodic boundaries conserve mass to a relative 10^{-12} in both schemes; a spreading pulse's variance grows by exactly $2Dt$ per axis; a one-dimensional Gaussian matches the analytic heat kernel, with an error near 10^{-6} at $D\,\Delta t/h^2 = 1/6$, where the explicit scheme is fourth-order in one dimension; upwind drift moves the centre of mass by exactly vt; decay follows e^{-kt}; with absorbing boundaries what remains plus what flowed out equals the initial mass; logistic growth matches its closed form for four integrators; and a model with an implicit field and an ODE saves and loads bit for bit. Two zoo models use it. In `nutrient_foragers` (a nutrient entering at a river bank and diffusing inland, eaten by foragers) the mean population is within 3% of the carrying capacity from the energy balance, within 1% once the measured decay and starvation losses are counted, the nutrient budget closes to round-off, and forager density falls with the distance from the river. In `hospital_epidemic` (SIR agents whose admissions drive a hospital stock-flow ODE that damps contacts) the attack rate matches the final-size relation within five standard deviations without the contact response, occupancy and beds match the exact matrix-exponential solution within 10^{-6}, admissions equal patients in hospital plus discharged to round-off, and a stronger response lowers both the peak and the attack rate. Fields work on orthogonal grids with uniform spacing and coefficients (a hexagonal grid is refused), the advection scheme smears sharp fronts, and since decay uses `exp` and `solve_ivp` chooses its steps from an error norm, these two outputs are compared across platforms with a tolerance (Section 6).

Equations inside agents (`tests/couple`, `paper/scripts/odes.py`, `benchmarks/ode/`). `solve_batch` keeps, per system, the time, the step size, the last derivative and a status, and applies SciPy's rules (the initial step, the error norm scaled by `atol + rtol |y|`, the safety

factor and the bounds on the step's growth and shrinkage) to each system; a stage evaluates the right-hand side once on every system still running, so the systems that need small steps do not slow the others, and a system that has finished drops out. Events are located by bisection on each system's Runge–Kutta interpolant (`solve_ivp` uses Brent's method on the same interpolant), terminal events resume within the step, and the dense output reads any system at any time of its span. A delay's history and the stochastic schemes' generator are saved in checkpoints, so a resumed run equals a straight one bit for bit. On 200 random systems of the viral model, with RK45 and RK23 at `rtol` $= 10^{-3}$, `atol` $= 10^{-6}$ and at 10^{-8}, 10^{-10}, all 200 take exactly the accepted steps of their own `solve_ivp` call in each of the four cases, and the final states differ from it by a relative 1.9×10^{-11} at most (RK45 at the looser tolerance; 1.2×10^{-14} for RK23 there, 6×10^{-15} for both at the tighter one; `paper/data/odes.json`). The timings (`benchmarks/ode/compare.py` under `taskset -c 2` at the fixed frequency of Section 5.2, `paper/data/ode_compare.json`) integrate the target-cell-limited model of influenza [8], three equations per agent, over ten days with RK45 at `rtol` $= 10^{-6}$ and `atol` $= 10^{-9}$, for 10 to 100,000 agents in two populations: alike, whose parameters vary by 10% between agents, and diverse, whose infection rate, virus production and initial virus span one to four orders of magnitude, so that the infections peak at different times. Each time is the best of three runs, except the per-agent loop, run once past 1,000 agents and not at all past 10,000. The loop takes 0.054, 0.54, 5.4 and 54.0 s at 10, 100, 1,000 and 10,000 alike agents (0.049, 0.53, 5.2 and 52.1 s diverse), about 5 ms per agent; `solve_batch` takes 0.017, 0.019, 0.038, 0.25 and 3.42 s at 10 to 100,000 alike agents and 0.025, 0.037, 0.079, 0.40 and 4.56 s diverse; one stacked `solve_ivp` call takes 0.010, 0.011, 0.025, 0.17 and 2.88 s alike and 0.017, 0.025, 0.060, 0.41 and 7.28 s diverse. Accuracy is each agent's error against a solution at `rtol` $= 10^{-12}$, in units of its own tolerance `atol + rtol |y|` (root mean square over its three equations), for the worst agent: the loop and `solve_batch` have the same worst error at every size to nine digits, 0.15 to 0.18 alike and 8.3 to 21.0 diverse (28.5 for `solve_batch` at 100,000, where the loop was not run), the global error that a per-agent control of the local error leaves; the stacked call, whose error norm is the root mean square over all agents, has 0.22 to 0.28 alike and 0.81 to 10.0 diverse, more accurate on the diverse population because every agent takes the smallest agent's steps. The within-host zoo model (`benchmarks/ode/within_host.py`, same core and frequency, one run of each, `paper/data/ode_within_host.json`) runs 100,000 hosts to day 20, the epidemic's peak, and integrates the next day twice from the same state: with its `AgentODE`, onset and clearance events included, in 1.07 s, and with one `solve_ivp` call per exposed host (49,850 of them), same method and tolerances (10^{-6} and 10^{-9}), in 49.1 s, 46 times as long; the two final states differ by at most 4.0×10^{-9} of the tolerance.

G.7 Emulators, active learning and history matching

`modelly.emulate` trains a Gaussian-process emulator (the process of `modelly.calibrate`, with restarts, covariances and conditioning) or a `numpy`-only ensemble of small networks on an experiment's runs; `ActiveLearning` picks the next runs by integrated variance reduction (the default) or largest variance [70], in batches conditioned on each pick; `HistoryMatch` [5, 122] rules out the parameter space whose implausibility exceeds 3, in waves, refitting the emulators inside the space that remains.

Measured (`tests/emulate`, `docs/emulators.md`). History matching of the age-structured SEIR zoo model over three parameters, from three observed outputs (peak share infectious, its timing, final size) at the zoo defaults: four waves of 20 runs leave 0.1705, 0.0444, 0.0307 and 0.0246 of the box, the truth inside after every wave (its largest implausibility 2.2), and `MODELLY_PURE_PYTHON=1` rules out exactly the same points. Active learning against a Latin hypercube (`benchmarks/emulate/active_vs_lhs.py`: normalised root-mean-square error on 500 test points, the hypercube's median of three seeds, the factor being the hypercube runs

needed for the active design's error divided by the active runs): on the Bass diffusion zoo model the factor is 1.64, 2.64, 1.33 and 2.67 at 10 to 40 runs and above 2.4 at 50 and above 2.0 at 60 (no hypercube of up to 120 runs reached the active design's error); on Branin [23] 1.13 to 1.43 below 80 runs; on the Ishigami function 0.79 to 1.41. Active learning halves the runs a hypercube needs (a factor of 2) on the zoo model at most budgets, and not on the two analytic functions, where a well-spread design is already close to the best a Gaussian-process variance, which does not depend on the outputs, can do; the largest-variance criterion (the alternative to integrated variance reduction) is worse than a hypercube on Ishigami at four of six budgets (0.20 to 0.40) and better at 20 runs (1.82). The tests assert what was measured (40 active Branin runs match a hypercube of 50; 20 active Bass runs match 40).

G.8 Synthetic populations and models on real data

`modelly.population` builds a starting population from aggregate tables: iterative proportional fitting [22] with truncate-replicate-sample integerisation [67], simulated annealing over whole records of a sample, households from a size table with an adult head, households placed in regions, schools and workplaces assigned to the nearest place with room within a distance limit, and a contact network with household, school and work layers; the age-structured SEIR zoo model can start from it. No personal data is bundled: the marginals of the tests are made up. Three small samples of Burlington, Vermont, ship with their licences in a `NOTICE.md` file: OpenStreetMap streets (ODbL 1.0, © OpenStreetMap contributors; 255 junctions, 607 directed segments), a 120×120 window of ESA WorldCover 2021 [131] (CC BY 4.0) and 41 census tracts (public domain), with a script that downloads the full sources, checks their SHA-256 and rebuilt the samples byte for byte.

Measured (`tests/population`, `tests/zoo`): about 20,000 synthetic people over the 41 tracts reproduce their totals and age counts exactly in every tract, their sex counts within fewer people than there are age levels (under 1% in total), household-size shares within 0.02, every household with an adult, no school or workplace over its 600 seats or beyond its distance, and the SEIR model started from them follows the final-size relation within 0.05. Three zoo models run on the real samples: `road_traffic`, whose shortest-path flows equal NetworkX's edge betweenness on all 607 segments and stay within five binomial standard deviations over three seeds; `habitat_selection`, whose occupancy per land-cover class reaches its Metropolis stationary law within five standard deviations and never enters water; and `schelling_regions`, whose dissimilarity at a threshold of 0.45 ends more than five standard deviations above random allocation and rises with the threshold.

G.9 Modelly in the browser

`modelly export-web model.py:Model` writes a static site that runs the model with Pyodide [118] (pinned version 314.0.7, loaded from its content delivery network): the parameter panel of Studio, run controls, a space view, a reporter plot, the model in a Web Worker so the page answers during long runs, Modelly as a trimmed pure-Python wheel of about 0.9 MB, and only the packages the model was observed to import. The C modules also build for WebAssembly with Emscripten through `pyodide build`, and the export can ship that wheel.

Measured (`tests/web/test_browser.py`, `docs/browser.md`). The 14 bundled examples (the three split-run reference models were not exported) and the 18 zoo models, 32 models, each run from its export in headless Chromium with the pure wheel and with the compiled one, 64 of 64 runs without error. Because Pyodide's GDAL reads no Deflate-compressed GeoTIFF and its geopandas has neither pyogrio nor fiona, the bundled land-cover window is stored uncompressed and the tracts are read through `GeoDataFrame.from_features`. A cold first visit

transfers 14.5 MB for the age-structured SEIR model (Pyodide, its standard library, `numpy` and `pandas` are 13.6 MB of it) and is ready at step 0 after 4.3 s unthrottled, 6.5 s on a 50 Mbit/s line with 40 ms round trips, and 19.4 s on Chrome's "Fast 4G" profile (about 1 MB/s, 165 ms), where the transfer alone takes 14 s; the forest fire, a model that needs `scipy` (as the grid, continuous and network spaces do), transfers 28.4 MB and takes 4.8 s unthrottled, 9.0 s on the 50 Mbit/s line and 33.4 s on Fast 4G (medians of three runs; `tools/web/measure_load.py`, reports in `paper/data/browser_load/`). A first load under ten seconds on a good home connection is reached without `scipy` and, narrowly, with it, and not reached on a mobile profile. The documentation gallery embeds three live models.

H Every comparison of the agreement study

Table 27. Every comparison of the agreement study at the small size (100 seeds): standardised difference d with its paired 95% interval, correlation r between the implementations' runs on the same seed, and the paired t, Kolmogorov–Smirnov and paired TOST p-values, uncorrected.

Workload	Modelly variant	Statistic	d	95% interval	r	p_t	p_{KS}	p_{TOST}
Grid epidemic	cell lists	`final_S`	+0.17	[−0.10, +0.44]	+0.07	0.22	0.15	0.008
Grid epidemic	cell lists	`final_I`	+0.22	[−0.06, +0.50]	+0.01	0.12	0.47	0.024
Grid epidemic	cell lists	`final_R`	−0.23	[−0.50, +0.03]	+0.09	0.08	0.15	0.026
Grid epidemic	cell lists	`peak_I`	−0.21	[−0.49, +0.07]	+0.01	0.14	0.15	0.021
Grid epidemic	cell lists	`peak_step`	+0.10	[−0.16, +0.36]	+0.12	0.45	0.47	0.002
Grid epidemic	cell lists	`mean_I`	−0.20	[−0.47, +0.07]	+0.08	0.15	0.21	0.014
Grid epidemic	columns	`final_S`	−0.08	[−0.36, +0.20]	−0.01	0.58	0.70	0.002
Grid epidemic	columns	`final_I`	+0.03	[−0.25, +0.31]	+0.02	0.82	0.91	0.001
Grid epidemic	columns	`final_R`	+0.08	[−0.20, +0.37]	−0.03	0.57	0.70	0.002
Grid epidemic	columns	`peak_I`	+0.10	[−0.18, +0.38]	+0.00	0.48	0.91	0.003
Grid epidemic	columns	`peak_step`	+0.13	[−0.16, +0.42]	−0.09	0.39	0.70	0.007
Grid epidemic	columns	`mean_I`	+0.08	[−0.21, +0.36]	−0.02	0.60	0.58	0.002
Predator-prey	cell lists	`final_sheep`	−0.25	[−0.51, +0.00]	+0.16	0.05	0.28	0.031
Predator-prey	cell lists	`final_wolves`	+0.15	[−0.10, +0.41]	+0.17	0.24	0.70	0.004
Predator-prey	cell lists	`mean_sheep`	−0.30	[−0.55, −0.06]	+0.23	0.02	0.02	0.059
Predator-prey	cell lists	`mean_wolves`	+0.23	[−0.03, +0.49]	+0.15	0.08	0.21	0.022
Predator-prey	cell lists	`mean_grass`	+0.27	[−0.00, +0.54]	+0.08	0.05	0.01	0.044
Predator-prey	columns	`final_sheep`	−0.10	[−0.38, +0.18]	+0.03	0.48	0.70	0.002
Predator-prey	columns	`final_wolves`	+0.09	[−0.19, +0.36]	+0.04	0.52	0.70	0.002
Predator-prey	columns	`mean_sheep`	−0.06	[−0.35, +0.24]	−0.10	0.70	0.58	0.002
Predator-prey	columns	`mean_wolves`	+0.13	[−0.14, +0.41]	+0.02	0.34	0.58	0.005
Predator-prey	columns	`mean_grass`	+0.01	[−0.29, +0.32]	−0.18	0.93	0.47	0.001
Flocking	columns	`final_polarisation`	+0.00	[−0.00, +0.00]	+1.00	0.90	1.00	<0.001
Flocking	columns	`mean_polarisation`	−0.00	[−0.00, +0.00]	+1.00	0.94	1.00	<0.001
Flocking	columns	`mean_neighbors`	+0.00	[+0.00, +0.00]	+1.00	1.00	1.00	<0.001
Network contagion	columns	`final_S`	−0.02	[−0.26, +0.23]	+0.22	0.90	0.99	<0.001
Network contagion	columns	`final_I`	−0.13	[−0.41, +0.15]	+0.02	0.36	0.28	0.005
Network contagion	columns	`final_R`	+0.04	[−0.20, +0.29]	+0.22	0.73	0.99	<0.001
Network contagion	columns	`peak_I`	+0.05	[−0.24, +0.34]	−0.07	0.74	0.97	0.001
Network contagion	columns	`peak_step`	−0.16	[−0.39, +0.08]	+0.27	0.20	1.00	0.003
Network contagion	columns	`mean_I`	−0.15	[−0.41, +0.12]	+0.12	0.27	0.21	0.004
Insect population	columns	`final_eggs`	+0.16	[−0.14, +0.46]	−0.13	0.28	0.37	0.014
Insect population	columns	`final_larvae`	+0.01	[−0.27, +0.30]	−0.06	0.92	0.97	0.001
Insect population	columns	`final_pupae`	+0.12	[−0.13, +0.38]	+0.17	0.34	0.82	0.002
Insect population	columns	`final_adults`	−0.33	[−0.61, −0.05]	−0.01	0.02	0.04	0.120
Insect population	columns	`mean_adults`	−0.12	[−0.41, +0.17]	−0.07	0.41	0.91	0.006
Insect population	columns	`peak_adults`	−0.26	[−0.56, +0.03]	−0.10	0.08	0.21	0.057
Insect population	cohorts	`final_eggs`	+0.23	[−0.05, +0.51]	+0.03	0.10	0.47	0.028
Insect population	cohorts	`final_larvae`	+0.20	[−0.09, +0.48]	−0.00	0.17	0.21	0.017
Insect population	cohorts	`final_pupae`	+0.05	[−0.23, +0.32]	+0.03	0.73	0.99	0.001
Insect population	cohorts	`final_adults`	+0.01	[−0.25, +0.28]	+0.10	0.92	1.00	<0.001
Insect population	cohorts	`mean_adults`	−0.02	[−0.29, +0.25]	+0.07	0.87	0.91	<0.001
Insect population	cohorts	`peak_adults`	+0.01	[−0.24, +0.26]	+0.19	0.93	0.97	<0.001
Schelling	cell lists	`final_happy`	+0.18	[−0.03, +0.39]	+0.47	0.09	0.70	0.001
Schelling	cell lists	`mean_happy`	+0.23	[+0.02, +0.45]	+0.42	0.03	0.70	0.008
Schelling	cell lists	`total_moves`	−0.23	[−0.44, −0.02]	+0.45	0.03	0.58	0.006
Schelling	cell lists	`segregation`	+0.12	[−0.03, +0.28]	+0.70	0.12	0.37	<0.001
Schelling	columns	`final_happy`	+0.17	[−0.03, +0.36]	+0.54	0.09	0.47	<0.001
Schelling	columns	`mean_happy`	+0.15	[−0.03, +0.34]	+0.56	0.11	0.28	<0.001
Schelling	columns	`total_moves`	−0.15	[−0.34, +0.03]	+0.57	0.10	0.28	<0.001
Schelling	columns	`segregation`	+0.24	[+0.08, +0.40]	+0.67	<0.01	0.37	0.001
Boltzmann wealth	columns	`final_gini`	−0.26	[−0.54, +0.01]	+0.04	0.06	0.21	0.046
Boltzmann wealth	columns	`mean_gini`	+0.13	[−0.16, +0.41]	−0.04	0.39	0.47	0.006
Boltzmann wealth	columns	`final_zero_share`	−0.29	[−0.56, −0.03]	+0.09	0.03	0.28	0.065
Boltzmann wealth	columns	`final_max_wealth`	+0.06	[−0.24, +0.35]	−0.08	0.71	1.00	0.002
Sugarscape	cell lists	`final_population`	+0.03	[−0.03, +0.08]	+0.96	0.36	1.00	<0.001
Sugarscape	cell lists	`mean_population`	+0.02	[−0.02, +0.07]	+0.98	0.33	0.82	<0.001
Sugarscape	cell lists	`mean_vision`	+0.00	[−0.03, +0.04]	+0.99	0.82	0.97	<0.001
Sugarscape	cell lists	`mean_metabolism`	+0.02	[−0.02, +0.06]	+0.98	0.39	0.99	<0.001
Sugarscape	cell lists	`mean_wealth`	−0.03	[−0.09, +0.03]	+0.95	0.36	1.00	<0.001
Sugarscape	cell lists	`sugar_left`	−0.02	[−0.10, +0.07]	+0.91	0.71	0.91	<0.001
Sugarscape	columns	`final_population`	−0.01	[−0.06, +0.05]	+0.96	0.76	1.00	<0.001
Sugarscape	columns	`mean_population`	+0.01	[−0.03, +0.05]	+0.98	0.67	0.97	<0.001
Sugarscape	columns	`mean_vision`	−0.01	[−0.04, +0.02]	+0.99	0.65	0.99	<0.001
Sugarscape	columns	`mean_metabolism`	+0.00	[−0.04, +0.05]	+0.97	0.91	1.00	<0.001
Sugarscape	columns	`mean_wealth`	+0.00	[−0.06, +0.06]	+0.95	0.96	0.99	<0.001
Sugarscape	columns	`sugar_left`	+0.01	[−0.07, +0.09]	+0.92	0.85	0.99	<0.001